

中国科学院大学
University of Chinese Academy of Sciences

第2课
对象技术导论

面向对象程序设计
Object-Oriented Programming

说明

- 仅供选修国科大“面向对象程序设计”课程的学生复习功课时参考，**不得用于其它目的的翻印制作。**

- 这里所提供的只是课堂讲授的部分内容，**万不可误认为课件以外的内容都不要求掌握。**

中国科学院大学
University of Chinese Academy of Sciences

课程成绩

- 大作业+课后作业/课堂测验+课堂研讨，**70%**

  大作业及课堂研讨65%（其中课堂研讨35%）

  其他课后作业/课堂测验5%

  要求：认真思考，鼓励交流，按时提交，迟交扣分

- 期末成绩，**30%**

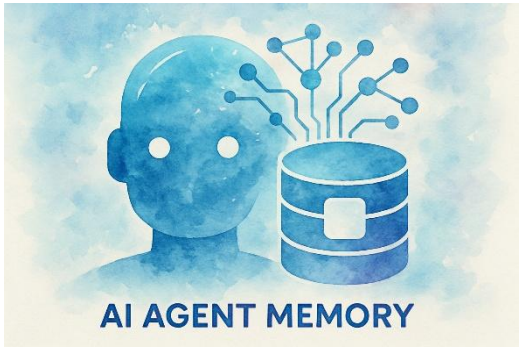
  开卷

中国科学院大学
University of Chinese Academy of Sciences

大作业：大模型辅助的面向对象软件分析、设计与开发

目标开源系统

- AI框架（智能体记忆系统）
- 太空计算（天基云）
- 韧性计算（云原生）
- 其他：学生自荐





大作业内容

- **围绕真实软件系统开展性能优化、需求扩展或模块开发，完成系统理解、需求分析与建模、核心流程与对象设计、方案实现与效果验证，形成完整的系统分析与设计报告，并提交可运行的代码。**

开源系统导师

- 吴国全（博士，软件所研究员）
- 许利杰（博士，软件所研究员）
- 唐　震（博士，软件所高级工程师）



http://www.tcse.cn/~gqwu



http://www.tcse.cn/~xulijie



http://www.tcse.cn/~tangzhen12

大作业：大模型辅助的面向对象软件分析、设计与开发

候选项目1：Ripple消息引擎

服务于国家级金融、电网系统

- **扩展点**：太空计算体验版，内存、CPU、网络等优化重构
- **仓库地址**：
  https://github.com/ISCAS-SSG/Ripple
- **导师**：唐震副研究员

候选项目2：Mandol智能体记忆

指标显著优于国内外开源系统

- **扩展点**：AI Coding增强、融合索引增强、超长记忆增强
- **仓库地址**：
  https://github.com/AgentCombo/Mandol
- **导师**：许利杰研究员

候选项目3：ChaosBlade混沌工程

阿里云社区明星项目

- **扩展点**：智能化混沌生成
- **仓库地址**：
  https://github.com/chaosblade-io/chaosblade-space-exploration
- **导师**：吴国全研究员

候选项目X：自选推荐

院所科研实践 + 代码规模万级 + 面向对象

- **例如**：软件所如意团队、计算所香山团队、…

Ripple：发现并解决问题 | Mandol：理解并扩展系统 | OpenChaos：从需求到模块实现

均需建立在对对象、职责、关系和接口的理解之上，基于面向对象方法完成设计与实现

大作业：大模型辅助的面向对象软件分析、设计与开发



Mandol 智能体记忆系统

2 Mandol 项目

Mandol 面向大模型智能体的记忆组织、存储与检索。本项目侧重需求驱动的系统扩展：学生需要理解现有记忆模型、数据结构和相关接口，分析新需求与已有架构之间的差距，选择合理位置完成扩展，并验证设计与实现的效果。

仓库地址：https://github.com/AgentCombo/Mandol

指导教师：许利杰研究员

联系人：张宇涵 zhangyuhan25@otcaix.iscas.ac.cn

2.1 AI Coding 增强

本方向聚焦面向 Code Agent 的记忆数据结构。代码开发中的记忆涉及代码实体、Issue、修改过程、测试结果和失败尝试等信息，需要支持“某个函数曾出现什么问题”“某次修改为什么被撤销”等访问需求。学生应分析现有记忆表示对这类场景的支持程度，扩展相应的数据结构、关联关系与访问接口。

主要挑战在于如何划分领域对象、关联开发过程与结果，以及处理代码版本变化和历史信息的追溯。要求基于 Mandol 完成可运行的扩展，通过典型代码开发场景验证其有效性，并说明对象设计及接口选择的依据。无需开发完整的 Code Agent。

2.2 融合索引增强

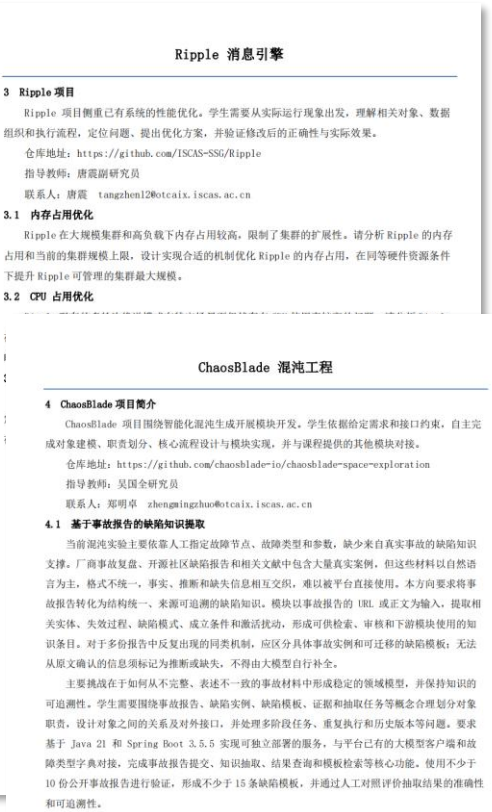
记忆访问可能同时包含语义相似、精确标识、属性约束和关系查询等需求，单一访问方式或简单组合未必能有效支持复杂场景。学生应分析 Mandol 现有索引与检索机制，针对具体需求，扩展多种访问条件或检索方式的协同能力。可以围绕现有索引进行优化、增加新的索引或检索方式，也可以探索更复杂的组合查询，例如“语义相似 + 属性约束 + 关系约束”，支持查询“与给定内容语义相近、发生于某时间之后，并且属于某实体邻居的记忆”。

主要挑战在于如何组织不同查询条件、协调候选结果，以及保证新增能力与现有检索流程的良好衔接。要求完成与现有系统衔接的实现，并比较检索效果、访问开销及维护成本。具体采用索引优化、新增索引、结构融合、协同查询或其他方案，由学生依据分析和实验确定。

2.3 超长记忆增强

本方向聚焦长期运行场景下的记忆内容管理。随着智能体持续运行，记忆不断积累，系统需要兼顾资源开销、检索效率与有效信息的保留。学生应分析 Mandol 现有记忆组织方式，针对长期记忆中的冗余、重复或低价值信息，设计并实现相应的管理策略。

主要挑战在于如何判断不同记忆的信息价值，以及权衡记忆规模、检索效果和资源开销。要求基于现有系统完成扩展，并通过实验验证所采用策略的效果与局限。具体方案由学生确定，可以探索记忆筛选、合并、摘要或其他长期记忆管理方式。



Ripple 消息引擎

3 Ripple 项目

Ripple 项目侧重已有系统的性能优化。学生需要从实际运行现象出发，理解相关对象、数据组织和执行流程，定位问题、提出优化方案，并验证修改后的正确性与实际效果。

仓库地址：https://github.com/ISCAS-SSG/Ripple

指导教师：唐震崩研究员

联系人：唐震 tangzhen12@otcaix.iscas.ac.cn

3.1 内存占用优化

Ripple 在大规模集群和高负载下内存占用较高，限制了集群的扩展性。请分析 Ripple 的内存占用和当前的集群规模上限，设计实现合适的机制优化 Ripple 的内存占用，在同等硬件资源条件下提升 Ripple 可管理的集群最大规模。

3.2 CPU 占用优化



ChaosBlade 混沌工程

4 ChaosBlade 项目简介

ChaosBlade 项目围绕智能化混沌生成开展模块开发。学生依据给定需求和接口约束，自主完成对象建模、职责划分、核心流程设计与模块实现，并与课程提供的其他模块对接。

仓库地址：https://github.com/chaosblade-io/chaosblade-space-exploration

指导教师：吴国全研究员

联系人：郑明卓 zhengmingzhuo@otcaix.iscas.ac.cn

4.1 基于事故报告的缺陷知识提取

当前混沌实验主要依靠人工指定故障节点、故障类型和参数，缺少来自真实事故的缺陷知识支撑。厂商事故复盘、开源社区缺陷报告和相关文献中包含大量真实案例，但这些材料以自然语言为主，格式不统一，事实、推断和缺失信息相互交织，难以被平台直接使用。本方向要求将事故报告转化为结构统一、来源可追溯的缺陷知识。模块以事故报告的 URL 或正文为输入，提取相关实体、失效过程、缺陷模式、成立条件和激活扰动，形成可供检索、审核和下游模块使用的知识条目。对于多份报告中反复出现的同类机制，应区分具体事故实例和可迁移的缺陷模板；无法从原文确认的信息项标记为推断或缺失，不得由大模型自行补全。

主要挑战在于如何从不完整、表述不一致的事故材料中形成稳定的领域模型，并保持知识的可追溯性。学生需要围绕事故报告、缺陷实例、缺陷模板、证据和抽取任务等概念合理划分对象职责，设计对象之间的关系及对外接口，并处理多阶段任务、重复执行和历史版本等问题。要求基于 Java 21 和 Spring Boot 3.5.5 实现可独立部署的服务，与平台已有的大模型客户端和故障类型字典对接，完成事故报告提交、知识抽取、结果查询和模板检索等核心功能。使用不少于 10 份公开事故报告进行验证，形成不少于 15 条缺陷模板，并通过人工对照评价抽取结果的准确性和可追溯性。

4.2 基于异构系统的缺陷迁移

缺陷知识库中的模板通常来源于其他组织的生产系统，而不同系统在服务命名、技术栈、配置方式和运行指标上存在明显差异，因此无法直接依据表层名称或原事故参数完成迁移。平台现有实验主要检查指标是否越过阈值，难以进一步判断被测系统是否存在某类已知缺陷，也无法区分“系统不存在该缺陷”和“当前扰动未能触发该缺陷”。本方向要求将缺陷模板与被测系统的拓扑、配置和代码信息进行匹配，识别可能存在的风险位置，并根据目标系统自身条件生成可执行的混沌实验。模块需要形成系统画像，将模板中的因果角色和成立条件映射到具体系统对象，生成符合 ChaosBlade 规范的故障场景，并结合实验结果判断目标机制是否被激活、预期后果是否出现以及结论是否具有充分证据。

主要挑战在于如何跨越系统和技术栈差异建立稳定的映射关系，以及如何表达无法判断、执行器不支持或证据不足等非确定性结果。需要围绕系统画像、缺陷模板、匹配结果、故障场景和验证结论等核心概念完成对象建模，明确各对象的职责、关系、状态和接口；技术栈差异应通过可扩展的适配机制处理，使新增技术栈不会破坏已有匹配逻辑。要求基于平台现有技术栈实现可独立部署的服务，通过公开接口读取和写入平台数据，生成可由故障调度服务直接执行的实验配置。使用一个目标系统和不少于 5 条缺陷模板完成验证，比较自动匹配结果与人工判断，并报告匹配准确性、无法判定情况及实验结论。至少完成一组包含修改前后对照的实验，用于说明模块能否区分缺陷存在、未被触发和已被修复等不同情况。

4.2 参考链接与实验环境

参考事故报告链接：

1. AWS DynamoDB，2015-09-20 —— 元数据服务过载后重试放大，成员表请求超时又触发更多重试。
https://aws.amazon.com/message/5467D2/
2. AWS Kinesis，2020-11-25 —— 扩容使前端机线程数越过操作系统上限，缓存重建全量失败
https://aws.amazon.com/message/11201/
3. Roblox，2021-10-28 至 10-31 —— Consul 流式接口与 BoltDB 写放大互相推高负载，持续 73 小时。
https://corp.roblox.com/newsroom/2022/01/roblox-return-to-service-10-28-10-31-2021
4. Discord 语音，2026-03-25 —— 积压使单次处理成本上升，成本上升又加剧积压。
https://discord.com/blog/behind-the-scenes-of-the-3-25-26-voice-outage
5. Cloudflare，2019-07-02 —— 新增 WAF 规则中的正则回溯耗尽边缘 CPU。
https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019/
6. Cloudflare，2025-11-18 —— 机器学习特征文件体积越过预设上限，边缘代理拒绝服务。
https://blog.cloudflare.com/18-november-2025-outage/
7. Google Cloud，2025-06-12 —— 配额策略数据含非预期空字段，全球分发后使配额检查进程崩溃。
https://status.cloud.google.com/incidents/ow5iJPPK96RduMcb1SsW
8. Fastly，2021-06-08 —— 客户配置激活了此前部署中的潜伏缺陷。
https://www.fastly.com/blog/summary-of-june-8-outage
9. Cloudflare，2026-02-20 —— 见原文。
https://blog.cloudflare.com/cloudflare-outage-february-20-2026
10. AWS S3，2017-02-28 —— 运维命令参数输入错误，摘除的服务器数量远超预期。
https://aws.amazon.com/message/41926/
11. AWS us-east-1，2021-12-07 —— 自动扩容触发内部网络客户端异常行为，造成内网拥塞。
https://aws.amazon.com/message/12721/
12. Google Cloud，2019-06-02 —— 配置错误叠加软件缺陷，自动化同时下线多个物理位置的独立集群。
https://status.cloud.google.com/incident/cloud-networking/19009
13. Datadog，2023-03-08 —— 操作系统组件自动更新在多区域同时生效，节点网络中断。
https://www.datadoghq.com/blog/2023-03-08-multiregion-infrastructure-connectivity-issue/
14. GitHub 可用性月报，2026 年 7 月 —— 一份月报含多起独立事故，适合做同月对比。
https://github.blog/news-insights/company-news/github-availability-report-july-2026/
15. GitHub，2018-10-21 —— 网络分区后 MySQL 拓扑切换导致跨站点写入分叉。
https://github.blog/2018-10-30-oct21-post-incident-analysis/
16. GitLab，2017-01-31 —— 误删生产数据目录，五种备份子段均未生效。
https://about.gitlab.com/blog/2017/02/10/postmortem-of-database-outage-of-january-31/

大作业具体内容已发布在课程网站和课程群

大作业：大家关心的问题（1/2）

1. 如何选题：二选一

① **推荐项目：**课程提供开源项目建议清单，给定具体扩展要求

② **自荐项目：**学生参与的研究所科研实践任务，自荐扩展实现（源码规模达到万级）

2. 是否组队：不组队，以个人为单位

3. 作业形式：

① **报告 - 系统扩展的分析与设计报告**

② **代码 - 系统扩展的源代码实现**

③ **汇报 - 课堂研讨：按学号、每人2次（扩展点需求分析研讨、结题研讨）**

4. 作业提交地址：提交至Gitee等源码托管平台的个人repo

5. 如何评分：

① 报告的质量

② 扩展点能否正常运行，是否有配套的测试代码，扩展后的实际效果（测试对比、社区贡献等）

③ 课堂研讨表现占比35%

根据国科大相关要求，《报告》应对大模型使用情况进行标注和说明，将作为大作业评分关键依据

大作业：大家关心的问题（2/2）

6. 进度安排：

第2周，授课团队给出开源项目及方向候选清单

第3周，学生确定选题，需要具体到开源项目或特定科研实践任务的具体方向

第8、9周，课堂研讨（问题分析与方案研讨），提交系统分析与设计报告（初版）

- **大模型辅助：**利用大模型辅助理解目标系统的代码结构、核心对象与执行流程，结合具体任务开展**问题或需求分析、对象建模与初步方案设计**；对大模型生成的分析结果进行**验证和修正**，形成自己的**工程判断**。
- **侧重点包括：**性能现象、问题定位、支撑证据、需求与架构矛盾、扩展定位、需求理解、对象划分...
- **课堂研讨：**主要围绕“问题或需求分析与初步方案”进行 PPT 分享和讨论。

第13周，修订并提交系统分析与设计报告（第二版）

- **验证并判断**大模型方案，重点识别其中与真实系统不一致或不合理的部分。
- 形成初步实现。记录方案**迭代过程**，包括尝试过的不同方案、失败的思路及其原因，并结合实际结果对原有判断进行**修正**。

第15-18周，课堂研讨（结题研讨），对目标系统及本学期开发工作进行整体汇报

第18周，提交系统分析与设计报告（最终版）及最终代码

中国科学院大学
University of Chinese Academy of Sciences

2026年秋季课程安排

- 第1周，导言
- 第2周，认识软件系统的复杂性、有组织的复杂系统
- 第3周，程序语言、开发环境的发展
- 第4周，9.25中秋假期
- 第5周，10.2国庆假期
- 第6周，建模方法和建模语言
- 第7周，抽象
- 第8周，课堂研讨1：扩展点的需求分析与初步方案
- 第9周，课堂研讨2：扩展点的需求分析与初步方案
- 第10周，封装和模块化
- 第11周，复用与解耦1：层次结构（组合、继承、多态）
- 第12周，复用与解耦2：类型信息、抽象类、接口
- 第13周，持久化-时间
- 第14周，持久化-空间
- 第15周，并发
- 第15周，课堂研讨3：设计模式的优化与重构（择机补课）
- 第16周，课堂研讨4：大作业汇报
- 第16周，课堂研讨5：大作业汇报（择机补课）
- 第17周，课堂研讨6：大作业汇报
- 第17周，课堂研讨7：大作业汇报（择机补课）
- 第18周，1.1元旦假期
- 第19/20周，考试

第2周给出推荐开源项目选题清单

第3周确定选题，需要具体到特定方向

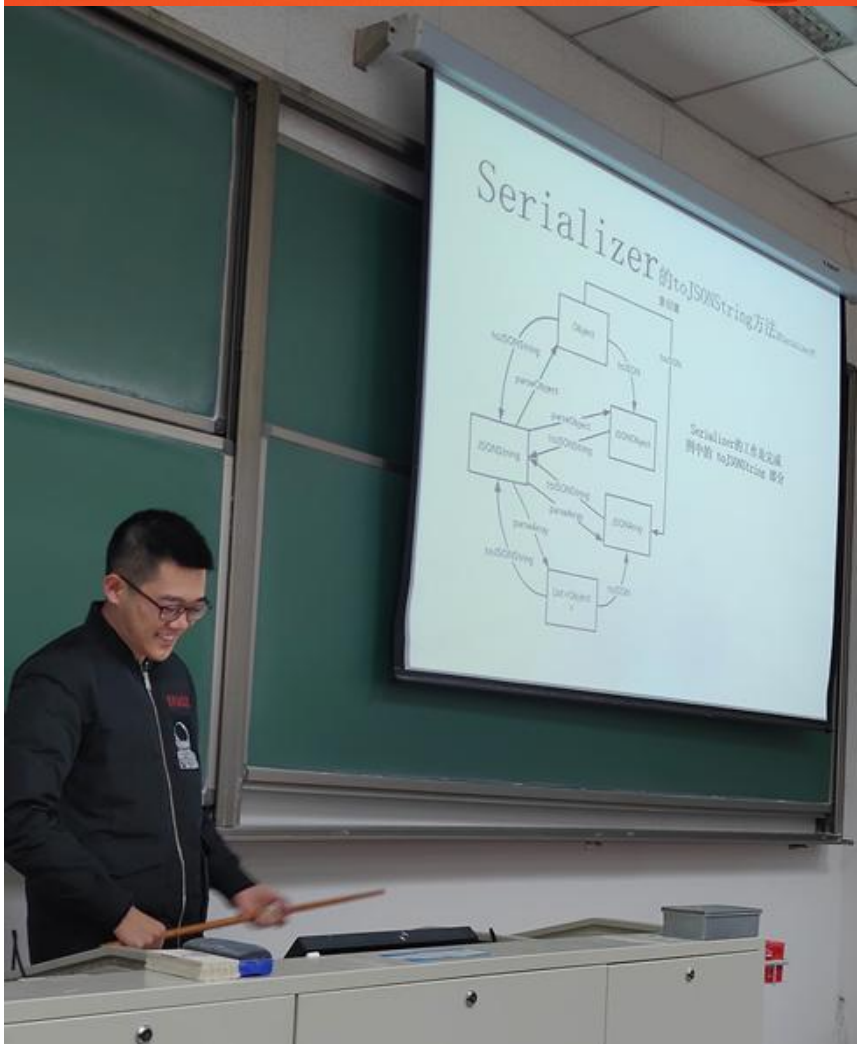
第8周提交大作业报告（初版）

第13周提交大作业报告（第二版）

第18周提交大作业报告（第三版），以及扩展代码

中国科学院大学 University of Chinese Academy of Sciences





fastjson 学习报告，雷正宇

https://842376130.gitbook.io/fastjson-learning-report/



fastjson 学习报告

Edit on GitHub

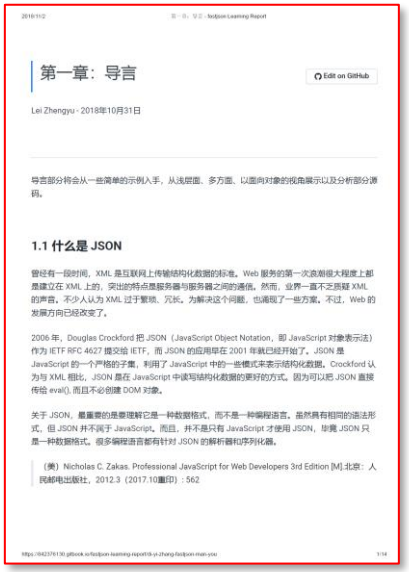
Lei Zhengyu - 2018年10月31日

fastjson

声明

欢迎任课老师和助教以及其他来自互联网的读者阅读此报告。我于 2018 年秋季加入国科大"面向对象程序设计"课程，并选择 fastjson 作为源码学习环节的目标项目。此文即为我的源码学习报告。由于这是面向对象课程的作业，所以和算法相关的内容不是解析的重点。

由于时间关系，我会从 2018 年 10 月 31 日起陆续上传并更新此报告。每一次更新除了会增加新的章节，还会修改过去的内容。鉴于整个报告只分为三个周期完成，所以每一次更新时我对 fastjson 的理解必定会有很大的不同，修改的内容也会非常多，一般不会注明哪些内容已经被修改或更正，批改作业的老师只需将每一次报告单独作为阶段作业提交的内容批改即可。望老师们谅解。



第一章：导言

Edit on GitHub

Lei Zhengyu - 2018年10月31日

导言部分将会从一些简单的示例入手，从浅层面、多方面、以面向对象的视角展示以及分析部分源码。

1.1 什么是 JSON

曾经有一段时间，XML 是互联网上传输结构化数据的标准。Web 服务的第一次浪潮很大程度上都是建立在 XML 上的，突出的特点是服务器与服务器之间的通信。然而，业界一直不乏质疑 XML 的声音，不少人认为 XML 过于繁琐、冗长。为解决这个问题，也涌现了一些方案，不过，Web 的发展方向已经改变了。

2006 年，Douglas Crockford 把 JSON（JavaScript Object Notation，即 JavaScript 对象表示法）作为 IETF RFC 4627 提交给 IETF，而 JSON 的应用早在 2001 年就已经开始了。JSON 是 JavaScript 的一个严格的子集，利用了 JavaScript 中的一些模式来表示结构化数据。Crockford 认为与 XML 相比，JSON 是在 JavaScript 中读写结构化数据的更好的方式。因为可以把 JSON 直接传给 eval()，而且不必创建 DOM 对象。

关于 JSON，最重要的是要理解它是一种数据格式，而不是一种编程语言。虽然具有相同的语法形式，但 JSON 并不属于 JavaScript。而且，并不是只有 JavaScript 才使用 JSON，毕竟 JSON 只是一种数据格式。很多编程语言都有针对 JSON 的解析器和序列化器。

（美）Nicholas C. Zakas. Professional JavaScript for Web Developers 3rd Edition [M].北京：人民邮电出版社，2012.3（2017.10重印）：562



1.3 从简单示例漫游 fastjson

一个针对 Java 的 JSON 解析框架最基本的功能是解析和序列化。下面示例定义了一个简单的复合 JSON Object 语法的 Java 字符串：

```
String jobjstr = "{" +
    "\"name\": \"John\"," +
    "\"age\": 18" +
    "}";
```

由于 JSON 本身有很好的可读性，所以这里不再对字符串的内容加以解释。现在我们使用 fastjson 框架，用 parse 方法将上述字符串解析为 Java 对象：

```
JSONObject jobj = JSONObject.parseObject(jobjstr);
System.out.println("name: " + jobj.getString("name") +
    ", age: " + jobj.getInteger("age"));
```

这段测试程序输出的结果是：

```
name: John, age: 18
```

现在将之前得到的 JSONObject 对象转化为字符串：

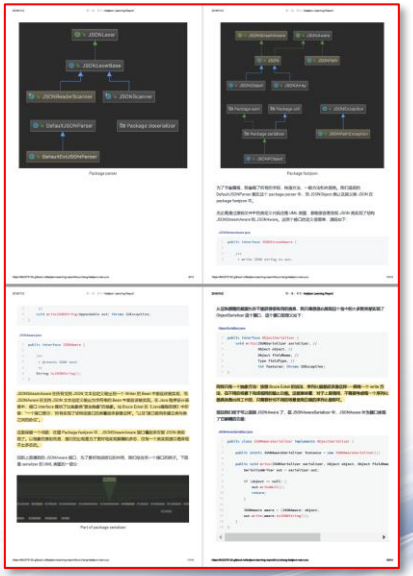
```
String str = jobj.toJSONString();
System.out.println(str);
```

输出结果为：

```
{"name":"John","age":18}
```

现在我们关注解析字符串的关键语句：

```
JSONObject jobj = JSONObject.parseObject(jobjstr);
```



中国科学院大学
University of Chinese Academy of Sciences



fastjson 学习报告，曾鸿斌

https://sbzeng.gitbook.io/fastjson/



Fastjson简析

这是笔者在中国科学院大学上王伟老师的面向对象程序设计一课时的作业要求——对Fastjson进行分析以考察面向对象思想在其中的应用。

因此，本文的重点不是Fastjson的介绍与分析，也不是具体源码实现的解析与算法设计的讨论，而是对需求建模、主要功能流程设计、类及类间关系、面向对象设计原则及设计模式的分析。总而言之，是对面向对象思想的探讨。

因此如果读者感兴趣的是Fastjson的实现解析，可以参照商宗海的fastjson-source-code-analysis，这篇文章提供了源码的解析。读者在阅读过程中也可以参照他的fastjson分支，其中添加了大量的注释，一定程度上解决了Fastjson的代码中注释过少带来的阅读不便。

如果读者在阅读过程中有任何意见或者建议，可以发邮件与我联系，我的邮箱是：zenghongbin16@mails.ucas.ac.cn。如果读者对此感兴趣，也很欢迎各位的加入，您同样可以通过邮件联系我。

此外，每节的开头都有主要内容的介绍，希望这可以帮助大家抓住重点。

曾鸿斌

2019.1.9

Next
一. Fastjson简介

Last updated 9 months ago

学习目标

- 对象技术导论
  # 认识软件系统的复杂性：为什么软件系统是复杂的？复杂系统有什么特点
  # 有组织的复杂系统
  # 控制软件系统的复杂性

- Java程序语言基础
  # 基本数据类型、运算符、流程控制
  # 类和对象
  # 类的组成，如何创建/操作它的对象

中国科学院大学
University of Chinese Academy of Sciences



对象技术导论（一）

认识软件系统的复杂性



中国科学院大学
University of Chinese Academy of Sciences

认识软件系统的复杂性

为什么软件系统是复杂的?

我们更关注“工业级”的软件系统

- “不复杂”的软件

  # 由编程新手或独立程序员提出、构建、维护和使用的软件系统

  # 系统的生命周期短，可以轻易被抛弃或被代替，不必尝试去修复Bug、升级功能

- “工业级”软件

  # 表现出丰富行为，反馈式系统，由真实世界的事件驱动，或发出驱动事件

  # 维护着海量规模信息记录的完整性

  # 允许大量并发的更新和查询

  # 系统可以控制真实世界的实体

  # 具有很长的生命周期

  # 随着时间的推移，用户逐渐依赖于这些软件系统的正常工作

  ...

中国科学院大学 University of Chinese Academy of Sciences

为什么软件在本质上是复杂的?

1. 问题域的复杂性

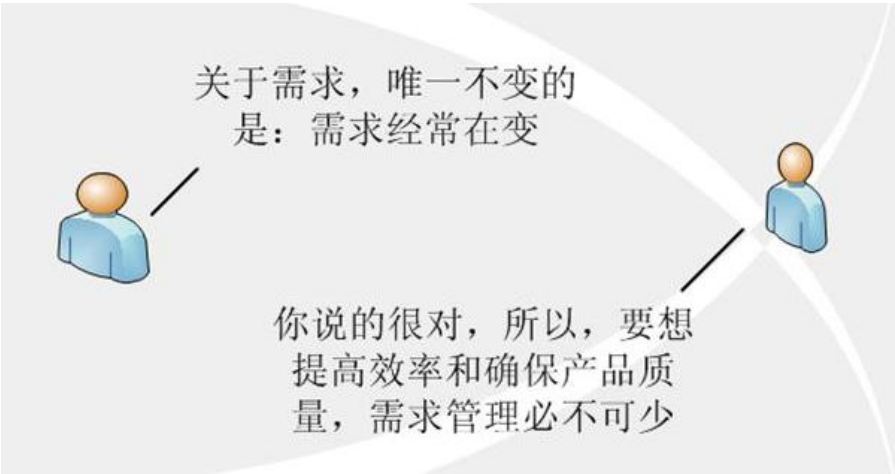
领域自身的复杂性（卫星导航系统、交通指挥系统、气象监测系统、电子交易系统…）

功能要求、非功能要求（性能、安全、可靠性、可用性…）

软件系统用户和开发者之间的沟通问题

需求经常变更（软件改变了用户“问题域”的行为规则，进而带来更好的需求理解和表述）





为什么软件在本质上是复杂的?

2. 管理开发过程的困难性

代码管理问题（几十年前，数千行的汇编程序；现在，高级语言编写的系统代码规模达到百万、千万行）

成百上千的独立模块、第三方程序包…

开发团队的沟通、协作问题，如何维持设计的一致性和完整性？



中国科学院大学
University of Chinese Academy of Sciences

为什么软件在本质上是复杂的？ cont.

3. 软件中随处可能出现的灵活性

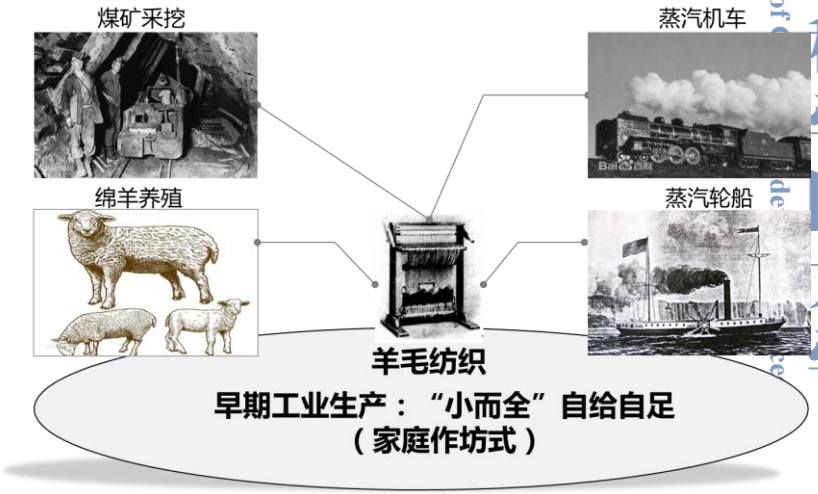
两个世纪前，英国的羊毛纺织厂，养羊、挖煤、修铁路，一个都不能少！

软件行业，这种情况依然存在！

软件开发人员可以表达任何形式的抽象，这种灵活性**“诱使”**开发人员打造几乎所有的初级构建模块

缺少品质规范和标准

软件行业至今仍是劳动密集型产业！





为什么软件在本质上是复杂的？ cont.

4. 描述离散系统行为的问题

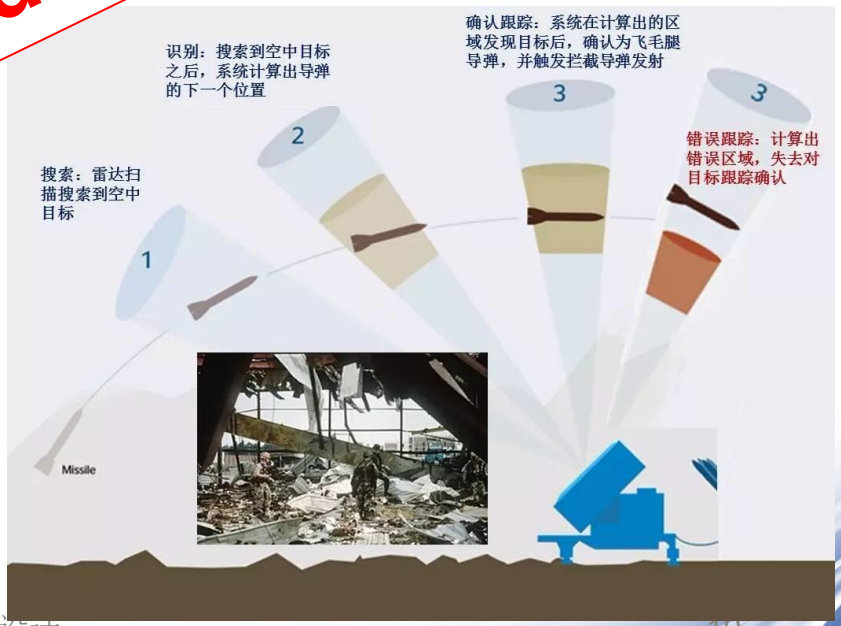
大型软件系统中，存在大量的程序变量、控制线程[illegible]这些变量值、地址、调用栈…等等构成了软件系统的[illegible]统的离散状态，即系统状态的变化不能够用连续[illegible]

因此，系统外部的输入、[illegible]能导致系统状态的非预期变化

宰赫[illegible]

在1991[illegible]发伊拉克发射的飞毛[illegible]地阿拉伯的宰赫兰基地，当[illegible]兵，炸伤100多人，造成美军海湾战[illegible]一次伤亡超过百人的损失

调查发现，由于软件系统的数据舍入误差，使基地的爱国者反导弹系统失效，未能在空中拦截飞毛腿导弹



Software Defined Everything?
Software Damaged Everything?

为什么软件在本质上是复杂的？ cont.

- 《没有银弹》[2]

  1987年，IBM大型机之父Brooks发表在《Computer》上的软件工程经典论文里指出：

  “爱因斯坦认为自然界必定存在着简单的解释，因为造物主不是反复无常或者随心所欲。但是，软件工程师没有这样的信仰来提供安慰。许多必须控制的复杂性是随心所欲的复杂性。”

  “Einstein argued that there must be simplified explanations of nature, because God is not capricious or arbitrary. No such faith comforts the software engineer. Much of the complexity that he must master is arbitrary complexity, forced without rhyme or reason by the many human institutions and systems to which his interfaces must conform. These differ from interface to interface, and from time to time, not because of necessity but only because they were designed by different people, rather than by God.”





[2] Brooks F P. No Silver Bullet Essence and Accidents of Software Engineering. Computer, 1987, 20(4):10-19
[中文相关材料] http://www.cnblogs.com/cute/archive/2012/09/27/2705253.html.

“系统越复杂，就越容易全面崩溃”

- 建筑师不会想要在一幢建成的100层大楼里增加一层地下停车场，成本是个大问题，结构风险更是致命的

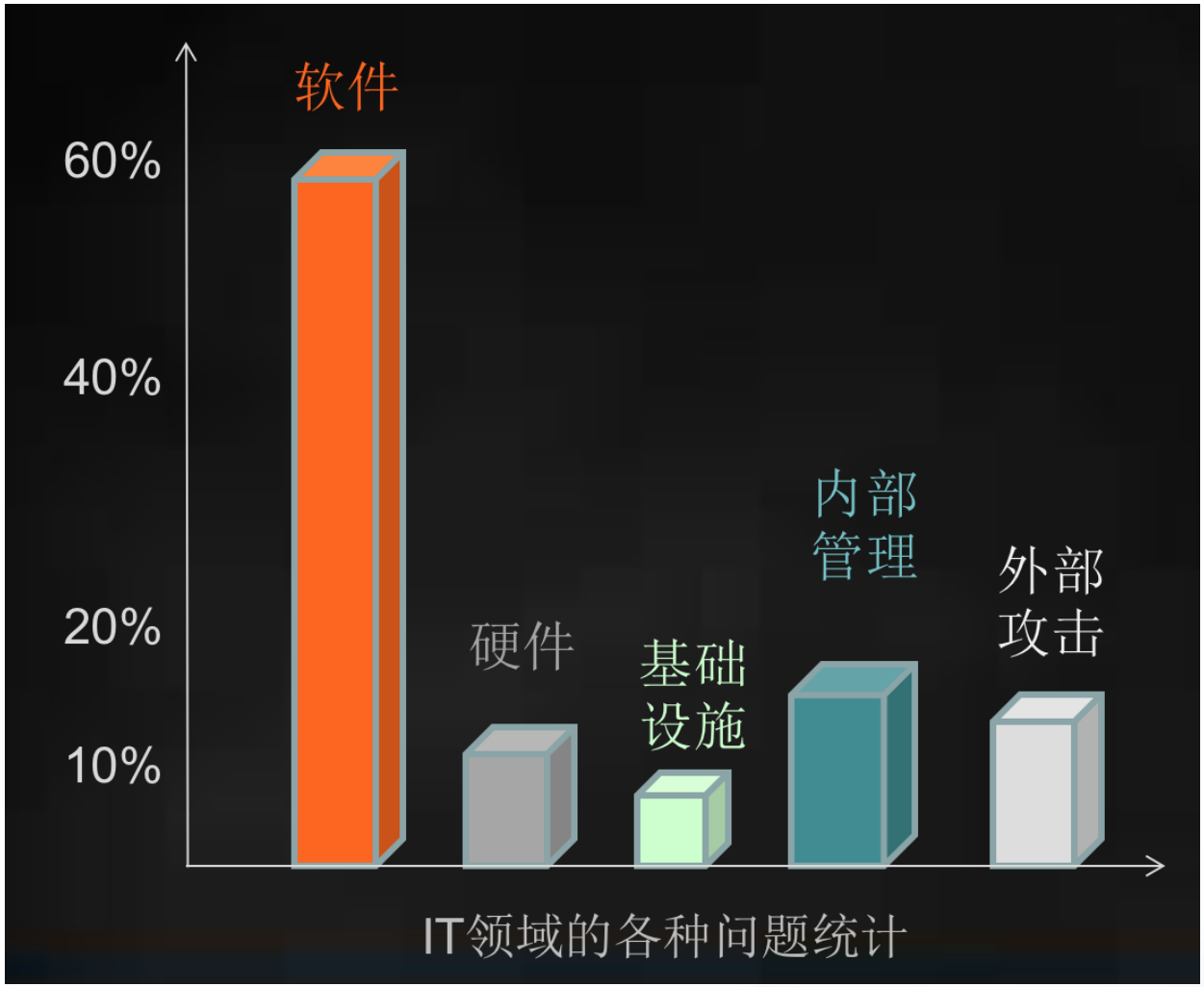


2009年2月9日

烧再烂也得留着…



我们更关注“工业级”的软件系统



“系统越复杂，就越容易全面崩溃”

- 阿丽亚娜五号运载火箭由于软件系统缺陷而导致安全事故

首次测试发射在1996年6月4日，发射后37秒自身毁灭，原因是控制火箭飞行的软件故障

具体原因是由于火箭导航系统SRI的超负荷运作所致，而这套导航系统则是工程师从4号火箭上原封不动地照搬而来…但是，5号火箭的速度是4号火箭的5倍，导航系统SRI计算的数据在4号火箭上从未出现过

火箭点火升空后测算所得的速度以64位浮点数值表示，导航系统SRI无法将其转换为16位的整数值，最终导致系统的内存溢出，并复写了部分数据，指引火箭偏离既定轨道、最终爆炸



视频地址
http://v.ku6.com/show/IfO3WygujSbTA4P9fa55qw...html

“系统越复杂，就越容易全面崩溃”

- 程序错误和漏算方位角，一箭十九星成为太空垃圾





2017年11月28日，俄罗斯航天集团从位于俄远东地区的东方发射场发射了一枚携有19颗卫星的“联盟－2.1B”运载火箭，但火箭未能将这些卫星送入预定轨道。这些卫星来自俄罗斯、加拿大、挪威、日本、德国、美国等国，其中约半数没有航天发射保险。

俄航天集团总经理科马罗夫指出，分析事故原因后，发现了此前从未被研究过的问题，即佛盖特上面级控制系统软件算法的特殊性。

该算法是大约20年前制定出来的，有62次成功发射记录，但此前佛盖特上面级都是从别的航天发射场升空的。他指出，“联盟号火箭-佛盖特上面级-东方发射场”这样的搭配还是首次出现。

“系统越复杂，就越容易全面崩溃”

- 波音737MAX发生两起空难

2018年10月29日，印尼狮航一架波音737Max飞机坠毁…

2019年3月10日，埃塞俄比亚航空公司一架波音737Max飞机坠毁…

波音公司承认**在这两起空难中MCAS均被迎角传感器的错误数据激活，并同驾驶员争夺飞机的控制权，导致飞机最终失控**

波音公司随后又承认737Max飞机的软件系统还存在一个“相对较小”、但对飞行安全存在重要影响的问题，该问题会影响到飞机襟翼和其他保持飞行稳定的硬件，FAA已将该问题对于飞行安全的影响列为“紧急”项，并责令波音修复





“系统越复杂，就越容易全面崩溃”

- F-35研制项目延期、预算超出

美国《每日防务》2004年2月23日报道：…除超重原因外，项目推迟还因存在一些其他问题。例如，航空电子软件的进程也较计划滞后大约 6 -8周。 F -35项目现在已为软件开发争取了更多预算资金，项目延期一年为适应航电软件的滞后提供了额外的时间。…按照原始进度，首批F-35应在2010年具备初步作战能力…

F-35战机研发可能再超支延期 多国订单难保，2010年

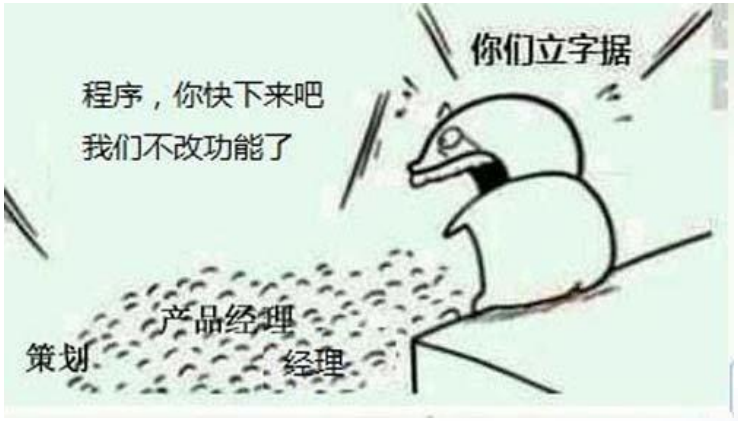
澳大利亚和加拿大对F-35交付延期表示关注，2011年

F-35战机交货延期大幅涨价 日本恐难以承受，2012年

美五角大楼强制文职人员休假或造成F-35项目延期，2013年

F-35软件测试延期导致费用增加 影响战力形成，2014年

F-35战机进行舰上作战测试，2015年



正在交付给海军陆战队的F-35 Block 2B版本的软件依然充斥着大量Bug，2016年

“系统越复杂，就越容易全面崩溃”

2025年1月28日美国空军F-35A阿拉斯加坠毁事故

1、结冰使起落架上的机轮负重传感器(WoW)错误判定飞机已着陆，飞行控制系统自动切换至“地面模式”。此时飞机实际仍在空中，丧失空中操控能力，最终失控坠毁。

2、F-35的自动地面操作模式被指出存在逻辑漏洞:系统仅依赖传感器信号切换飞行控制律，未设置空中/地面状态双重验证机制。



2018年9月28日，F-35B首度发生坠毁事件

2018年9月28日，美国海军陆战队的一架F-35B战斗机在南卡罗来纳州的贝尔福特附近发生坠毁。这架F-35B是垂直/短起降型，执行训练任务时发生了机械故障，飞行员成功弹射。事故发生后，空军及海军陆战队对F-35B的飞行控制系统进行了详细检查，并对该型飞机的设计进行了一些调整。尽管该事件未造成任何飞行员伤亡，但它显现出F-35系列飞机可能存在的问题，特别是在复杂任务中的可靠性。

2019年4月9日，F-35A撞击跑道

2019年4月9日，一架F-35A战斗机在犹他州的一次训练飞行中坠毁，飞机撞击了跑道并造成了严重损毁。飞行员成功弹射，并在事故中毫发无伤。调查显示，事故原因与飞机的飞行控制系统出现异常，导致飞机在着陆前发生了剧烈的横向滚转。虽然该事件并未导致飞机完全摧毁，但它再次强调了F-35系列战斗机的系统可靠性问题，尤其是在极端操作条件下。

2020年5月19日，F-35A与引擎故障

2020年5月19日，美国空军一架F-35A在空中发生了引擎故障。飞行员在执行正常飞行任务时，突然发现飞机的发动机出现问题。幸运的是，飞行员成功弹射并安全着陆。虽然事故没有导致飞机完全摧毁，但这一事件引发了空军对F-35A引擎及其相关系统的安全性审查。

2021年5月19日，F-35B引擎故障

2021年5月19日，一架美国海军的F-35B在演习过程中发生了引擎故障，导致飞行员弹射并安全逃生。该事件发生在南太平洋某个训练场地附近。尽管飞行员安全逃生，但事件导致该型飞机在后续检查中发现了引擎系统存在的潜在隐患。事故调查显示，F-35B的发动机在高温条件下存在一些可靠性问题，这些问题可能导致突然的发动机停转。

“系统越复杂，就越容易全面崩溃”

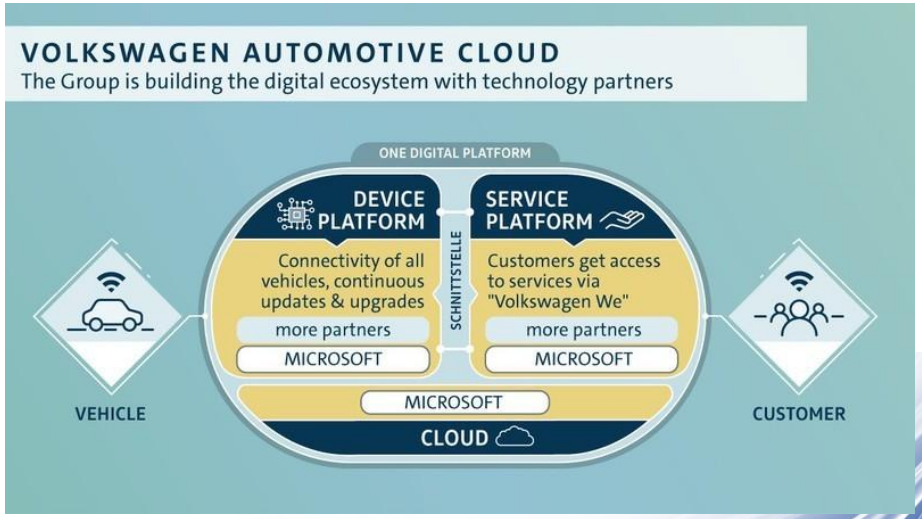
- 大众汽车帝国的软件危机

大众汽车的数字化转型到了“生死时刻”，智能电动车因饱受软件Bug困扰，一再延迟上市

已经生产出来的近2万辆大众全新一代电动车型ID.3，不能被交付给热情的用户（原计划2019年底交付，交付目标已多次变更，延期至2020年8月底），塞满了临时租来的停车场，像一堆破铜烂铁。这些车子，只有在工程师们手工往这些车上灌装软件之后，才能被开上马路和投放到市场。尴尬的是，软件还在开发的过程中。

由于软件研发上层出不穷的bug所带来的交付延迟，带来的冲击是大众汽车集团2020年雄心勃勃的电动车产销计划一再缩水。后果相当严重，如果不能产出足够多的纯电动汽车，在2020年开始实施的欧盟新汽车排放法规之下，大众汽车最高将会被课以100亿欧元的罚款。





【来源】https://mp.weixin.qq.com/s/nlTfv2RLodRHeqOwKCbN4g

“系统越复杂，就越容易全面崩溃”

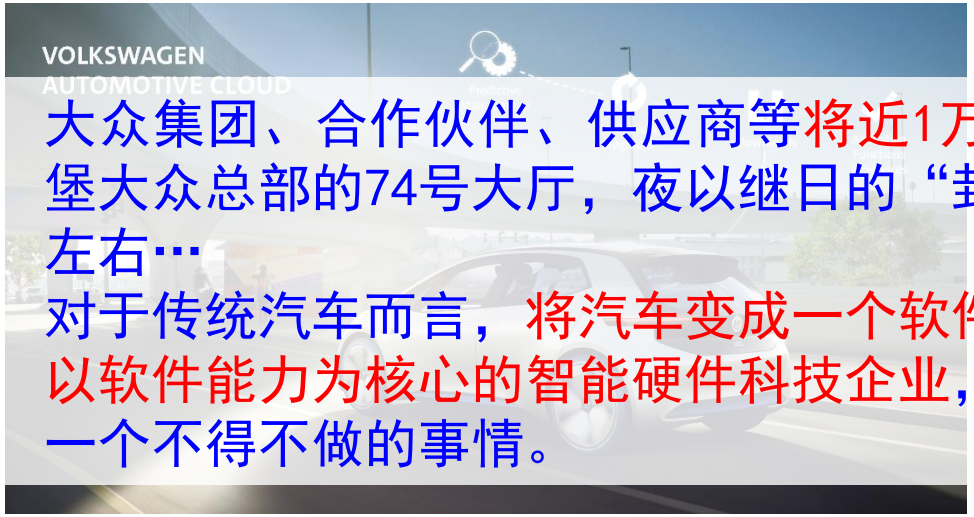
- 大众汽车帝国的软件危机

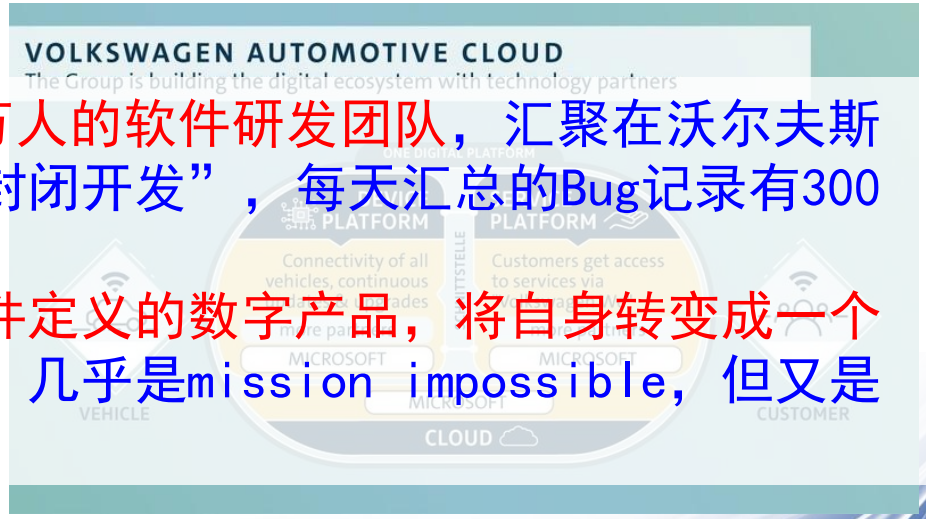
传统汽车→软件定义汽车

1、碎片化的70多个独立电子控制单元（ECU）→ 简化为3个集中式的域控制器（辅助驾驶控制、整车控制、娱乐控制）

2、开发人员面向70多个独立ECU操作系统开发 → 开发人员面向vw.OS开发，vw.OS统一管理整车所有芯片、内存、传感器、输入输出，所有上层应用程序由其管理和调度

3、vw Automotive Cloud，类似APP Store





大众集团、合作伙伴、供应商等将近1万人的软件研发团队，汇聚在沃尔夫斯堡大众总部的74号大厅，夜以继日的“封闭开发”，每天汇总的Bug记录有300左右…

对于传统汽车而言，将汽车变成一个软件定义的数字产品，将自身转变成一个以软件能力为核心的智能硬件科技企业，几乎是mission impossible，但又是一个不得不做的事情。

【来源】https://mp.weixin.qq.com/s/nlTfv2RLodRHeqOwKCbN4g

“系统越复杂，就越容易全面崩溃”

- 大众汽车帝国的软件危机

vm.OS，新世纪的IBM OS/360？

20世纪60年代，随着计算机用途的增多，人们逐渐意识到为每一台新的计算机重新编写相同功能的应用软件是一件低效的工作。

添加操作系统可以让用户更加方便地用高级编程语言编写程序和管理计算机，应用程序也可以移植到升级后的硬件上，大大方便了应用程序的开发、使用和维护。

前瞻到这一趋势，IBM决定为其第一款多功能大型机S/360开发配备后来软件工程领域颇具传奇色彩的OS/360操作系统。OS/360操作系统的设计和研发是当时计算机软件领域的一个重要突破。它的设计需要兼顾许多未曾尝试的领域，比如为充分利用中央处理器而实现多程序同时运行，兼顾系列中不同配置的机型，以及各种机型和其程序所需的不同文件的读写要求，这使得它的设计复杂性呈指数增长，导致该操作系统直到S/360大型机问世两年后才得以完成。

【来源】操作系统的兴衰，https://dl.ccf.org.cn/institude/institudeDetail?id=4537806934083584&_ack=1

中国科学院大学 University of Chinese Academy of Sciences

“系统越复杂，就越容易全面崩溃”

- 大众汽车帝国的软件危机

vm.OS，新世纪的IBM OS/360？

IBM为了加快进度，一度添加了很多人手到该项目中，但随着开发人员的不断增多，整个工程的进度反而减缓。软件开发与传统的工程开发有所区别，将软件分割成不同模块同时开发，需不断协调和整合。当模块分割过细时，协调不同模块的进度将变得过于复杂，以至于彻底抵消分工所带来的效率。

负责OS/360开发的项目经理是1956年从哈佛大学博士毕业的布鲁克斯。他根据上述观察写出了软件工程领域的经典论文《没有银弹：软件工程中的根本与次要工作》和家喻户晓的《人月神话》一书。

S/360大型机和OS/360操作系统的成功使得IBM牢牢把握住了20世纪70年代美国计算机市场。因为操作系统的软件兼容性，购买了S/360计算机的公司会更加频繁地进行系统升级，为IBM带来的利润甚至超过了计算机本身的销售。IBM OS/360的后续版本一直延续到IBM最新一代大型机zSeries上的z/OS操作系统，而且都实现了对早期版本的兼容。

【来源】操作系统的兴衰，https://dl.ccf.org.cn/institude/institudeDetail?id=4537806934083584&_ack=1

中国科学院大学 University of Chinese Academy of Sciences



对象技术导论（二）

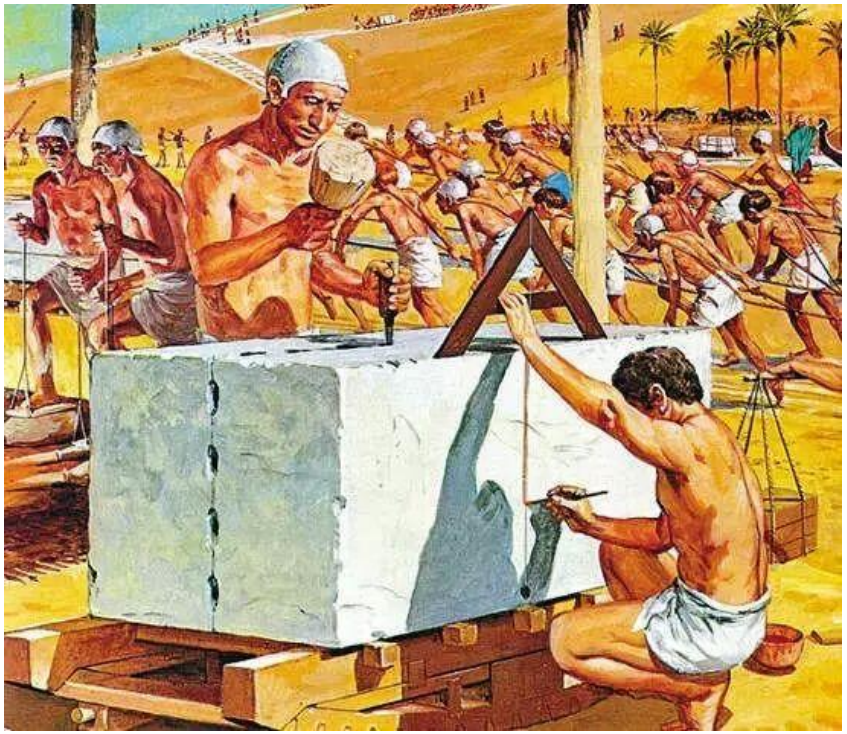
有组织的复杂系统

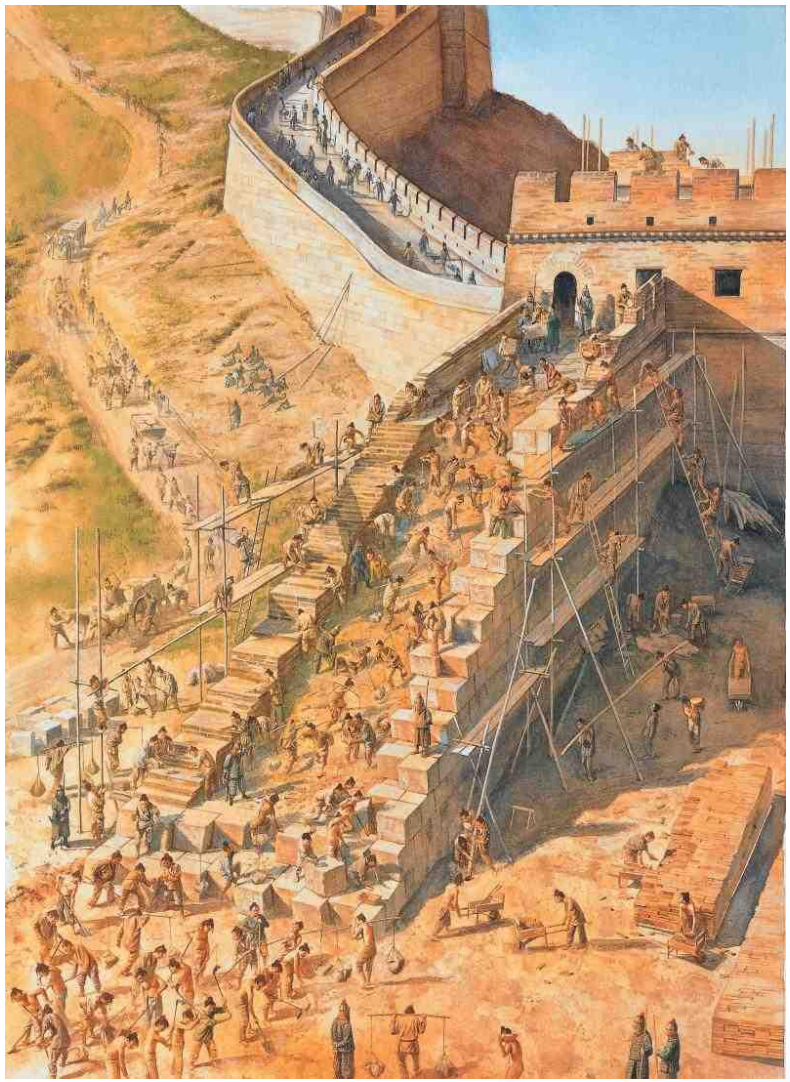


中国科学院大学
University of Chinese Academy of Sciences

控制软件系统的复杂性
来自成熟行业的启示

建筑行业

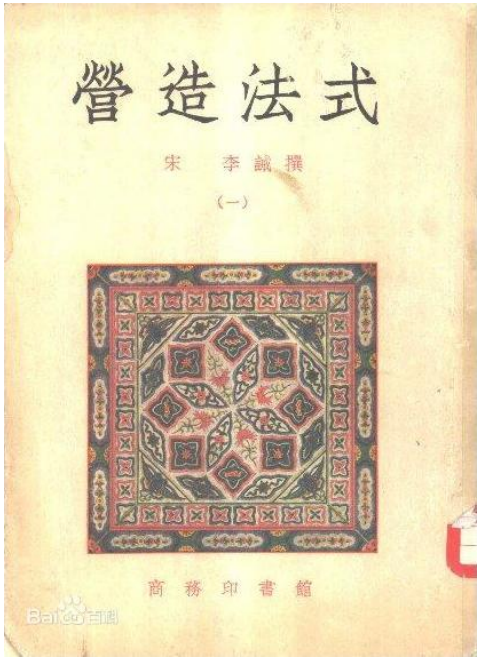


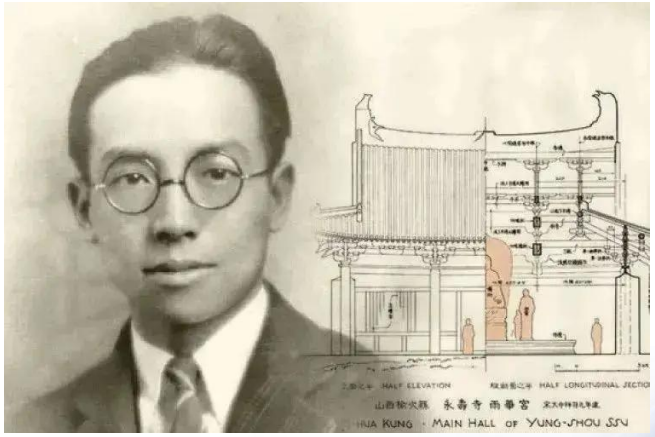


中国科学院大学
University of Chinese Academy of Sciences

建筑行业

卷数	卷名	内容
/	序	为什么修营造法式，其意义在何？ 古建筑的思想渊源和礼制精神； 传统建筑价值理念阐述
/	看详	**建筑理论知识概析** 建筑理论和建筑历史传统依据进行阐述
第1卷 第2卷	总释 总例	**建筑名词解释**：对各种建筑物及构件的名称、条例、术语进行规范并作出统一解释，指出其在各个不同时期的确切叫法
第3卷 ~第15卷	制度	**各工种的施工规范**：所谓制度即每一工种的选材、安装、加工方法（13个工种包括：壕寨、石作、大木作、小木作、雕作、旋作、锯作、竹作、瓦作、泥作、彩画作、砖作、窑作）
第16卷 ~第25卷	功限	**各工种的劳动力定额**： 即每一个工作，根据其实际要求需要用多少人力，如何计算工钱
第26卷 ~第28卷	料例	**各工种的用料定额**： 即每一个工作，根据其实际要求需要采买多少材料
第29卷 ~第34卷	图样	**施工中涉及到的图例**： 各种木制构件、屋架、雕刻、彩画、装修等或黑白、或彩色的图样

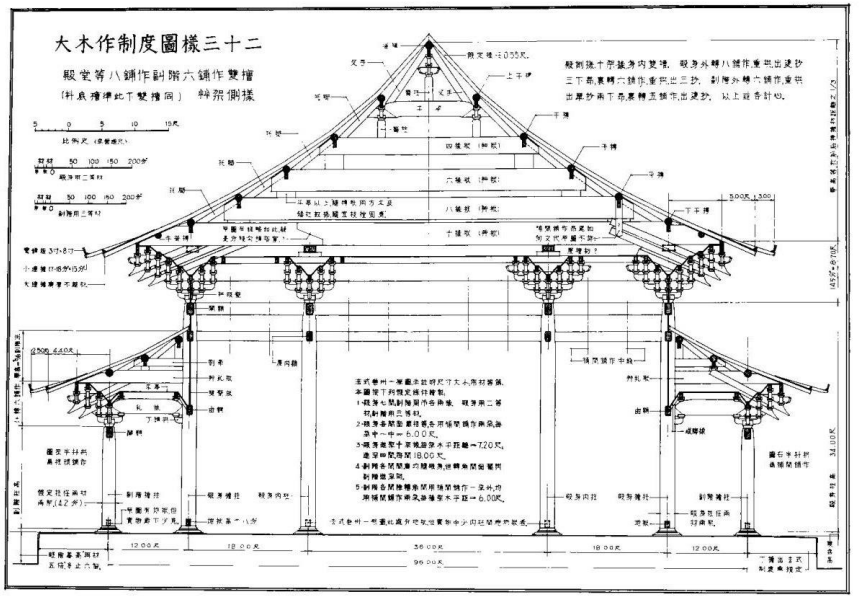


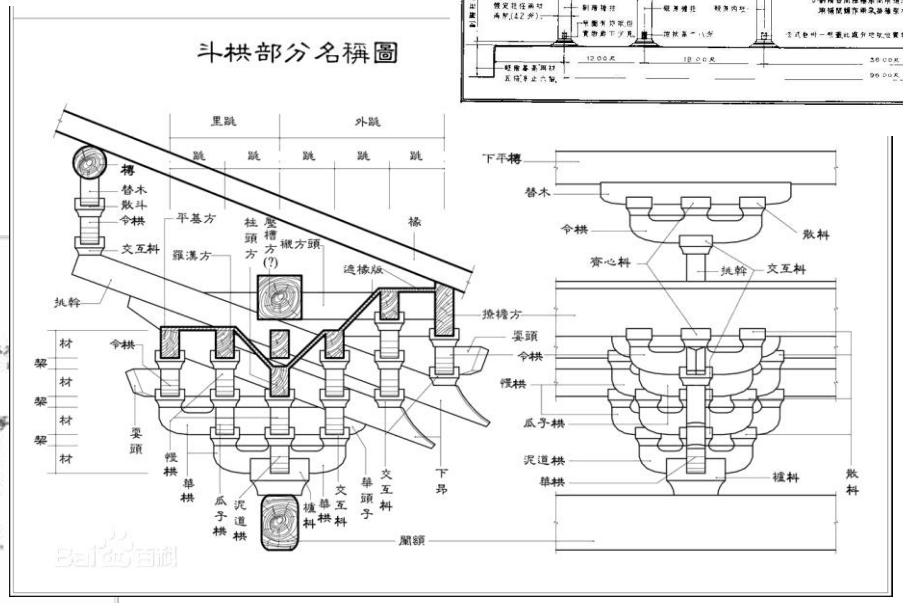


建筑行业

“凡构屋之制，皆以材为祖”

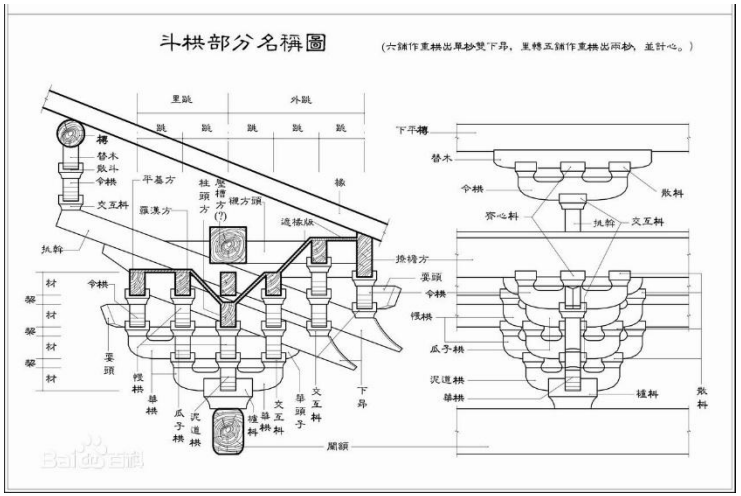
——宋·李诫

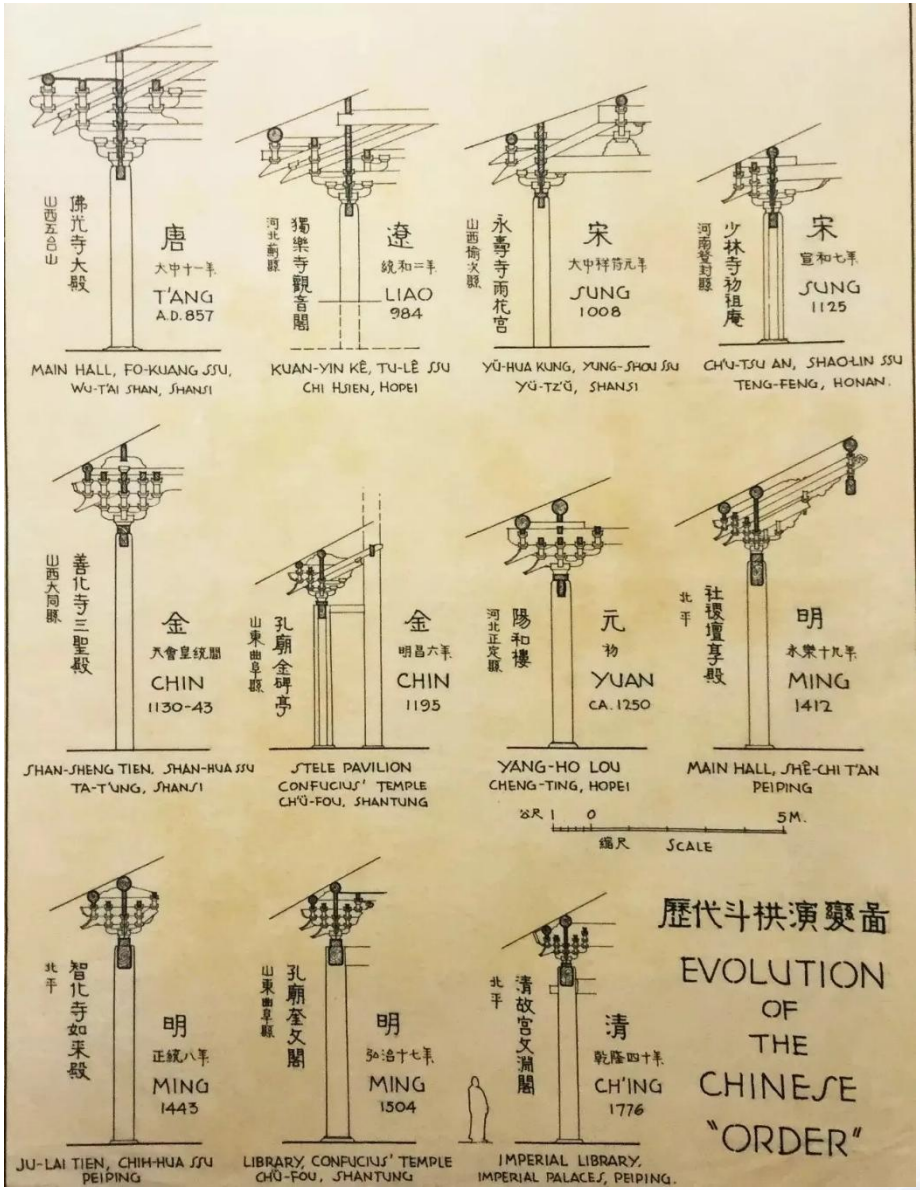






建筑行业


斗栱部分名稱圖
(六鋪作重栱出單杪單下昂，里轉五鋪作重栱出兩杪，並計心。)
里跳
外跳
跳
令栱
交互斗
平基方
櫨枓
柱頭方
屋槽方(?)
撩檐枋
遮椽版
下平槫
替木
散斗
令栱
丁華抹頦栱
交互斗
柱頭方
壓跳
令栱
慢栱
瓜子栱
泥道栱
華栱
交互斗
華栱
櫨枓
華頭子
下昂
交互斗
散斗
植斗
闌頭

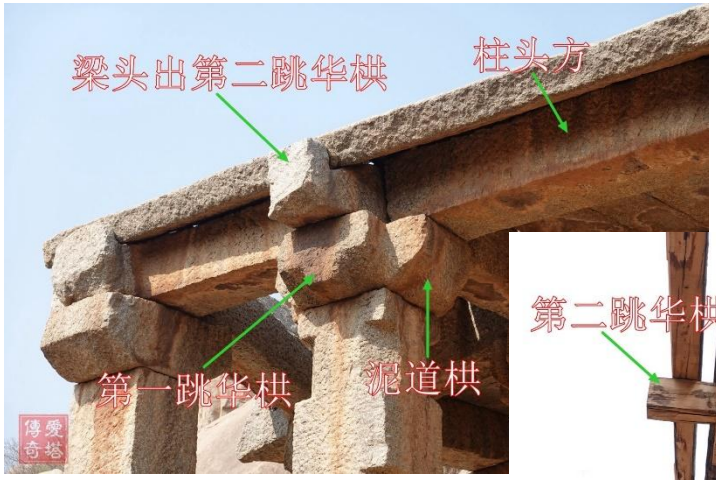

山西五台山
佛光寺大殿
唐
大中十一年
T'ANG
A.D.857
MAIN HALL, FO-KUANG SSU,
WU-T'AI SHAN, SHANSI
河北薊縣
獨樂寺觀音閣
遼
統和三年
LIAO
984
KUAN-YIN KÊ, TU-LÊ SSU
CHI HSIEN, HOPEI
山西榆次縣
永寧寺雨花宮
宋
大中祥符元年
SUNG
1008
YÜ-HUA KUNG, YUNG-NING SSU
YÜ-TZ'Ü, SHANSI
河南登封縣
少林寺初祖庵
宋
宣和七年
SUNG
1125
CH'U-TSU AN, SHAO-LIN SSU
TENG-FENG, HONAN.
山西大同縣
善化寺三聖殿
金
大會皇統間
CHIN
1130-43
SHAN-SHENG TIEN, SHAN-HUA SSU
TA-T'UNG, SHANSI
山東曲阜縣
孔廟金碑亭
金
明昌六年
CHIN
1195
STELE PAVILION
CONFUCIUS' TEMPLE
CH'Ü-FOU, SHANTUNG
河北正定縣
陽和樓
元
物
YUAN
CA. 1250
YANG-HO LOU
CHENG-TING, HOPEI
北平
社稷壇享殿
明
永樂十九年
MING
1412
MAIN HALL, SHÊ-CHI T'AN
PEIPING
SCALE
5M.
北平
智化寺如來殿
明
正統八年
MING
1443
JU-LAI TIEN, CHIH-HUA SSU
PEIPING
山東曲阜縣
孔廟奎文閣
明
弘治十七年
MING
1504
LIBRARY, CONFUCIUS' TEMPLE
CHÜ-FOU, SHANTUNG
北平
清故宮文淵閣
清
乾隆四十年
CH'ING
1776
IMPERIAL LIBRARY,
IMPERIAL PALACES, PEIPING.
歷代斗栱演變
EVOLUTION
OF
THE
CHINESE
"ORDER"

建筑行业

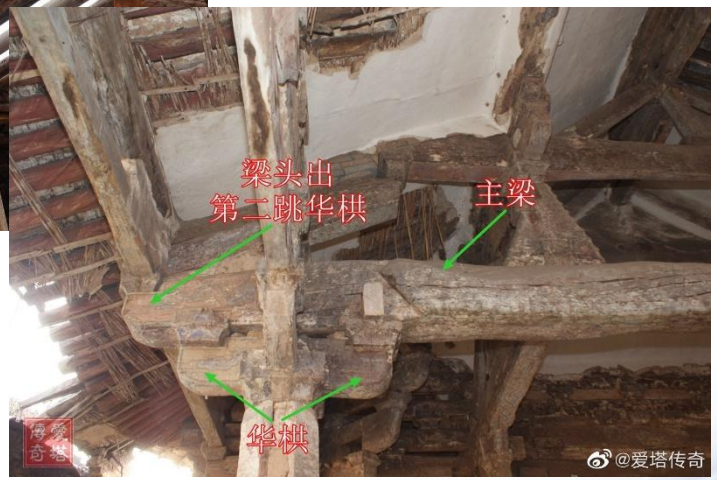
印度

西藏

山西

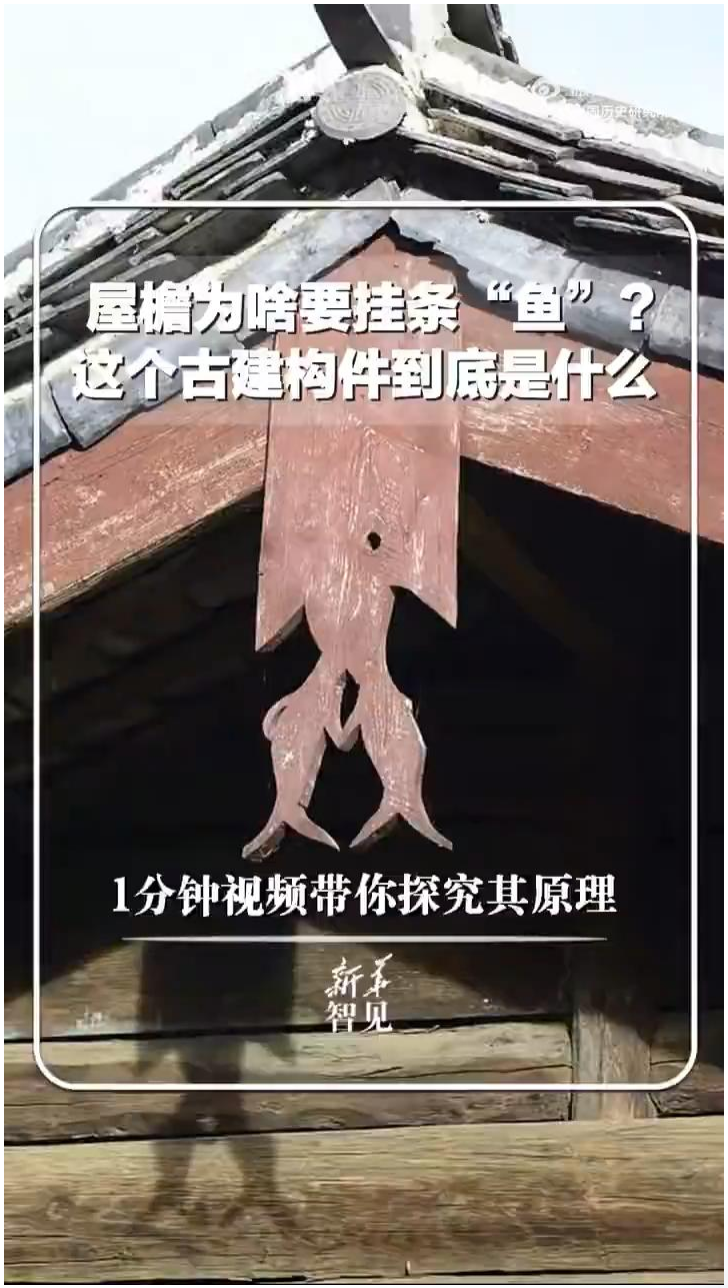






建筑行业

悬鱼，是一种建筑装饰件，位于悬山或歇山建筑两端山面的 博风板下，垂于正脊，大多用木板雕制而成



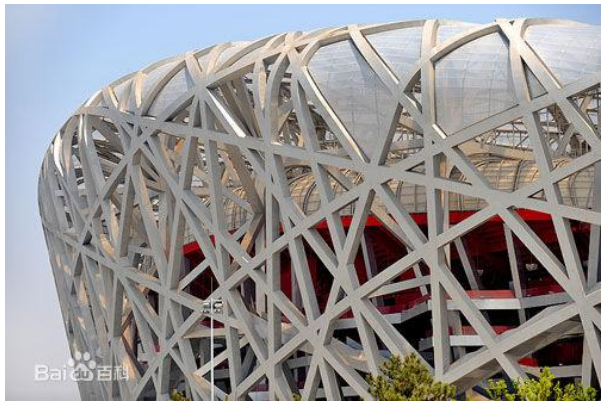
建筑行业



1500多根巨型预制梁架
12000个钢铁铸件
塔身重达7000吨
300个工人参与建造
金属塔身构件于1887年7月1日开始组装，21个月完成竣工

中国科学院大学
University of Chinese Academy of Sciences

建筑行业



门式钢架，共有24根桁架柱
钢结构用钢量达到4万吨
100多名工人参与合拢焊接



珠港澳大桥海底隧道
33节巨型沉管
单个长度超过170米
单个重量达到8万吨

工业化的启示

- 现代工业生产：精细分工与协作

航空工业、汽车工业，这些成熟行业的规范化抽象和机制，极大促进我们对复杂系统的控制

从制造商到用户（驾驶员、乘客），都是受益者



中国的汽车市场已经全面进入“新能源主导”新阶段

今年以来，新能源汽车零售渗透率连续4个月超过60%，这意味着每卖出3辆新车，就有2辆是新能源车。——（新华社，2026.8.27）



机工战略 II

26-9-1 15:51 来自 微博网页版

1.4万 阅读

麦肯锡研究显示，中国新兴电动车企业已将包含平台开发在内的新车开发周期压缩至约24个月，较传统车企的40—50个月缩短近一半。其速度优势主要来自平台和零部件复用、虚拟仿真、软硬件解耦、工装提前介入、供应链协同以及更精干的决策机制。

那么，中国车企为何能把新车开发周期压缩至24个月？

麦肯锡研究显示，中国新兴电动车企业从概念到上市约需24个月，而且包含平台开发；传统车企通常需要40—50个月甚至更长。速度差距主要来自六个方面。

一是平台化和零部件复用。电池包、电机、电子架构及大量隐性零部件在不同车型间通用，可节省1—3个月。二是虚拟开发。中国车企约65%的测试使用软件仿真和虚拟样车，其他地区为40%—50%，物理样车数量最多可减少一半，节省9—11个月。三是软硬件解耦，量产前优先完成安全和认证功能，其他功能通过OTA持续升级，可节省3—10个月。

四是提前启动模具和工装，在设计尚未完全冻结时并行推进，节省1—4个月。五是加强垂直整合，关键电池、电驱和软件能力掌握在内部，同时让供应商早期参与共同开发，节省3—4个月。六是压缩决策链，由4—5名高管组成项目小组，高频解决定位、设计和工程变更问题，减少层层汇报等待。

中国汽车的速度优势，核心是把串联流程改成并联，把重复开发改成平台复用，把大量实物试错转移到数字空间。汽车产业竞争正在从单纯比产品、成本和产能，进一步转向比研发效率、供应链协同和组织速度。

成功的复杂系统具有何种形式?

- 视角不同，复杂系统的层次结构也不同

  # 复杂系统通常表现出许多不同的层次结构——横看成岭侧成峰

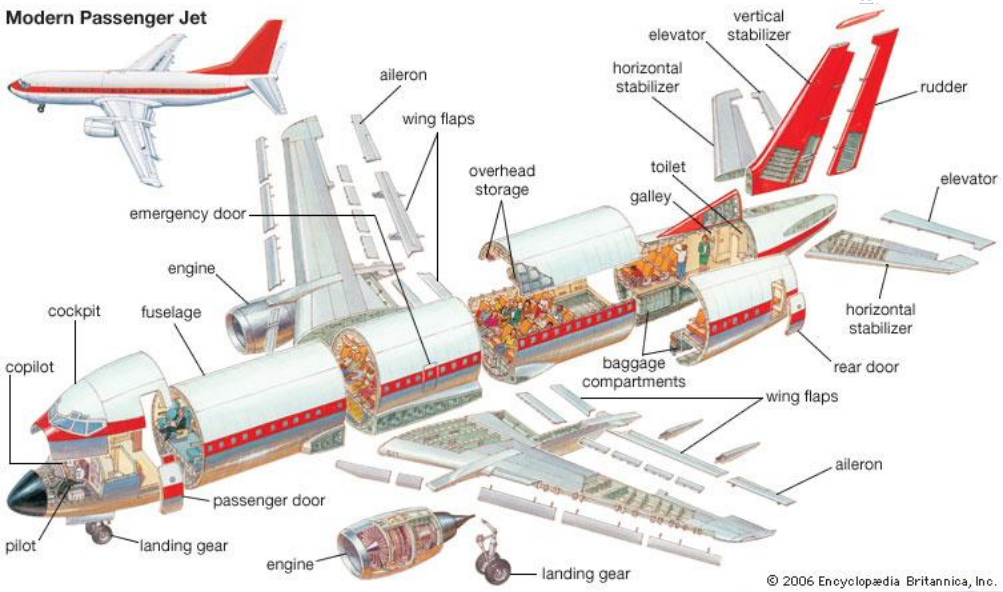
视角1——对象化*(组件化，part of)*

C919 = 涡轮发动机 + 电传飞控 + …

视角2——类型化*(分类化，is a)*

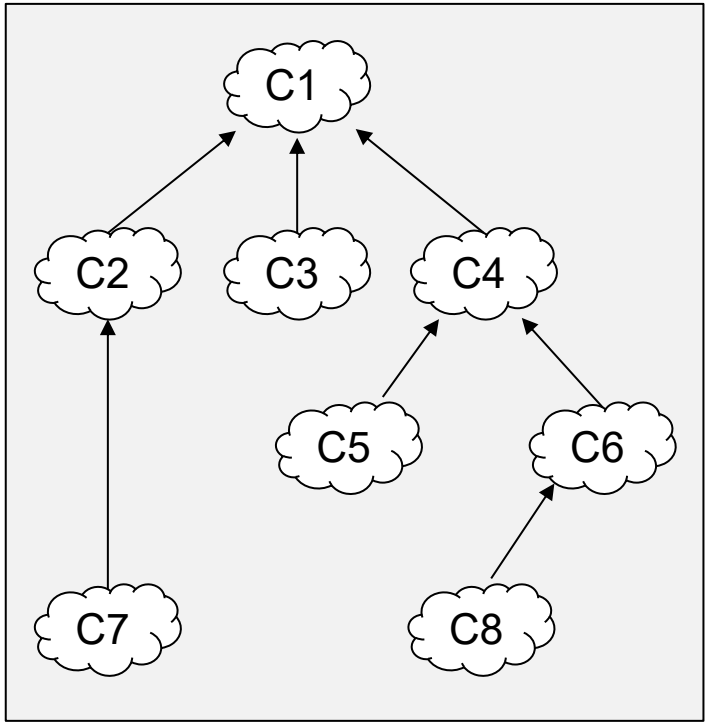
航空发动机 = {活塞式、涡喷、涡轮、…}
飞控系统 = {机械式、电传、光传、…}

上面两个视角都重要!

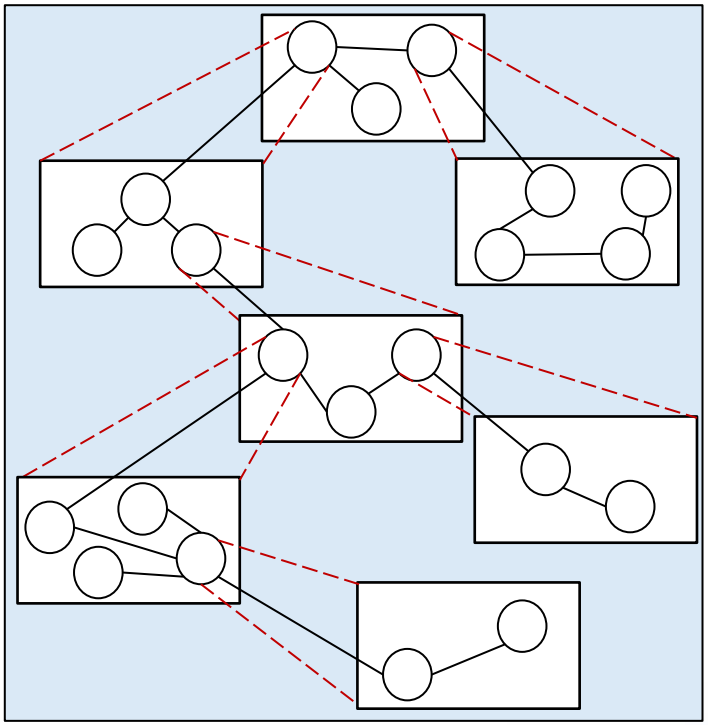


成功的复杂系统具有何种形式？ cont.

“类型”和“对象”是两个正交的层次结构，都是分层的
即从基础的类型、对象，进一步抽象，形成更高层的类型、对象



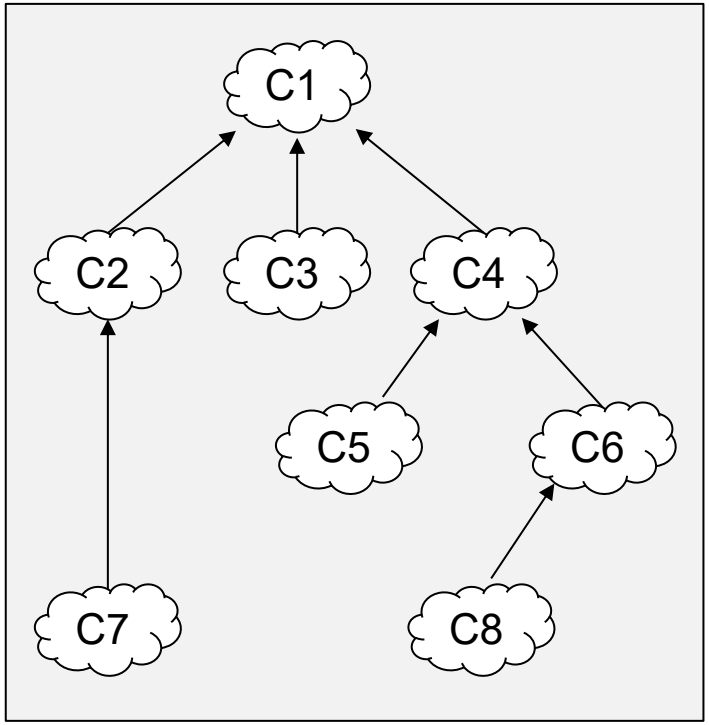
“类型”结构
(is a)



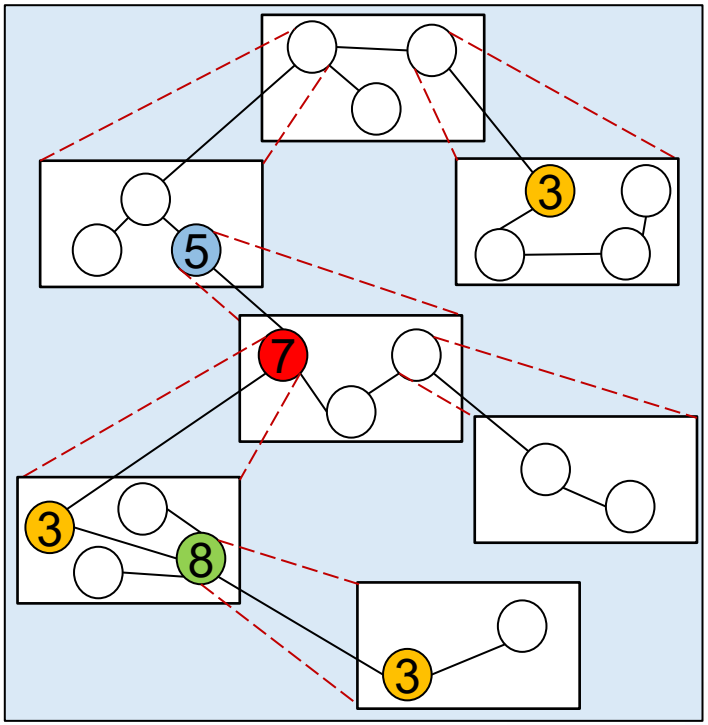
“对象”结构
(part of)

成功的复杂系统具有何种形式？ cont.

“对象”结构的每个对象都代表了某个类型的一个具体实例
“类型”结构将共性组件进行抽象，避免了系统冗余



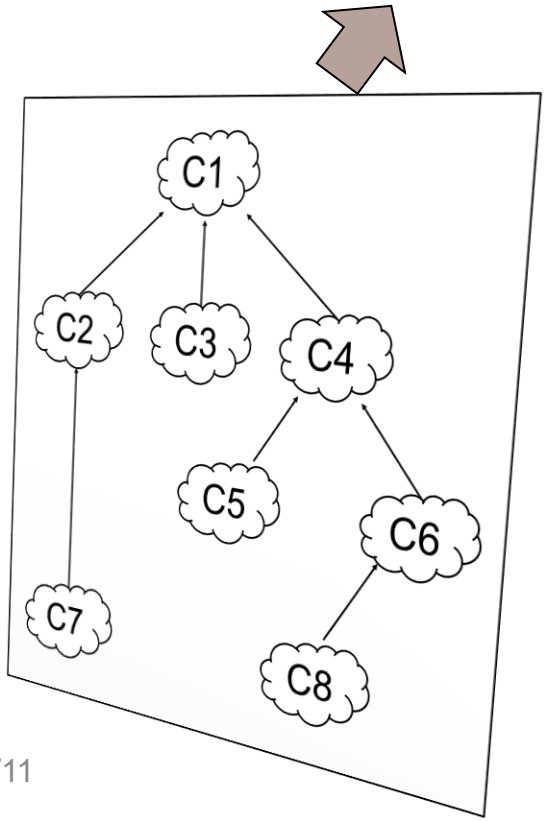
“类型”结构
(is a)

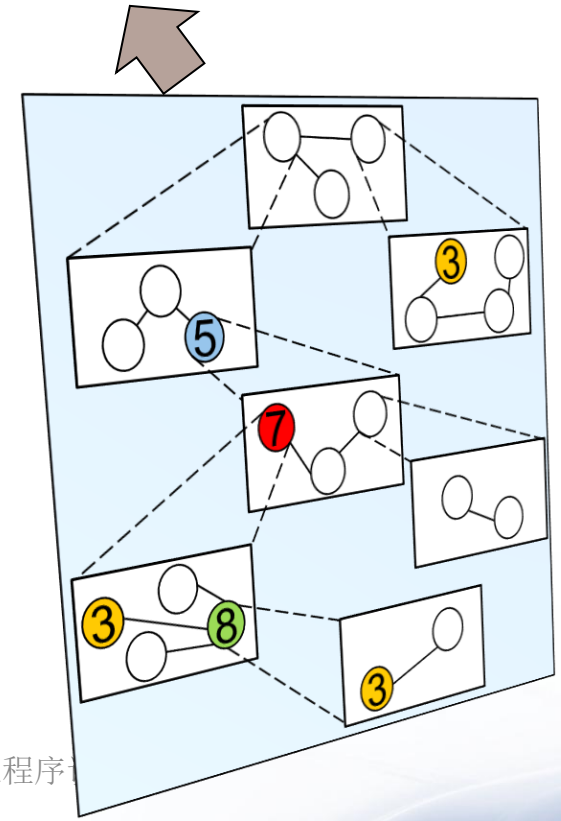


“对象”结构
(part of)

成功的复杂系统具有何种形式？cont.

成功的复杂系统就是在设计中包含了深思熟虑的“类型”结构和“对象”结构，并具备了复杂系统的5种属性，如果不考虑这些因素，我们就不太可能准时在预算范围内交付满足需求的系统







中国科学院大学
University of Chinese Academy of Sciences

控制软件系统的复杂性

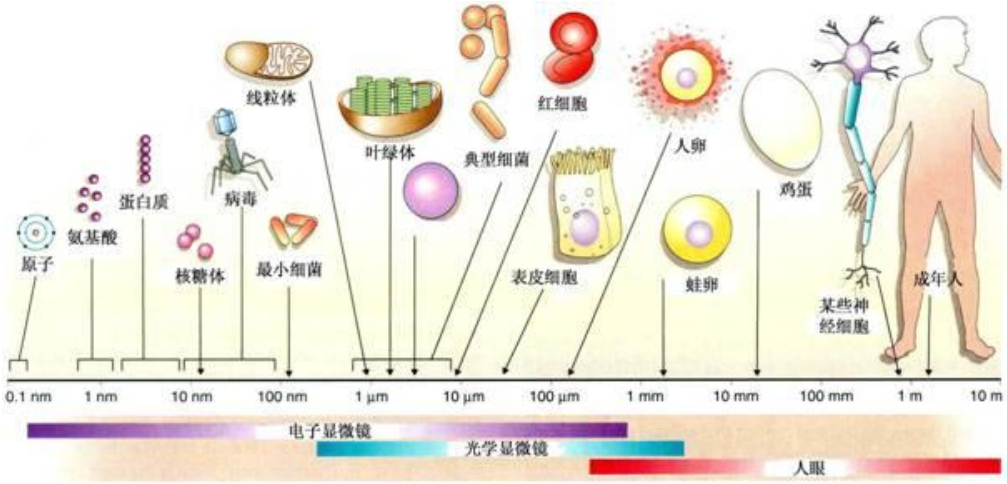
复杂系统有哪些特点?

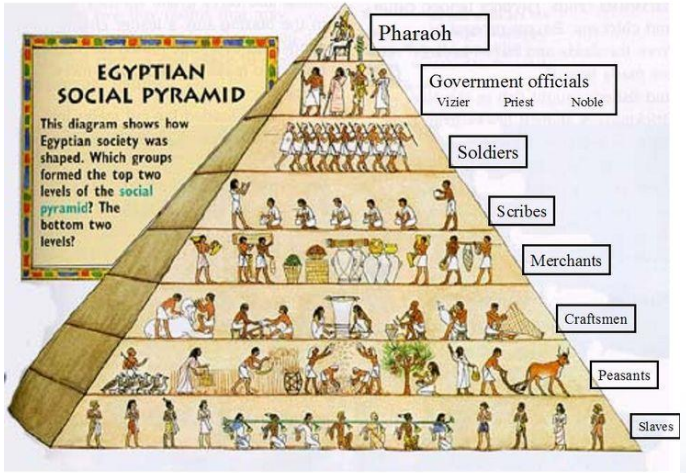
复杂系统的5个属性[3]

1. 层次结构

“复杂系统常以层次结构的形式存在，即由一些相关的子系统组成，子系统又有自己的子系统，直至达到某种最低层次的基本组件”[4]

动物的结构、物质的结构、社会系统的结构……我们更善于理解这些有层次结构的系统

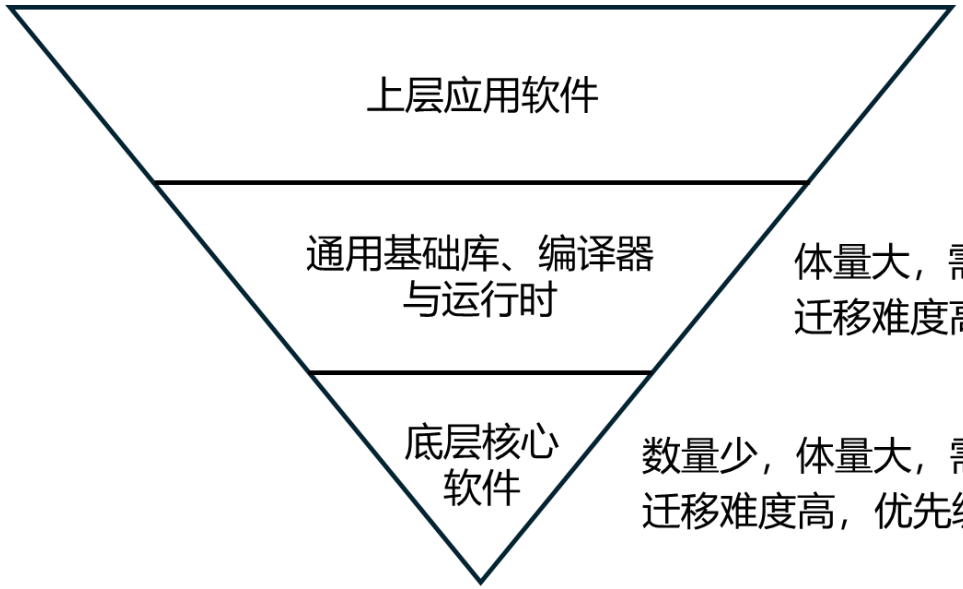




[3] (美)布奇(Booch, G.). 面向对象分析与设计: 第3版. 电子工业出版社, 2012.

[4] Courtois P J. On time and space decomposition of complex structures. Communications of the ACM, 1985, 28(28):590-603.

新兴指令集架构的软件生态迁移



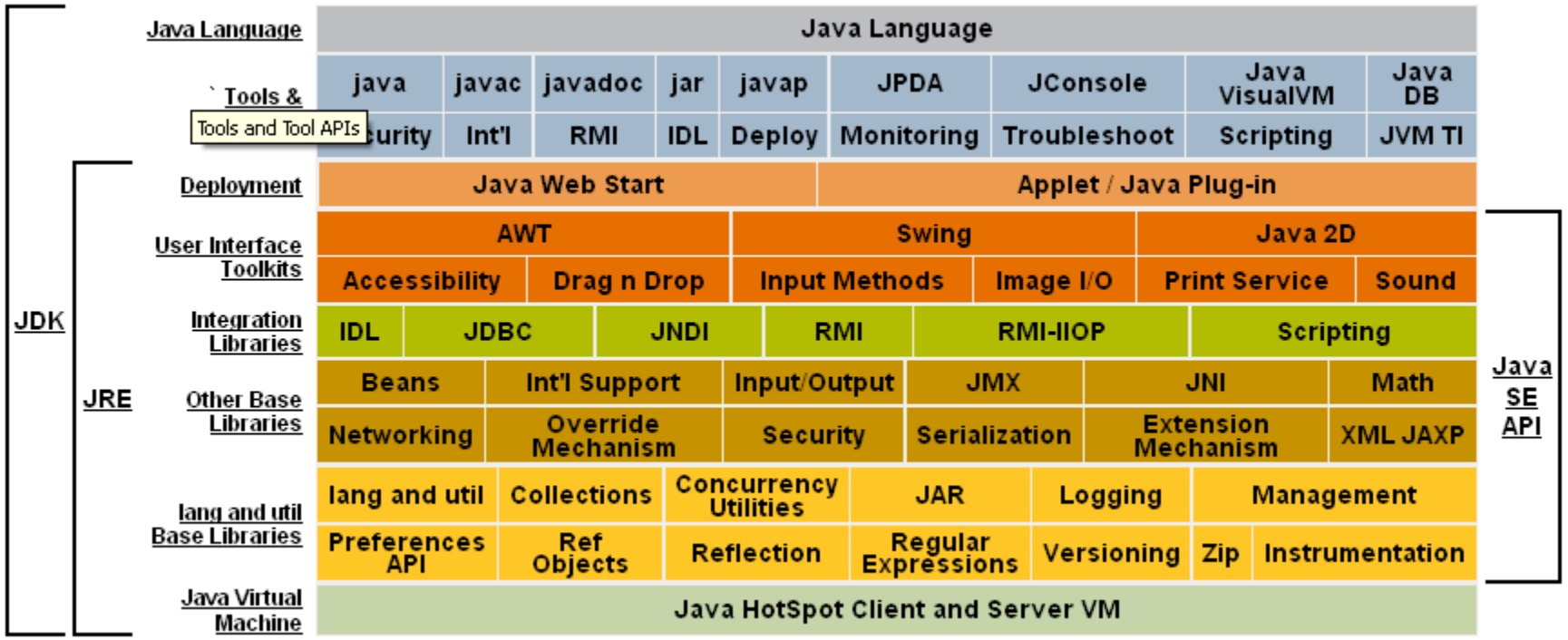
数量多，体量不定，多数代码与架构无关，
迁移难度一般或较低，优先级较低

体量大，需修改代码较多且较为复杂，可靠性要求高，
迁移难度高，优先级较高

数量少，体量大，需修改代码多且较为复杂，可靠性要求极高，
迁移难度高，优先级最高

层层依赖，环环相扣

解剖一个“麻雀”

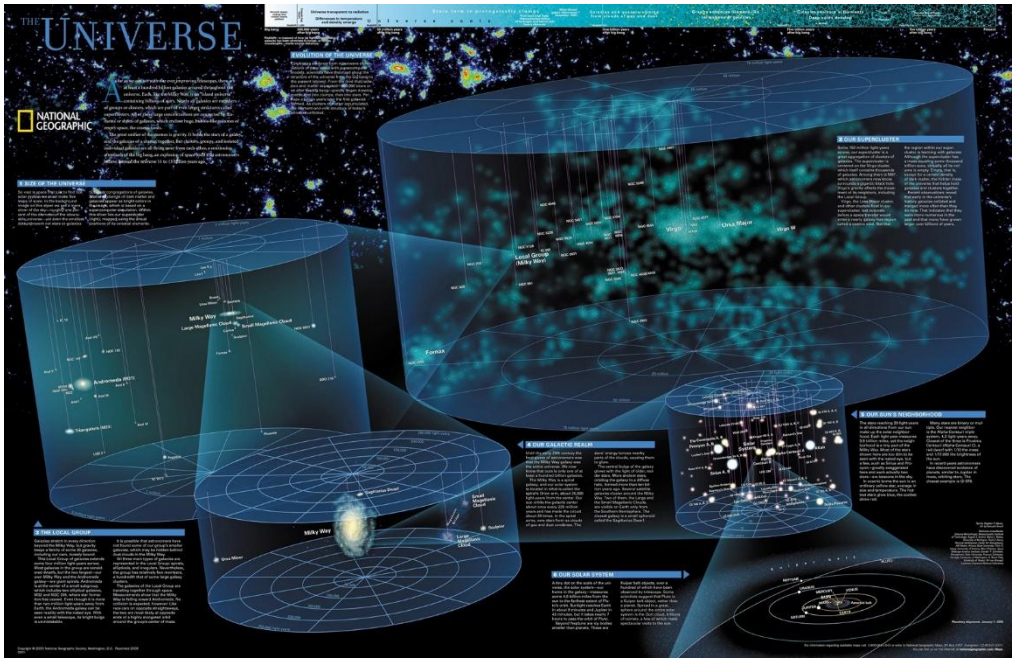

JDK
JRE
Java Language
Java Language
Tools &
Tools and Tool APIs
java
javac
javadoc
jar
javap
JPDA
JConsole
Java VisualVM
Java DB
Int'l
RMI
IDL
Deploy
Monitoring
Troubleshoot
Scripting
JVM TI
Deployment
Java Web Start
Applet / Java Plug-in
User Interface Toolkits
AWT
Swing
Java 2D
Accessibility
Drag n Drop
Input Methods
Image I/O
Print Service
Sound
Integration Libraries
IDL
JDBC
JNDI
RMI
RMI-IIOP
Scripting
Other Base Libraries
Beans
Int'l Support
Input/Output
JMX
JNI
Math
Networking
Override Mechanism
Security
Serialization
Extension Mechanism
XML JAXP
lang and util Base Libraries
lang and util
Collections
Concurrency Utilities
JAR
Logging
Management
Preferences API
Ref Objects
Reflection
Regular Expressions
Versioning
Zip
Instrumentation
Java Virtual Machine
Java HotSpot Client and Server VM
Java SE API

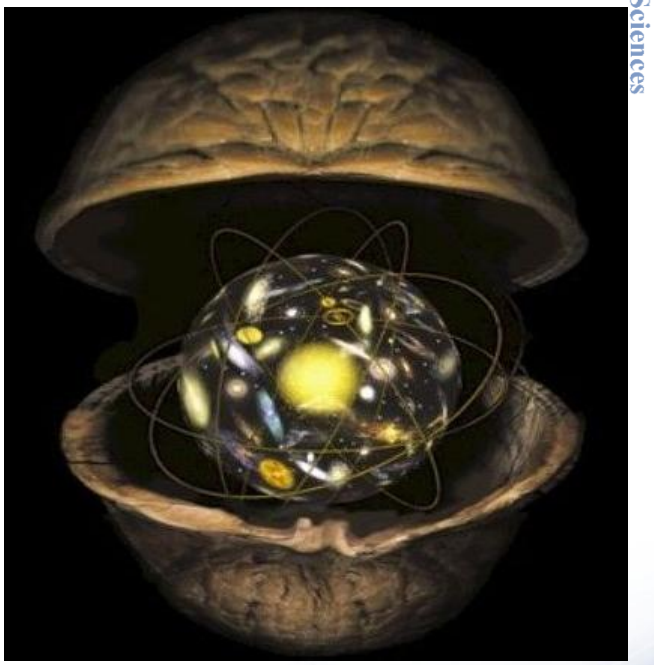
复杂系统的5个属性cont.

2. 相对基础

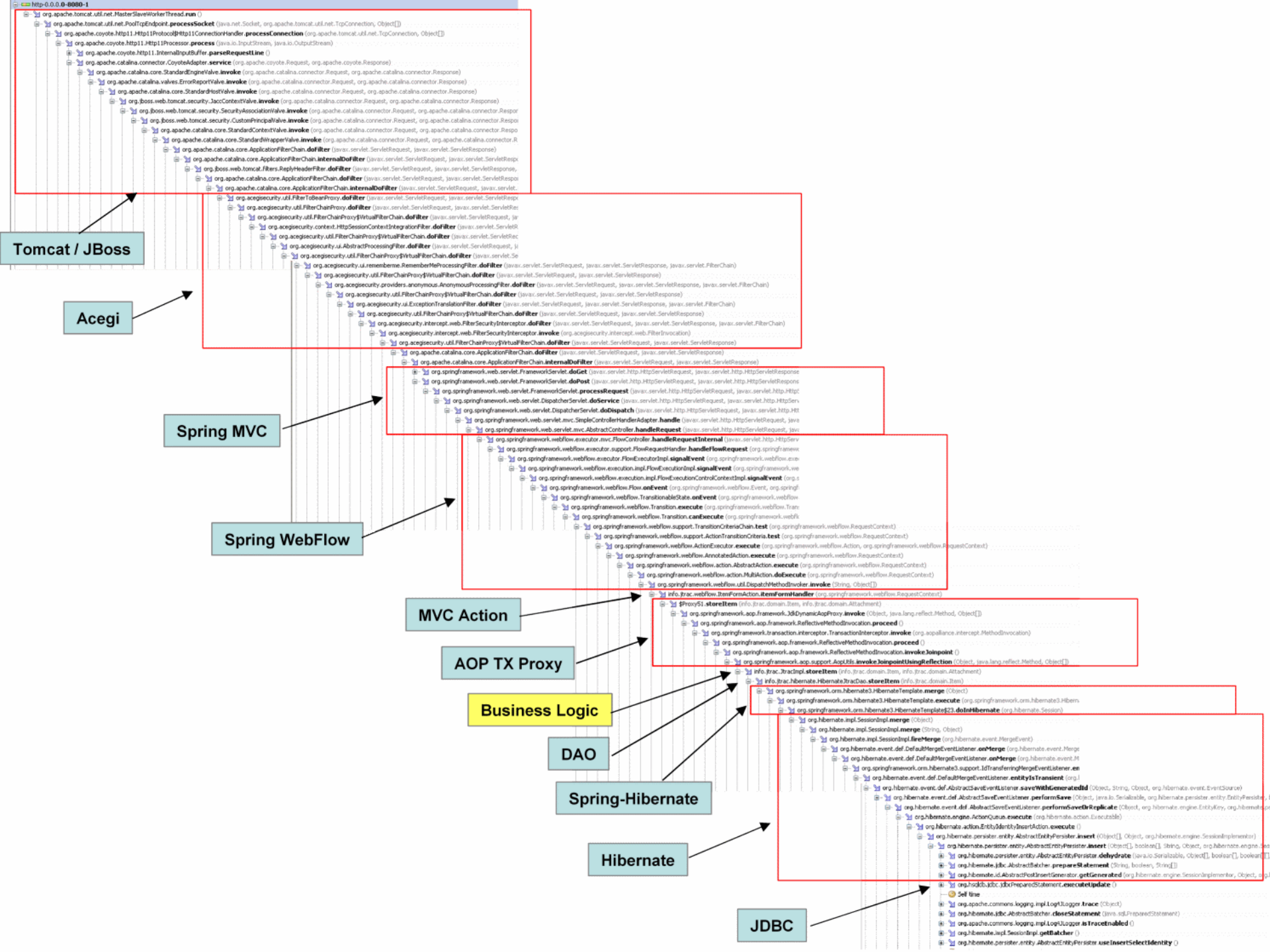
对基本组件的选择，很大程度上取决于系统观察者的判断

对于一个观察者来说很基础的东西，对另一个观察者可能具有很高的抽象





中国科学院大学
University of Chinese Academy of Sciences

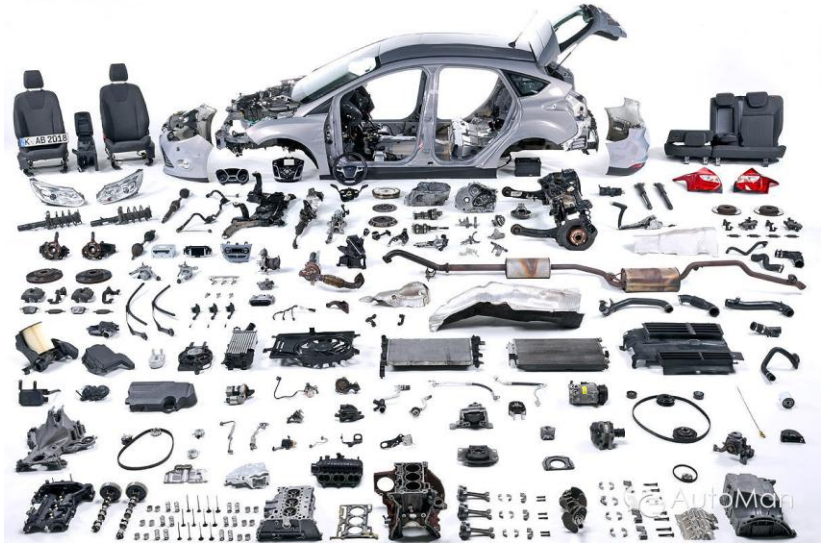

Tomcat / JBoss
Acegi
Spring MVC
Spring WebFlow
MVC Action
AOP TX Proxy
Business Logic
DAO
Spring-Hibernate
Hibernate
JDBC

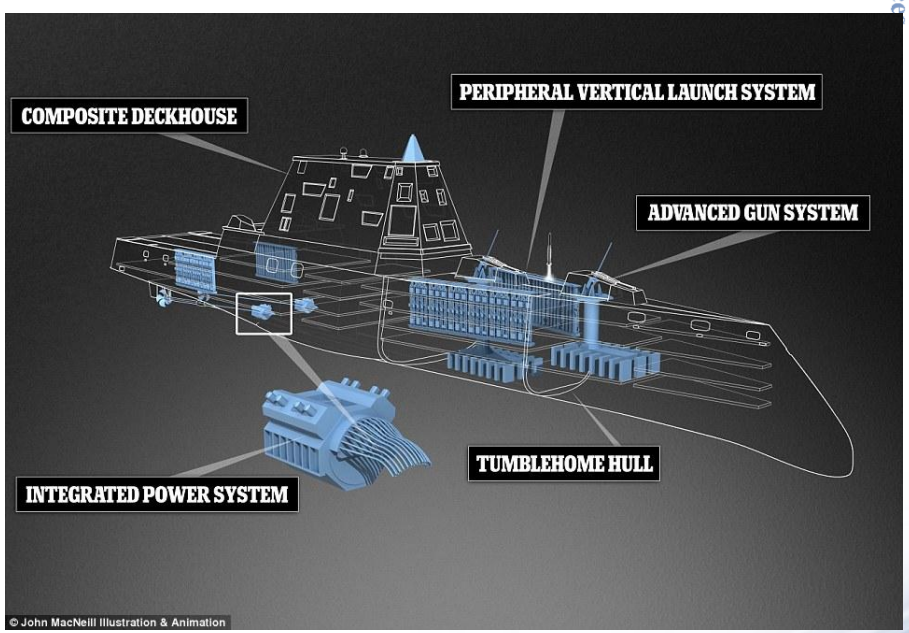
复杂系统的5个属性cont.

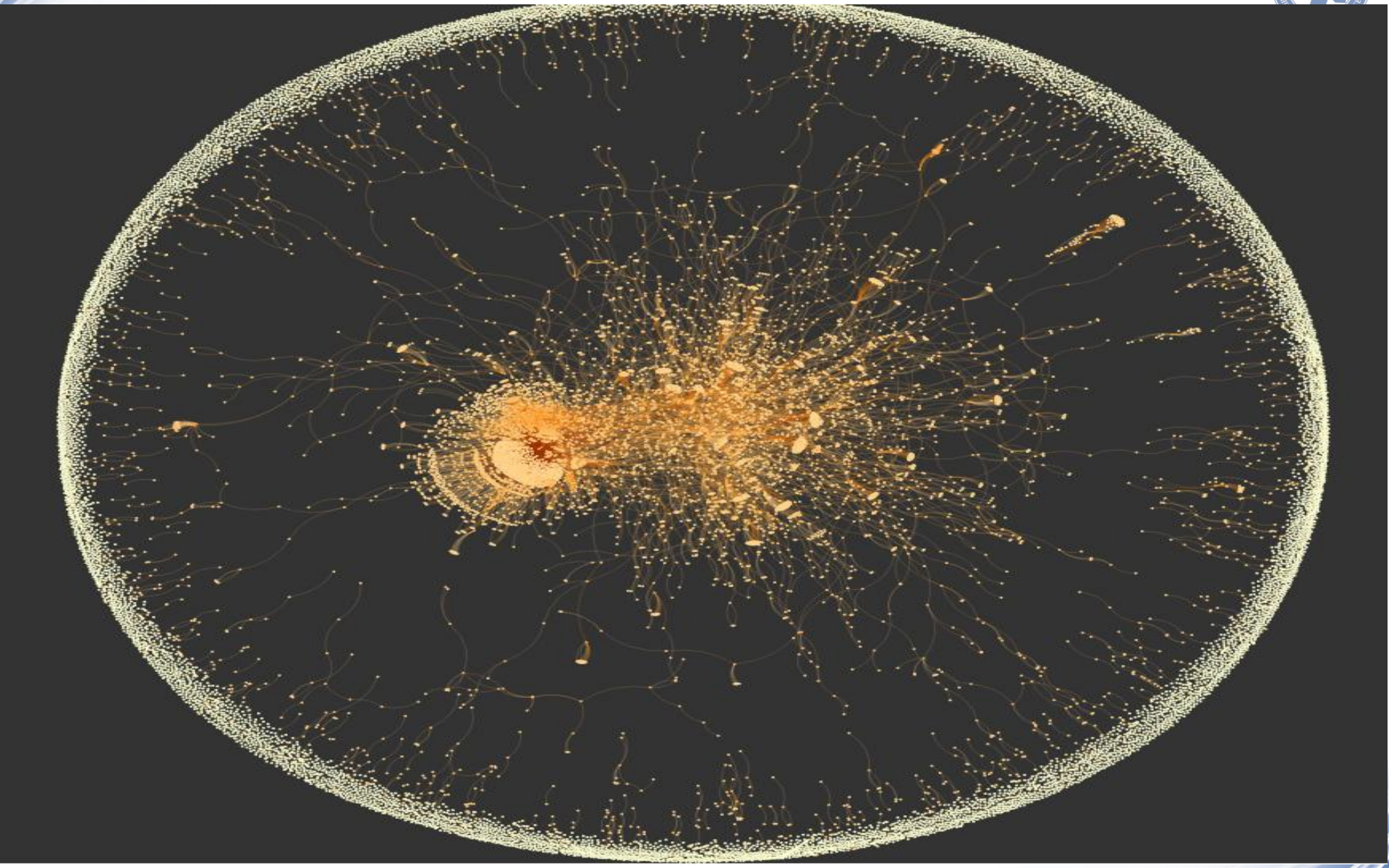
3. 关注点分离(separation of concerns)

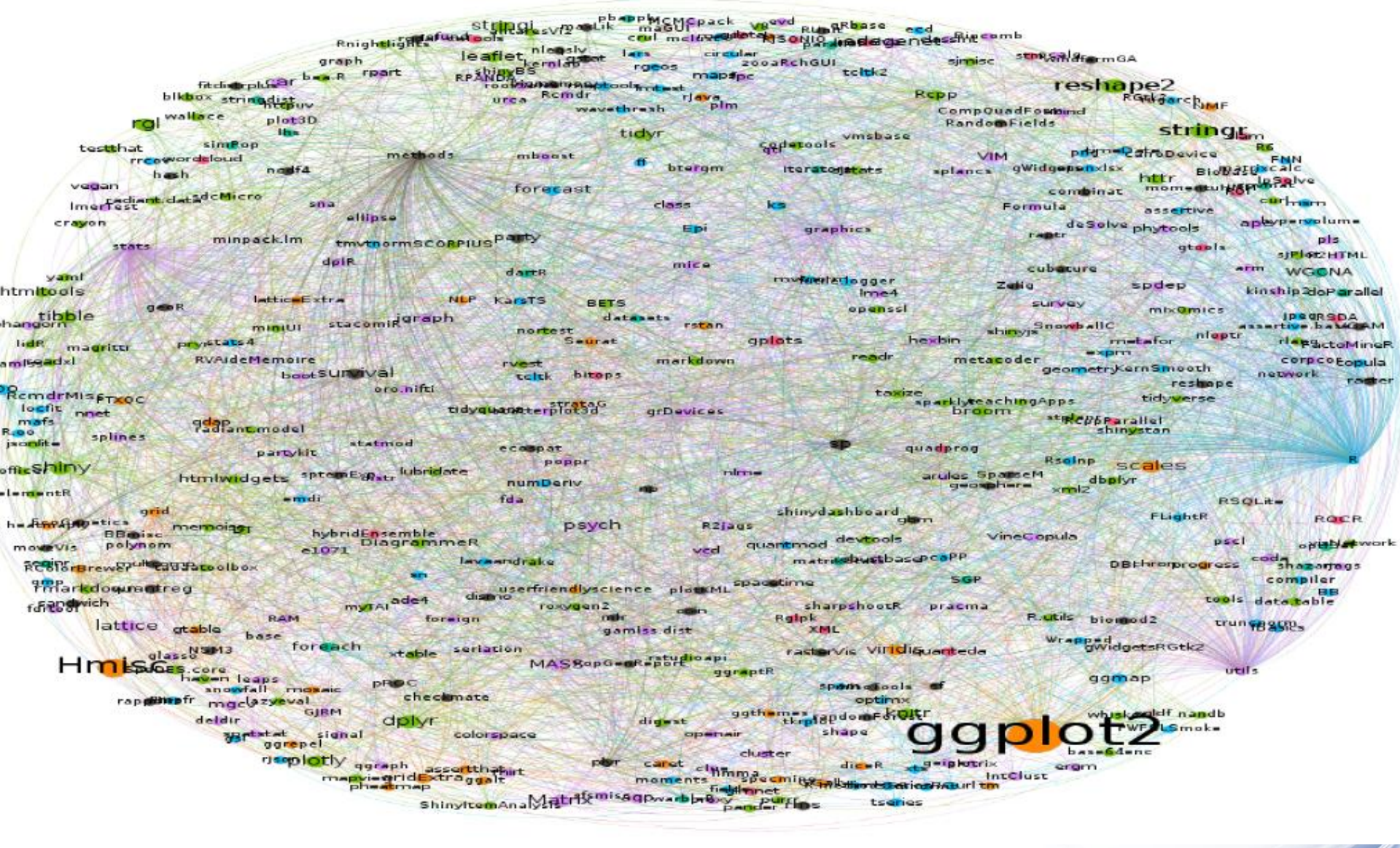
层次系统的可分解性，意味着子系统内部的联系通常比子系统之间的联系更强，子系统内部的高频行为与子系统之间的外部低频行为实现了分离

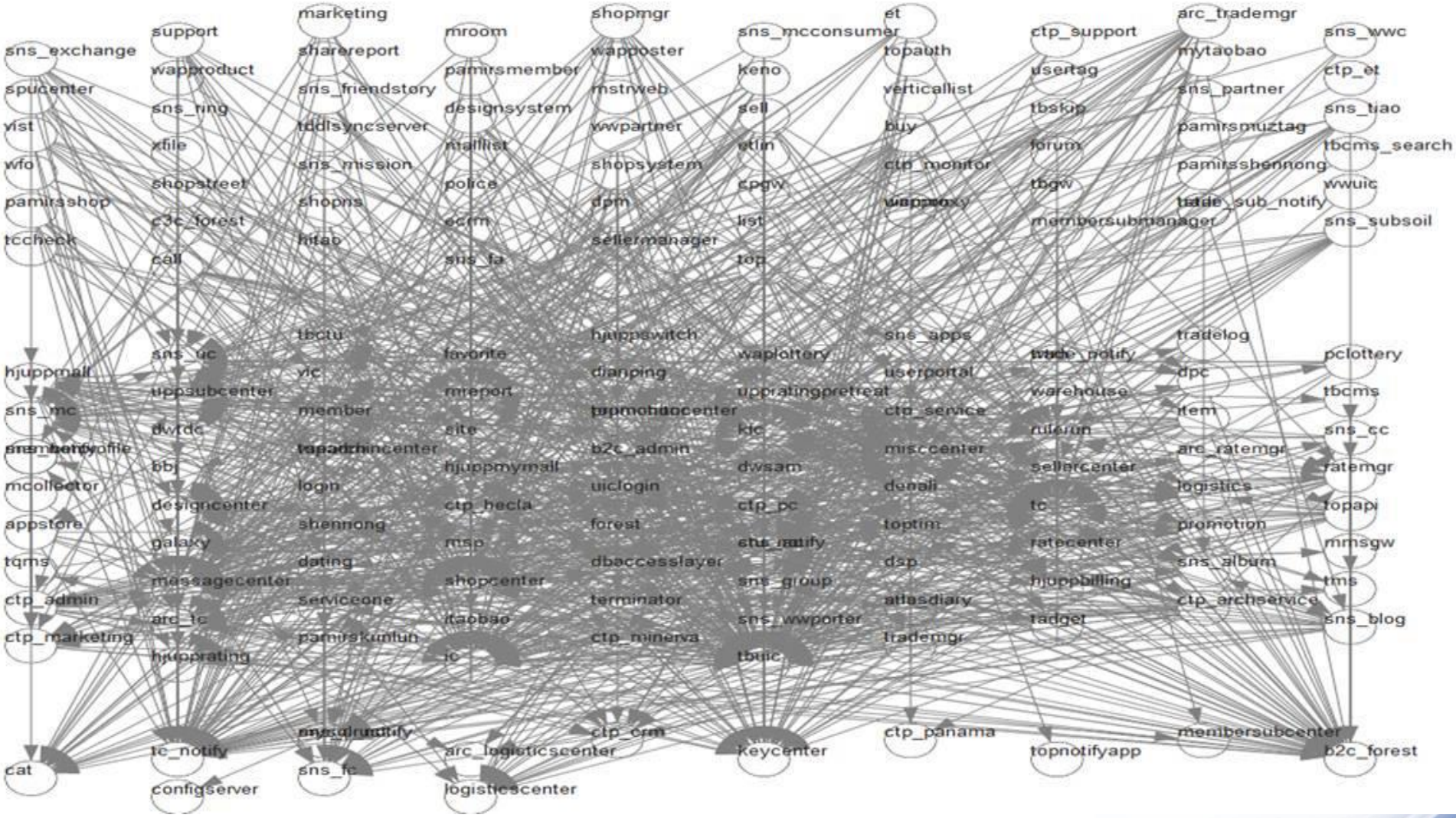
让我们能够以相对隔离的方式来研究每个部分，也让子系统的行为对其他子系统以及系统整体的影响有限








ggplot2
reshape2
stringr
Hmisc
dplyr
lattice
shiny
plotly
Matrix
rgl
tidyr
leaflet
forecast
survival
foreach
psych
scales
plyr
sp
tibble
htmltools
htmlwidgets
gplots
party
methods
mboost
caret
ggmap
RColorBrewer
rmarkdown
knitr
jsonlite
yaml
Rcpp
MASS
cluster
nlme
lme4
Formula
maps
mapproj
rgeos
maptools
gridExtra
igraph
Seurat
survey
openssl
httr
curl
XML
RSQLite
DBI
foreign
nnet
mgcv
lubridate
stringi
stringr

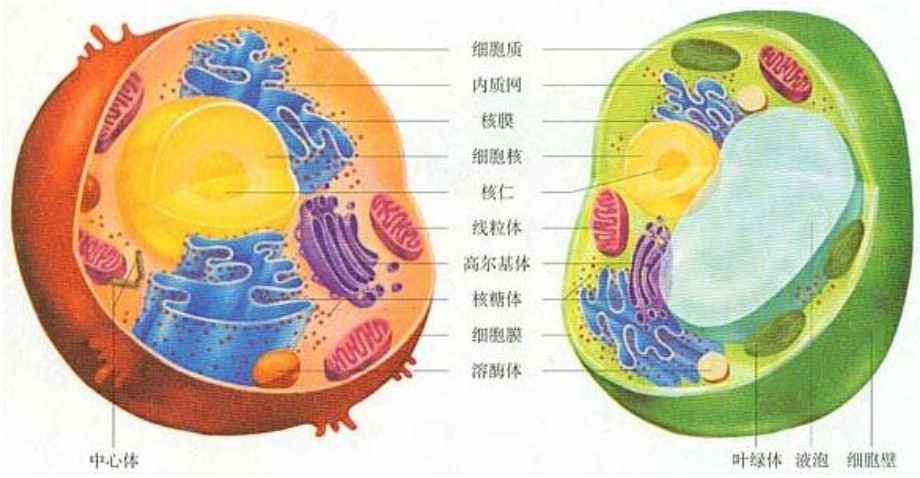

sns_exchange
spucenter
vist
wfo
pamirsshop
tccheck
support
wapproduct
sns_ring
xfile
shopatreet
c3c_forest
call
marketing
sharereport
sns_friendstory
tddlsyncserver
sns_mission
shopns
hitab
mroom
pamirsmember
designsystem
maillist
police
ecrm
sns_fa
shopmgr
wapposter
mstrweb
wwpartner
shopsystem
dpm
sellermanager
sns_mcconsumer
keno
sell
etlin
cpgw
list
top
et
topauth
verticallist
buy
ctp_monitor
wappsoxy
ctp_support
usertag
tbskip
forum
tbgw
membersubmanager
arc_trademgr
mytaobao
sns_partner
pamirsmuztag
pamirsshennong
tradе_sub_notify
sns_wwc
ctp_et
sns_tiao
tbcms_search
wwuic
sns_subsoil
hjuppmall
sns_mc
mcollector
appstore
tqms
ctp_admin
ctp_marketing
sns_uc
uppsubcenter
dwrdc
bbj
designcenter
galaxy
messagecenter
arc_tc
hjupprating
tbctu
vic
member
login
shennong
dating
serviceone
pamirsklumlub
favorite
mreport
site
hjuppmymall
ctp_hecla
msp
shopcenter
itaobao
ic
hjuppswitch
dianping
b2c_admin
uiclogin
forest
dbaccesslayer
terminator
ctp_minerva
waploltery
uppratingpretreat
kic
dwsam
ctp_pc
sns_group
sns_wwporter
tbuic
sns_apps
userpoital
ctp_service
misccenter
denali
toptim
dsp
atlasdiary
trademgr
tradelog
dpc
item
rulerun
arc_ratemgr
logistics
promotion
sns_album
ctp_archservice
pclottery
tbcms
sns_cc
ratemgr
topapi
mmsgw
tms
sns_blog
cat
tc_notify
configserver
sns_fc
arc_logisticscenter
logisticscenter
ctp_crm
keycenter
ctp_panama
topnotifyapp
membersubcenter
b2c_forest

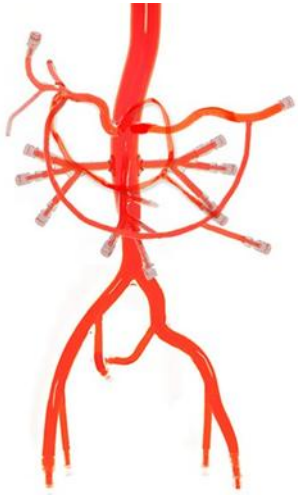
复杂系统的5个属性cont.

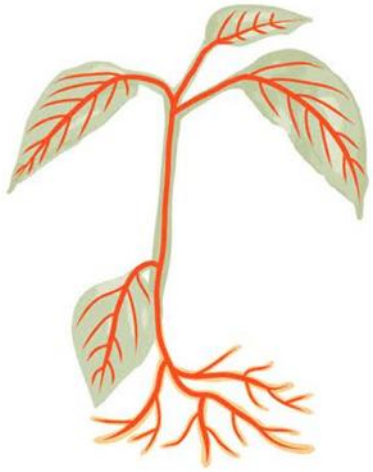
4. 共同模式

复杂系统往往是以一种经济的表达方式来实现的，即层次结构通常只是由少数不同类型的子系统，按照不同的组合和安排方式构成

这种共同的模式，其本质是对组件、子系统的复用，例如，细胞、脉管系统，在植物和动物系统中都存在







复杂系统的5个属性cont.

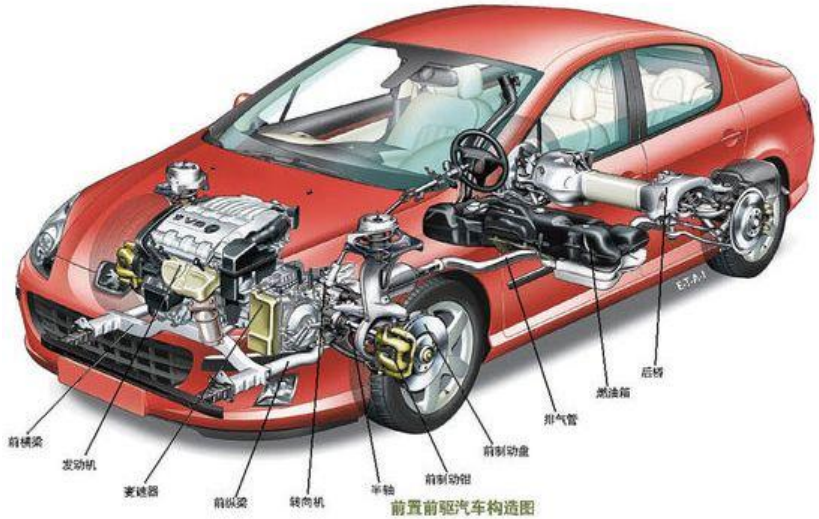
4. 共同模式

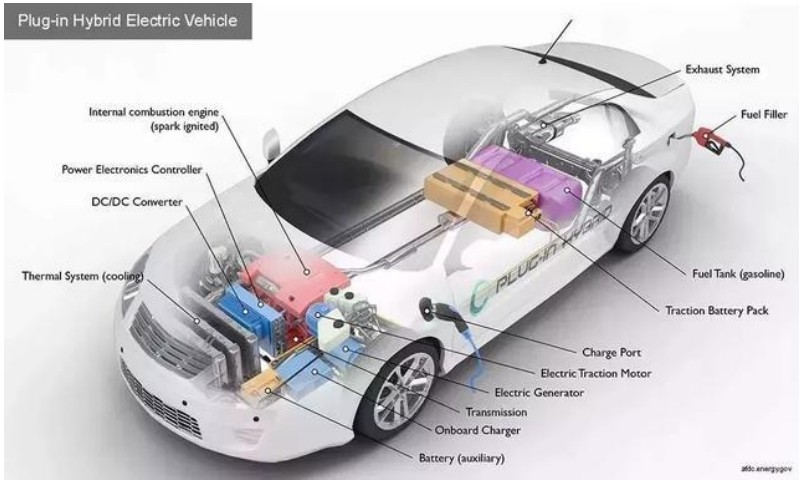
工业产品的例子：

燃油车

混动车

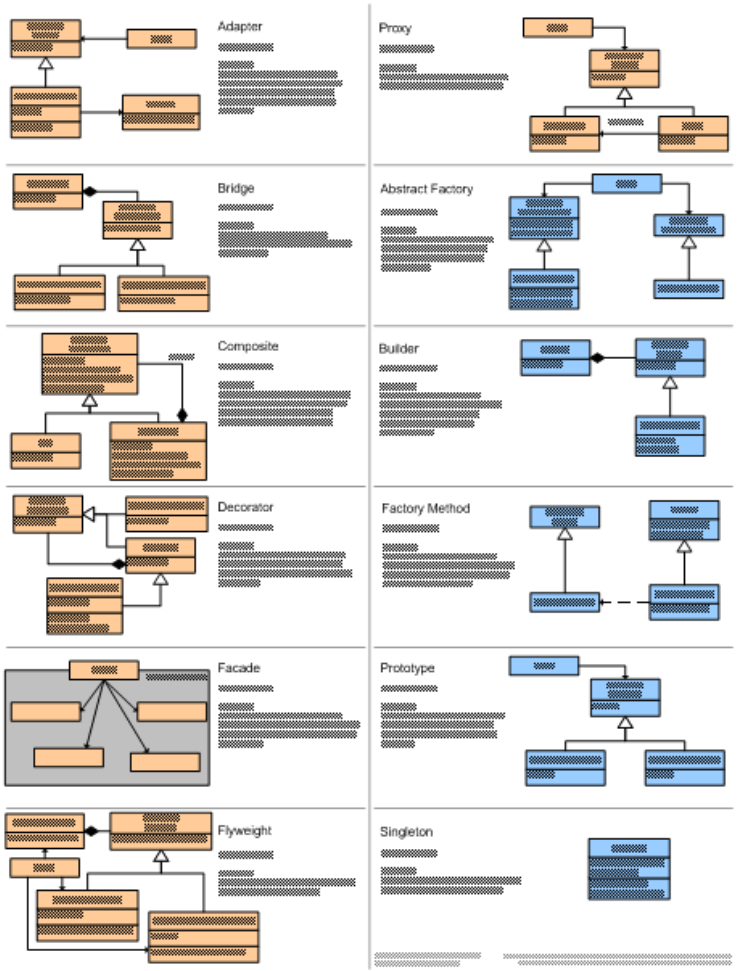
纯电车...







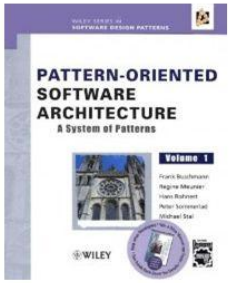
Design Patterns

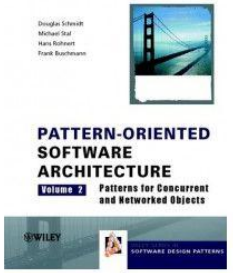


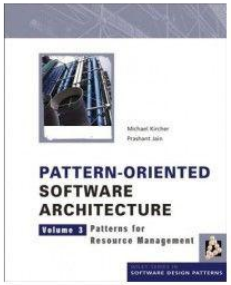
POSA



Pattern Oriented Software Architecture









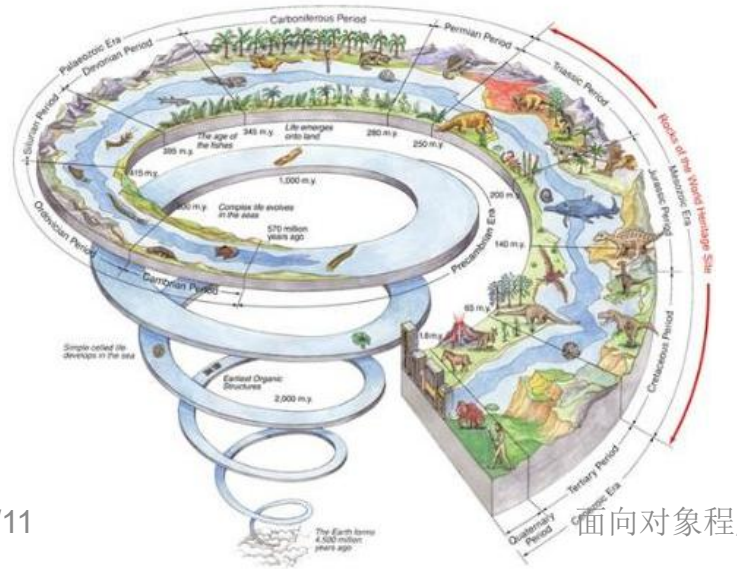


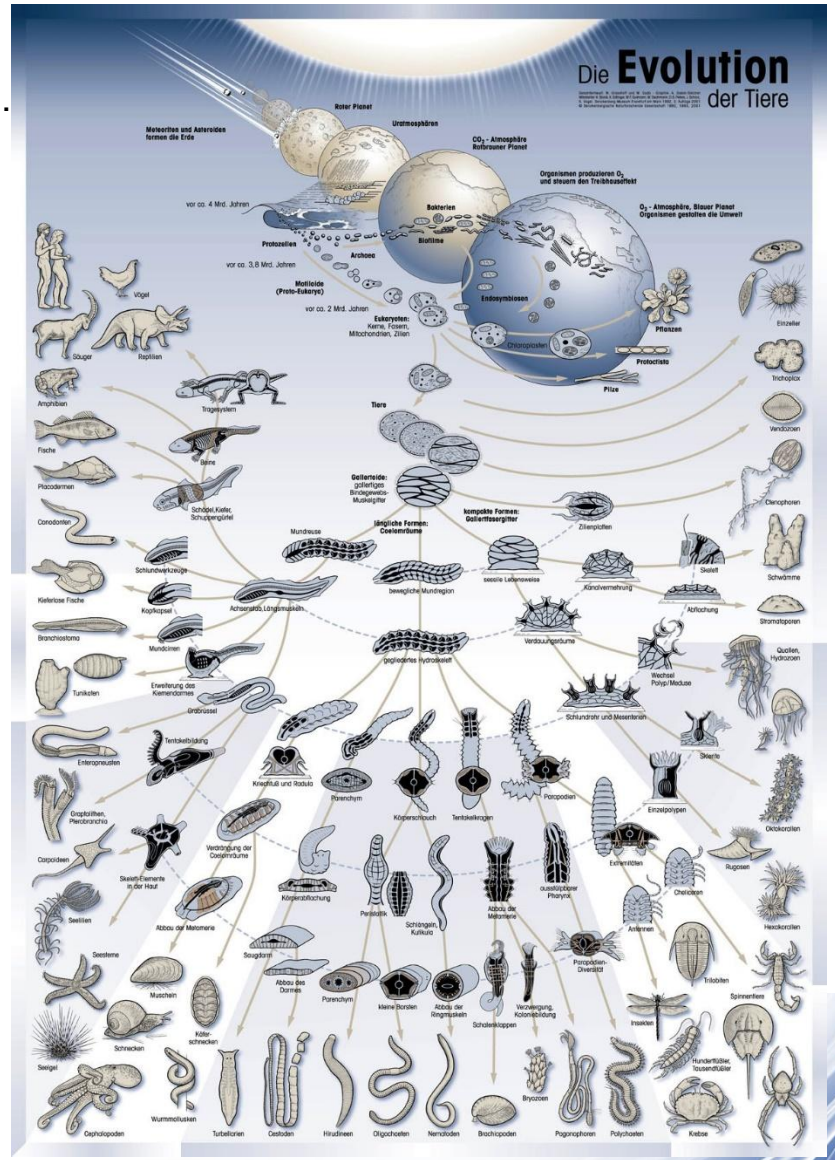
复杂系统的5个属性cont.

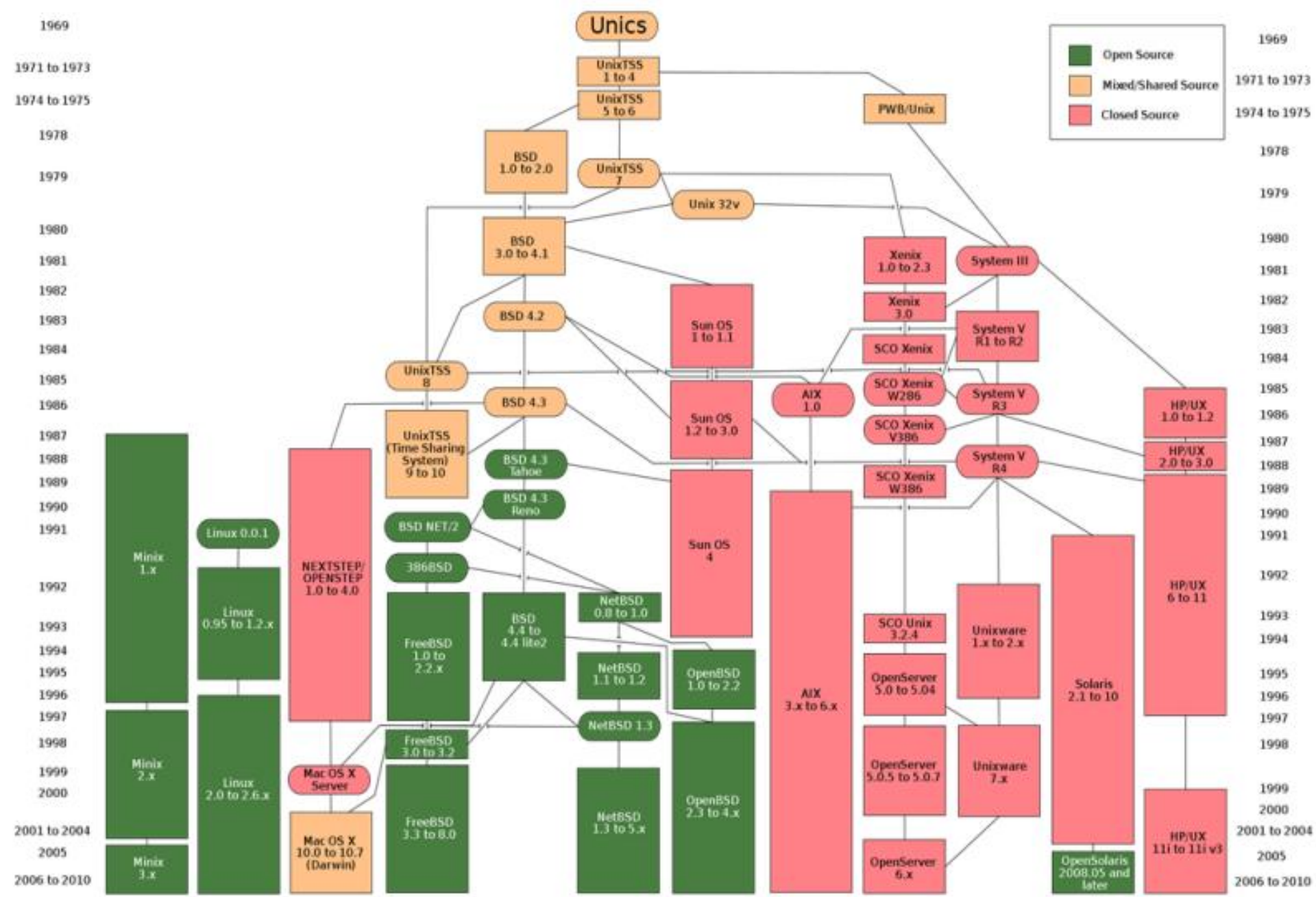
5. 稳定的中间形式（“螺旋”演变）

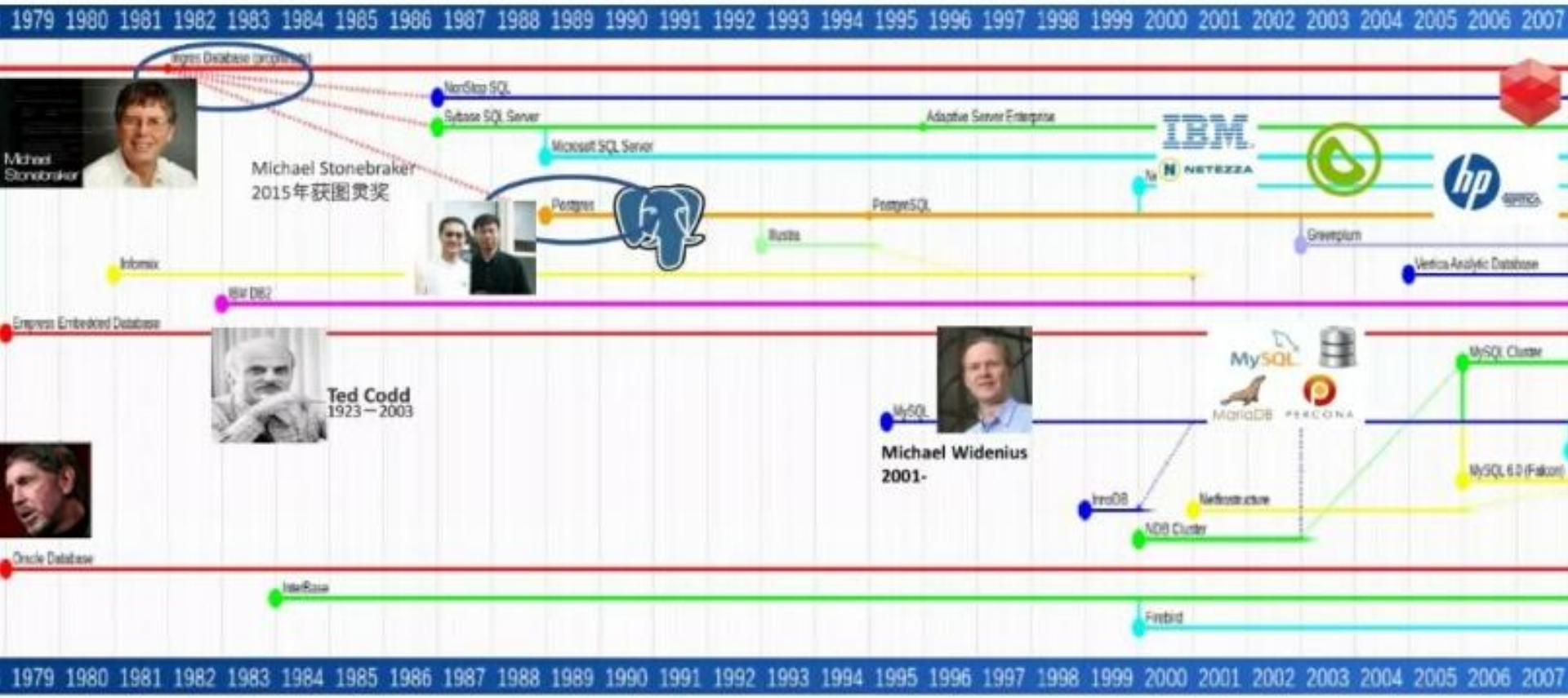
从头设计的复杂系统往往不能工作，打补丁的方式也难以有效，必须从能工作的简单系统开始，持续演进...

随着系统的演变，简单系统变得复杂，进一步又变为基础组件，在这些基础组件上构建更复杂的系统






Unics
Open Source
Mixed/Shared Source
Closed Source
1969
1971 to 1973
1974 to 1975
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001 to 2004
2005
2006 to 2010
UnixTSS 1 to 4
UnixTSS 5 to 6
PWB/Unix
BSD 1.0 to 2.0
UnixTSS 7
Unix 32v
BSD 3.0 to 4.1
Xenix 1.0 to 2.3
System III
Xenix 3.0
BSD 4.2
Sun OS 1 to 1.1
System V R1 to R2
SCO Xenix
UnixTSS 8
SCO Xenix W286
AIX 1.0
System V R3
BSD 4.3
Sun OS 1.2 to 3.0
SCO Xenix V386
HP/UX 1.0 to 1.2
UnixTSS (Time Sharing System) 9 to 10
BSD 4.3 Tahoe
System V R4
HP/UX 2.0 to 3.0
SCO Xenix W386
BSD 4.3 Reno
Minix 1.x
Linux 0.0.1
NEXTSTEP/ OPENSTEP 1.0 to 4.0
BSD NET/2
Sun OS 4
386BSD
HP/UX 6 to 11
Linux 0.95 to 1.2.x
NetBSD 0.8 to 1.0
SCO Unix 3.2.4
Unixware 1.x to 2.x
FreeBSD 1.0 to 2.2.x
BSD 4.4 to 4.4 lite2
NetBSD 1.1 to 1.2
OpenBSD 1.0 to 2.2
AIX 3.x to 6.x
OpenServer 5.0 to 5.04
Solaris 2.1 to 10
Minix 2.x
NetBSD 1.3
FreeBSD 3.0 to 3.2
OpenServer 5.0.5 to 5.0.7
Unixware 7.x
Linux 2.0 to 2.6.x
Mac OS X Server
OpenBSD 2.3 to 4.x
FreeBSD 3.3 to 8.0
NetBSD 1.3 to 5.x
HP/UX 11i to 11i v3
Minix 3.x
Mac OS X 10.0 to 10.7 (Darwin)
OpenServer 6.x
OpenSolaris 2008.05 and later


1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007
Michael Stonebraker
Michael Stonebraker
2015年获图灵奖
NonStop SQL
Sybase SQL Server
Microsoft SQL Server
Adaptive Server Enterprise
IBM
NETEZZA
Postgres
PostgreSQL
Illustra
Greenplum
Informix
Vertica Analytic Database
IBM DB2
Express Embedded Database
Ted Codd
1923－2003
MySQL
Michael Widenius
2001-
MySQL
MariaDB
PERCONA
MySQL Cluster
InnoDB
NDB Cluster
Oracle Database
Firebird
1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007

中国移动 11:53 AM

CCCF专栏 | 操作系统的兴衰

原创：万赟 中国计算机学会 8月12日

CCF 点击上方蓝字 关注我们

谷歌公司暂时中断对华为手机提供安卓操作系统的服务，这将导致华为无法使用安卓手机操作系统。随后传来华为将使用之前已经开发的鸿蒙操作系统来取代安卓作为反制对策。这一措施能否起到预期的效果？本文将通过计算机操作系统的发展历史来分析它的有效性。

专栏 中国计算机学会通讯 第15卷 第8期 2019年8月

关键词：操作系统 操作系统发展史 鸿蒙

2019年5月美国政府对华为公司进行制裁，使得谷歌公司暂时中断对华为手机提供安卓操作系统的服务，这将导致华为无法使用安卓手机操作系统。随后传来华为将使用之前已经开发的鸿蒙操作系统来取代安卓作为反制对策。这一措施能否起到预期的效果？本

中国移动 11:53 AM

中国计算机学会

IBM的OS/360

在计算机诞生之初的十几年时间里，业界对操作系统的必要性没有达成共识。大多数计算机公司和企业用户把计算机视为做固定的计算工作的专用机器，比如会计和生产管理。给计算机添加一个操作系统只会耗费更多的存储和计算资源，并不能提高效率。

后来随着计算机用途的增多，人们逐渐意识到为每一台新的计算机重新编写相同功能的应用软件是一件低效的工作。添加操作系统可以让用户更加方便地用高级编程语言编写程序和管理计算机，应用程序也可以移植到升级后的硬件上，大大方便了应用程序的开发、使用和维护。前瞻到这一趋势，IBM决定为其第一款多功能大型机S/360开发配备后来软件工程领域颇具传奇色彩的OS/360操作系统。

OS/360操作系统的设计和研发是当时计算机

中国移动 11:53 AM

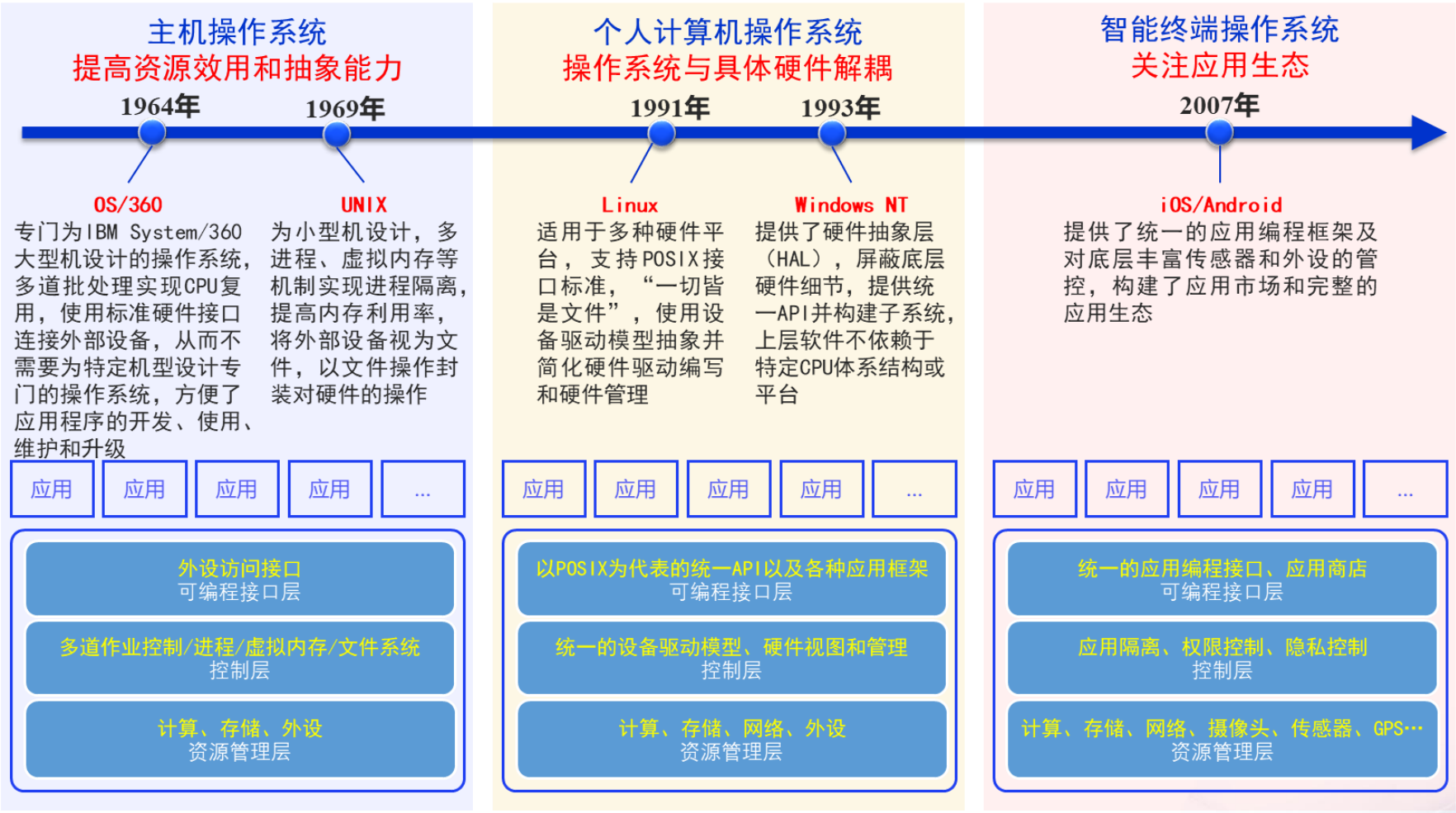
中国计算机学会

UNIX与开源运动

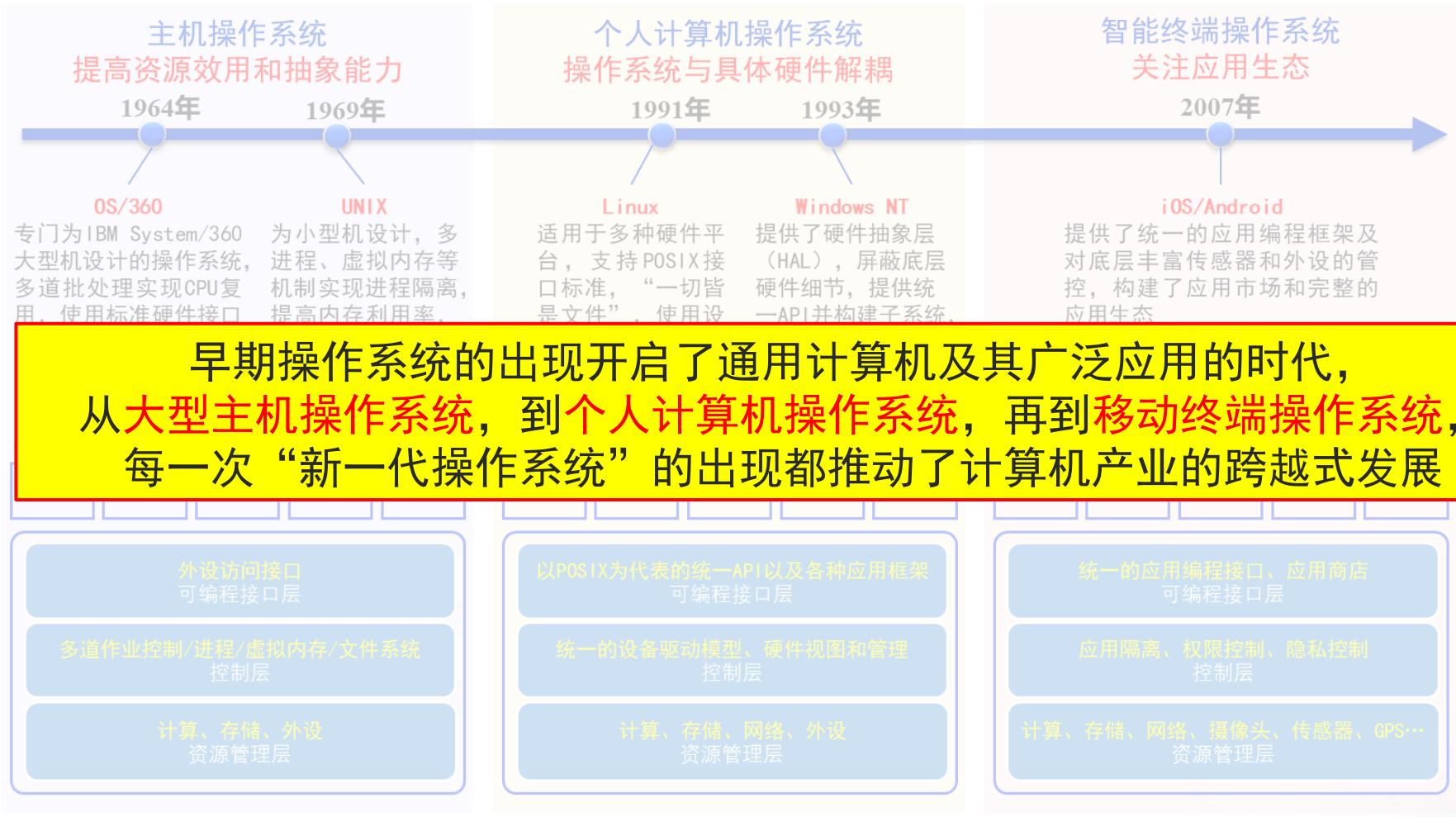
如果说OS/360是一个精雕细琢的系统工程结晶，UNIX则是一个无心插柳的赤子之作。它出现的背景是麻省理工学院、通用电气公司和贝尔实验室三方合作但是以失败告终的大型机操作系统Multics。尽管尝试失败，Multics很多新颖的设计启发了当时参与研发的贝尔实验室的汤普森、里奇和柯尼根。汤普森利用闲暇时间在实验室的通用电气大型机上编写了一个小游戏“太空旅行”。但是每次运行这个程序需要花费75美元的机时，而有一台大家不太用到的小型机PDP-7不但有非常好的显示器，而且机时可以不计。于是他们决定把游戏移植到这台小型机上。

在移植过程中，因为游戏的需要，他们需要设计一套基于PDP-7的文件处理系统。很快他们发现其实可以在此基础上为PDP-7编写一个分时操作系统，于是花了几个星期的时

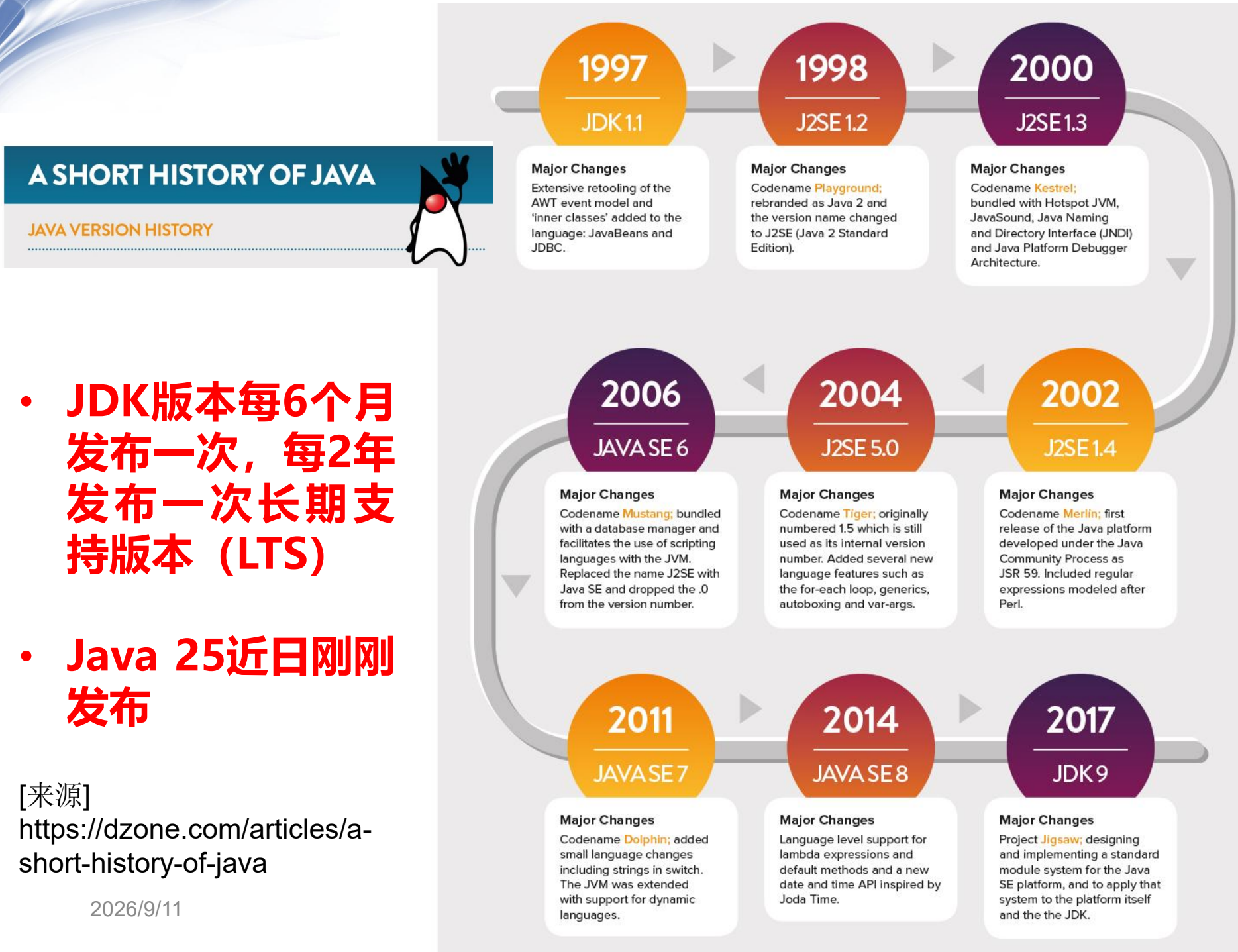
操作系统与计算机的协同发展



操作系统与计算机的协同发展



早期操作系统的出现开启了通用计算机及其广泛应用的时代，
从大型主机操作系统，到个人计算机操作系统，再到移动终端操作系统，
每一次“新一代操作系统”的出现都推动了计算机产业的跨越式发展



- **JDK版本每6个月发布一次，每2年发布一次长期支持版本（LTS）**
- **Java 25近日刚刚发布**

[来源]
https://dzone.com/articles/a-short-history-of-java



中国科学院大学
University of Chinese Academy of Sciences

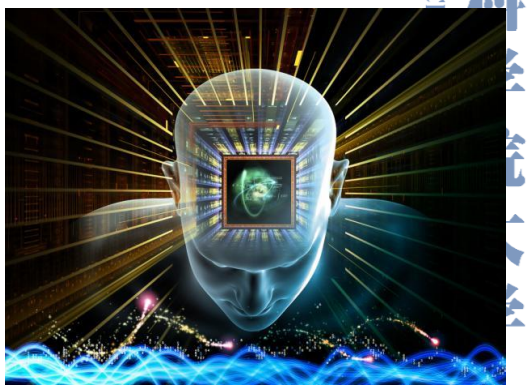
控制软件系统的复杂性

计算机软件领域的努力

人类程序员的局限性

- 人类在处理复杂问题时存在能力局限

  心理学家认为，一个人能够同时理解的最大信息数量是7±2个



- 复杂系统的设计和开发不能依赖天才和英雄

  程序员的十层楼[1]：

  菜鸟、大虾、牛人、大牛、专家、学者、大师、科学家、大科学家、大哲、“神”

- 显著增加软件系统的复杂性 vs 人类有限的能力

  # 专业分工——需求分析师、系统设计师、程序员、测试人员...

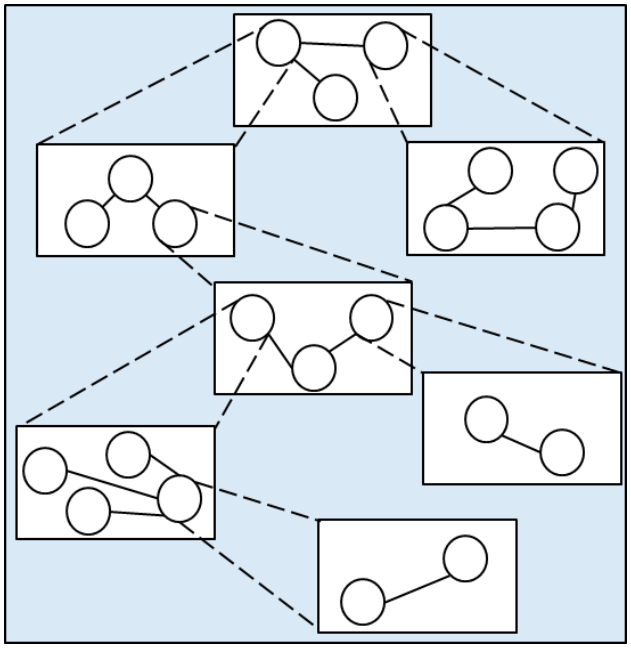
  # 指导思想——结构化、面向对象、敏捷开发、测试驱动、持续集成、开源...

  # 语言工具——建模语言、编程语言、开发环境（工具）、源码管理工具、测试工具...

[1] 周伟明. 程序员的十层楼. 程序员, 2009(6):134-136.

从混沌到有序——分而治之

- “控制复杂性的技巧我们从远古时代就知道了，即分而治之”



“算法”结构?

——面向过程的程序设计思想

“对象”结构?

——面向对象的程序设计思想

面向过程的分解——结构化程序设计思想

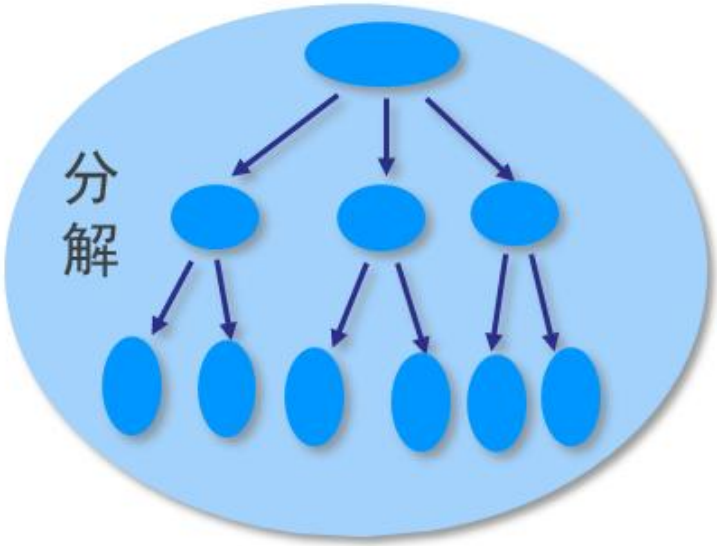


EDSGER WYBE DIJKSTRA
Netherlands - 1972



DONALD("DON") ERVIN KNUTH
United States - 1974

面向功能结构的结构化范型





ANTONY ("TONY") R. HOARE
United Kingdom - 1980

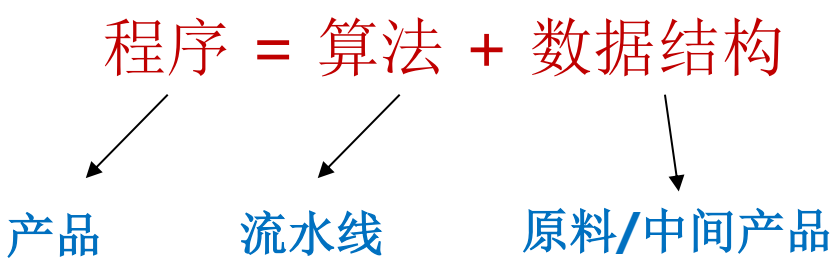


NIKLAUS E. WIRTH
Switzerland - 1984

面向过程的分解——结构化程序设计思想

- “自顶向下、逐步细化、模块化”

  # 系统程序被分解为若干子程序，子程序还可以进一步分解，最终系统程序具有了树的形态，本质是以过程为中心，进行算法分解



面向过程的分解——结构化程序设计思想

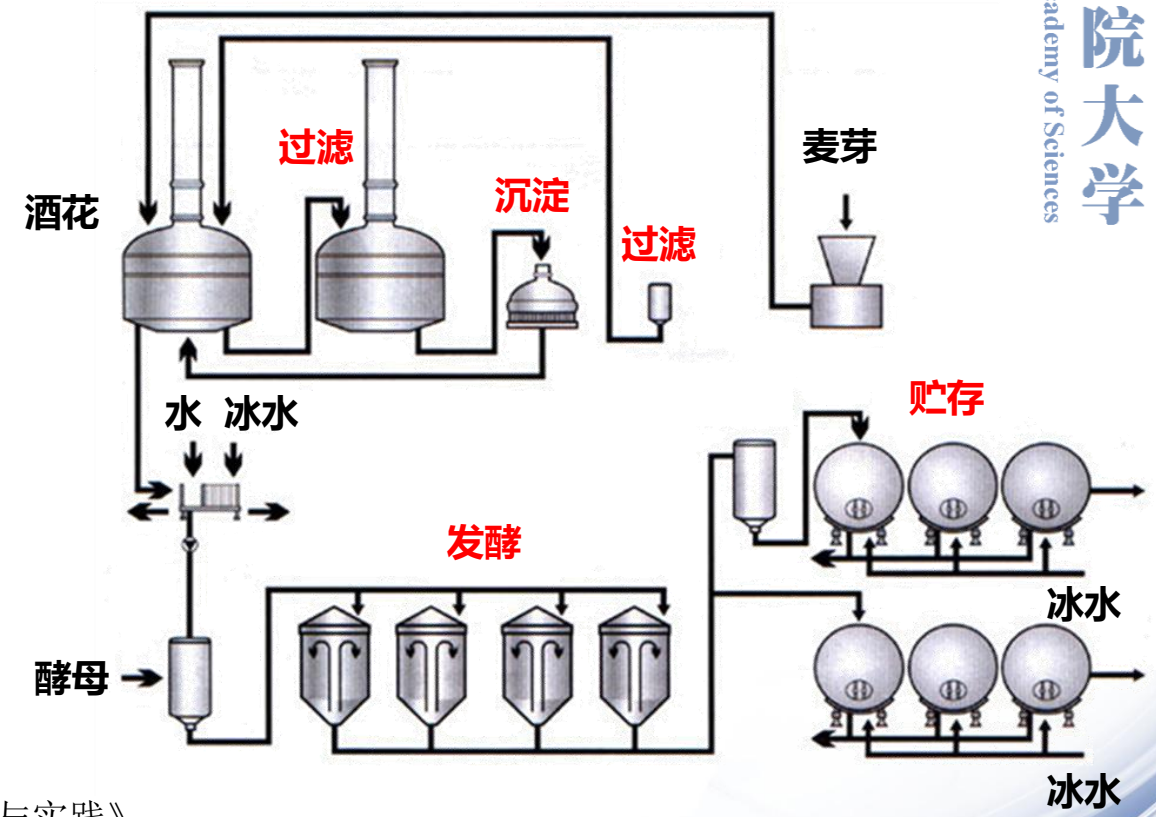
面向过程的啤酒生产线

数据

麦芽、酒花、酵母、水、冰水…

算法

过滤、煮沸糖化、沉淀、发酵、贮存…



[来源]《面向对象葵花宝典-思想、技巧与实践》

结构化程序设计思想面临的挑战

- 面向过程的软件系统，算法、数据相对固定，难以变更

啤酒厂老板对工程师说：
你把生产线改一下，酿醋，月底投产





面向对象思想的奠基人

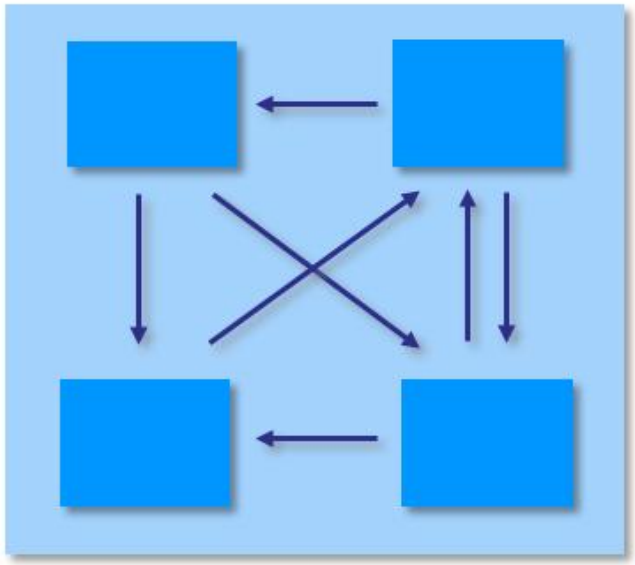


OLE-JOHAN DAHL
Norway - 2001



KRISTEN NYGAARD
Norway - 2001

面向问题结构的对象化范型





ALAN KAY
United States - 2003



BARBARA LISKOV
United States - 2008

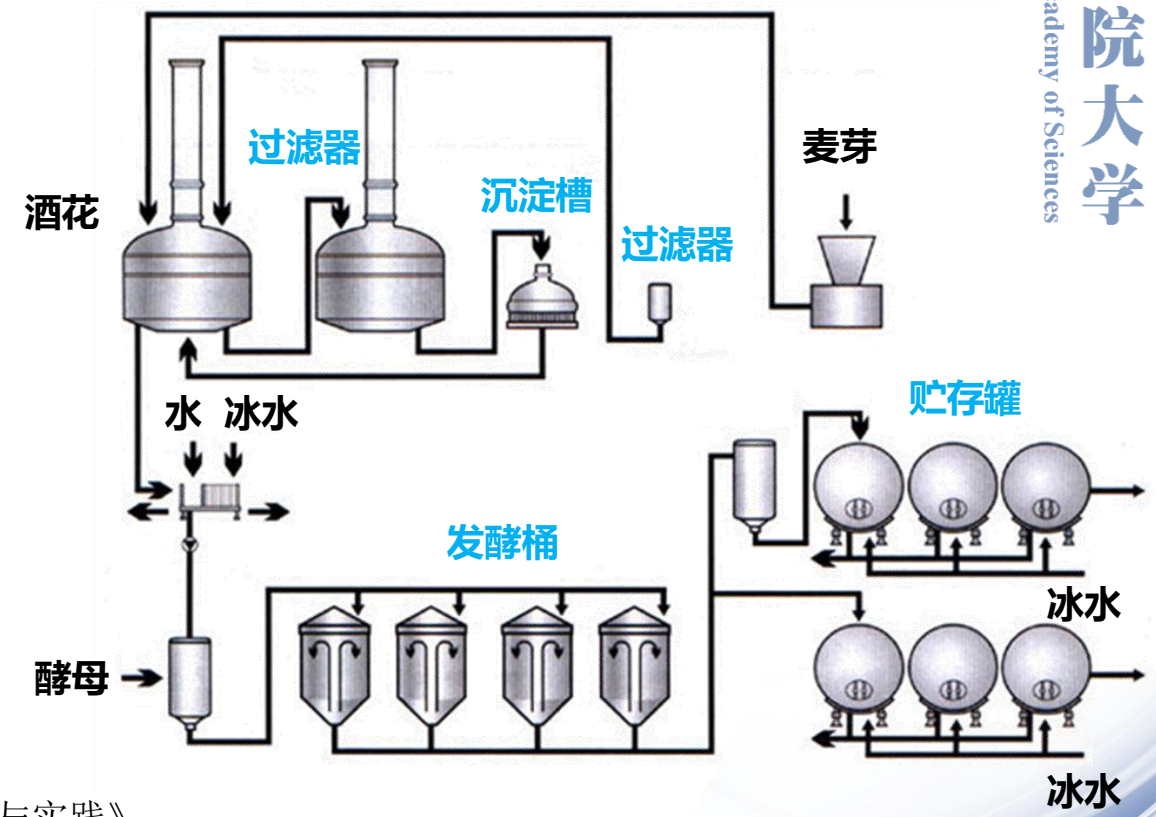
面向对象的分解——对现实世界的模拟

面向对象的啤酒生产线

对象

过滤器、煮沸糖化锅、沉淀槽、发酵桶、贮存罐…

程序 = 对象 + 交互



[来源]《面向对象葵花宝典-思想、技巧与实践》

面向对象的分解——对现实世界的模拟

- 再举个例子——五子棋程序

1、开始游戏
2、黑子先走
3、绘制画面
4、判断输赢
5、轮到白子
6、绘制画面
7、判断输赢
8、返回步骤2
9、输出最后结果

过程视角

1、对象：黑/白方，负责控制棋子
2、对象：棋盘，负责绘制画面
3、对象：裁判，负责判定犯规、输赢等

对象视角

为什么要面向对象 cont.

- 采用“面向对象”思想分解复杂系统的优点

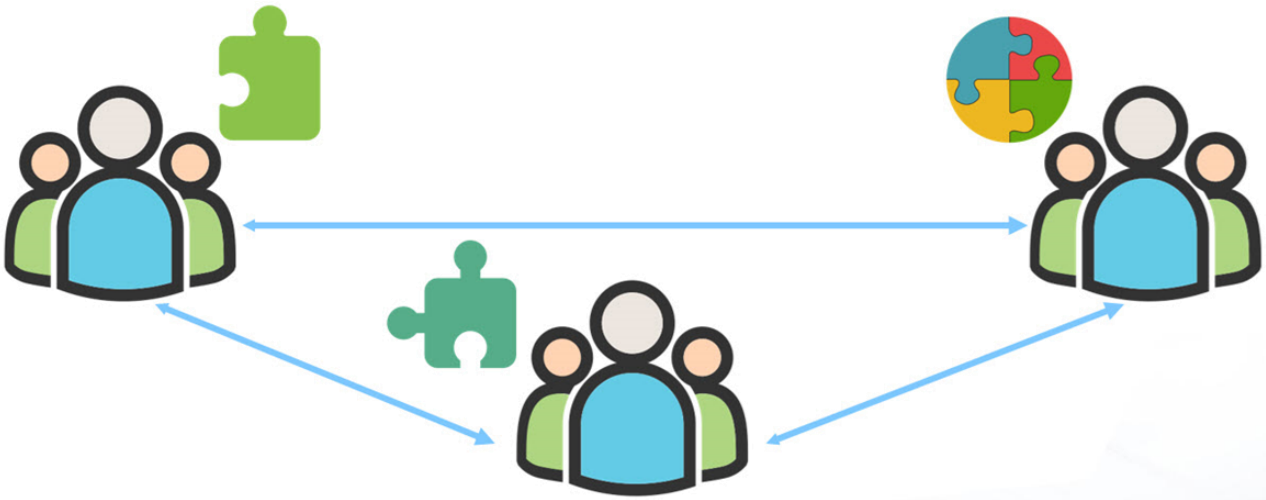
  # 共性的抽象更容易被复用

  编写更少的代码，保障成本和开发进度

  # 稳定的中间状态更容易演进

  需求的修改不会导致抛弃原来系统或重新设计

  …



小结

- “工业级”软件系统，本质是复杂系统
  # 问题域复杂，开发管理复杂，“软件定义一切”的诱惑，难以刻画离散系统行为
- 复杂系统的“特质”
  # 层次结构，相对基础，关注点分离（分而治之），共同模式，持续演变（由稳定的“简单”到“稳定”的复杂）
- 复杂系统的有效组织
  # 现代工业的启示
  # 复杂系统的“类型”结构和“对象”结构
- 如何控制软件系统的复杂性
  # 面向过程的思想 vs 面向对象的思想

中国科学院大学
University of Chinese Academy of Sciences



中国科学院大学
University of Chinese Academy of Sciences

Java程序语言基础（一）

导论

Java语言简介

- Java是Sun Microsystems公司推出的面向对象程序设计语言

  原名：Oak，1995年更名为Java

  发明者：James Gosling

  http://en.wikipedia.org/wiki/James_Gosling



- 2006年Sun Microsystems宣布Java开源

- 2010年Oracle收购Sun

ORACLE
Java

Java时间轴
http://oracle.com.edgesuite.net/timeline/java/

中国科学院大学
University of Chinese Academy of Sciences

Java语言特点

- 在Java语言白皮书中，Java被描述为“是一种简单、面向对象、分布式、解释、健壮、安全、结构中立、可移植、高性能、多线程、动态的语言。

“A simple, object-oriented, network-savvy, interpreted, robust, secure, architecture neutral, portable, high-performance, multithreaded, dynamic language”

—James Gosling, Henry McGilton. The Java™ Language Environment， A White Paper.

中国科学院大学
University of Chinese Academy of Sciences



中国科学院大学
University of Chinese Academy of Sciences

部分特点

- 简单易学

  去除C++中不容易理解的部分（如指针）

  不需要维护内存分配

- 面向对象

  有利于程序的开发、复用和维护

- 平台无关性

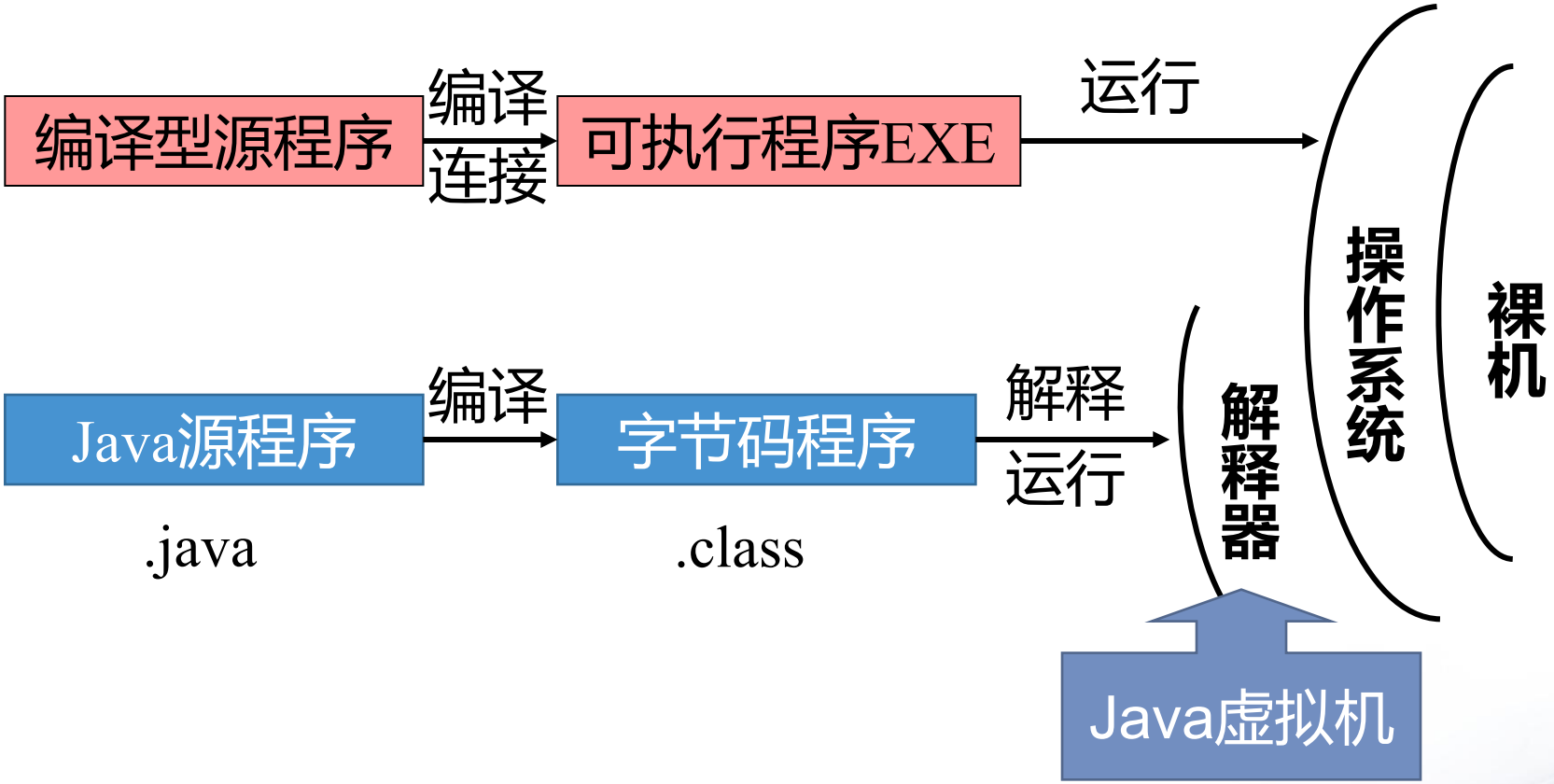
  Java编译器生成与体系结构无关的字节码，Java解释器得到字节码后，把它转换为目标平台的机器语言，使之能够在不同的平台运行

- 健壮性

  垃圾回收、异常机制

部分特点 cont.

Java是解释型的高级语言



Java程序开发、运行的基本过程

1. 利用编辑工具编写Java源程序

   源代码文件：类名.java

2. 利用编译器将源程序编译成字节码

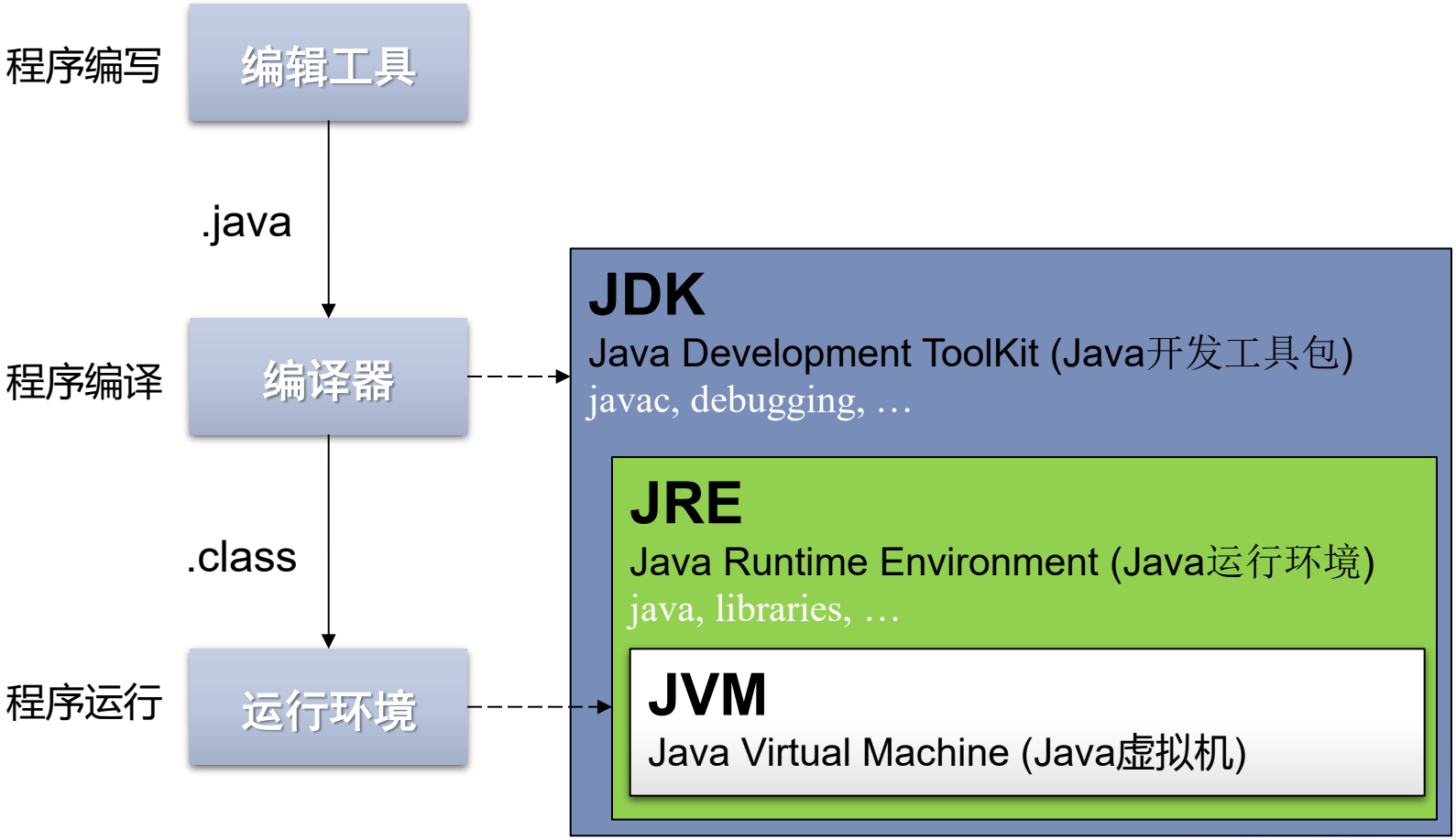
   字节码文件：类名.class

3. 利用Java虚拟机运行

   载入、代码校验、执行

中国科学院大学
University of Chinese Academy of Sciences

Java程序开发、运行的基本过程



安装并配置JDK

- 下载JDK

  http://www.oracle.com/technetwork/java/javase/downloads/index.html

- 配置环境变量

  JAVA_HOME

  JDK安装路径

  PATH

  ;%JAVA_HOME%\bin;%JAVA_HOME%\jre\bin

  CLASSPATH

  .;%JAVA_HOME%\lib;%JAVA_HOME%\lib\tools.jar

中国科学院大学
University of Chinese Academy of Sciences

第一个Java应用程序

import语句

```
class 主类名称
{
    public static void main(String[ ] args)
    {
        方法体
    }
}
```

示例



剖析Java程序代码

- 类名
- 主方法
- 语句
- 保留字
- 块
- 注释
- …

中国科学院大学
University of Chinese Academy of Sciences

类名

- 每个Java程序至少有一个类，类名首字母大写

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

中国科学院大学
University of Chinese Academy of Sciences

主方法

- 主方法是程序运行的“入口”

主方法不是类的必要组成，但如果要运行一个类，则该类需要包含一个主方法

除了参数名称，主方法的格式是固定的

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

语句

- 一条语句表示一个或一组行为
- 语句以“;”结束

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

中国科学院大学
University of Chinese Academy of Sciences

保留字

- 保留字，也称关键字，是对编译器有特定含义的单词

例如：当编译器看到单词class，就会明白class后面的单词就是这个类的

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

中国科学院大学
University of Chinese Academy of Sciences

块

- 一对大括号 { }

类块
方法块

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

注释

- 行注释
  行注释是在单行上最前面加两个斜杠（//）表示注释某行
- 段注释
  段注释是在一行或多行中用 /*与 */ 括住某一段表示注释某段
- javadoc注释
  javadoc注释是以 / 开始，以 */结束**

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

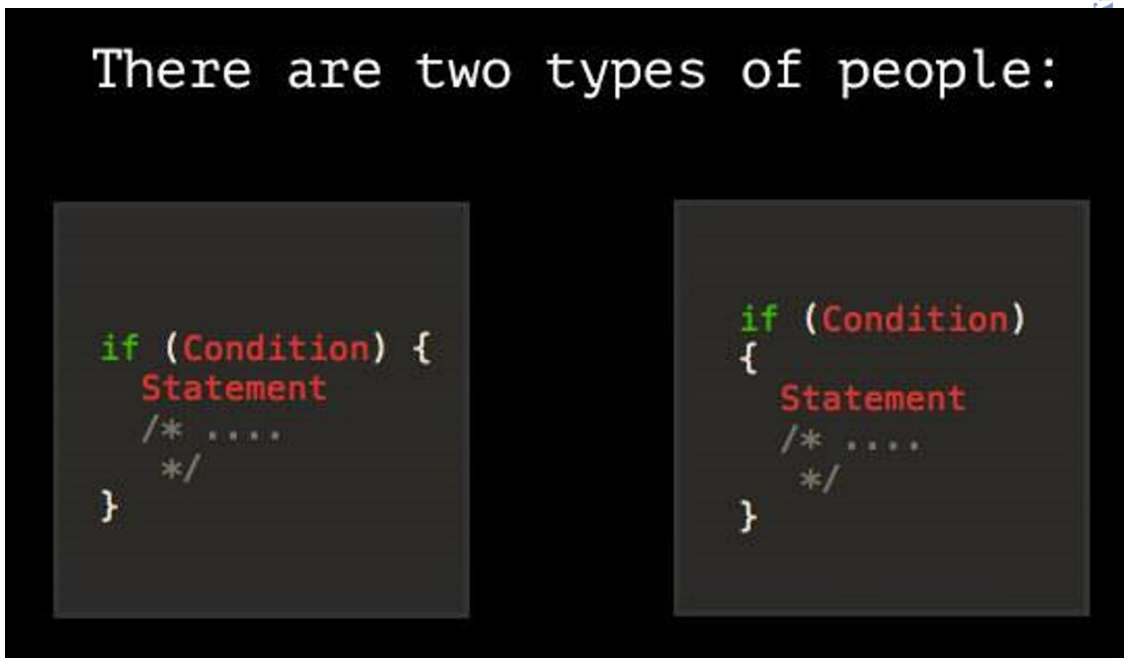
一些特殊符号

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

中国科学院大学
University of Chinese Academy of Sciences

程序编写风格

- 适当的注释
- 命名习惯
- 适当的缩进和空白
- 块的风格



程序错误

- 语法错误

  由编译器检测

- 运行错误

  导致程序不能正常运行

- 逻辑错误

  产生不正确的结果

中国科学院大学
University of Chinese Academy of Sciences

调试（debug）

- 程序的逻辑错误被称为bug，查找和更正错误的过程被称为debug
- 调试的一般方法是使用各种方法来逐步缩小bug所在范围

你可以手工跟踪程序（即通过读程序找错误）

也可以插入打印语句以显示变量的值或程序的执行流程

对于大型的、复杂的程序，最有效的调试方法是使用调试工具

中国科学院大学
University of Chinese Academy of Sciences



中国科学院大学
University of Chinese Academy of Sciences

Java程序语言基础（二）

基本数据类型、运算符、流程控制

数据类型

Name	Range	Storage Size
byte	-2^7 (-128) to 2^7-1 (127)	8-bit signed
short	-2^{15} (-32768) to $2^{15}-1$ (32767)	16-bit signed
int	-2^{31} (-2147483648) to $2^{31}-1$ (2147483647)	32-bit signed
long	-2^{63} to $2^{63}-1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
float	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
double	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754

数值运算符

Name	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction	34.0 - 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Remainder	20 % 3	2

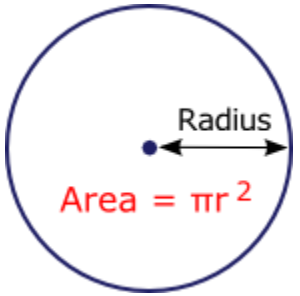
一个简单的程序示例

- 计算一个半径为特定值的圆形面积

半径 (radius) = 20

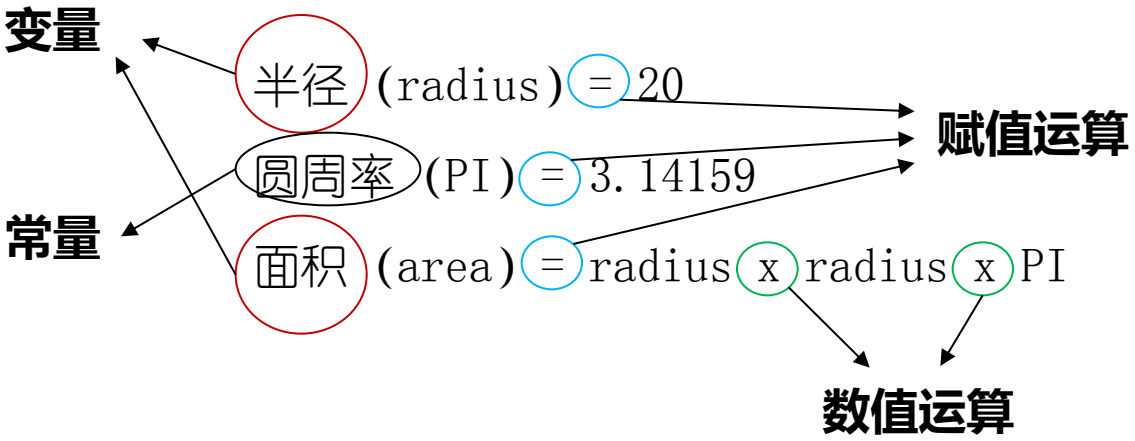
圆周率 (PI) = 3.14159

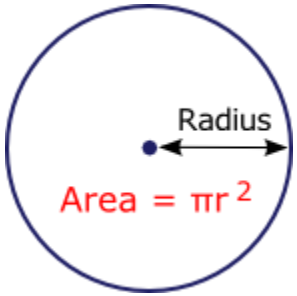
面积 (area) = radius x radius x PI



一个简单的程序示例

- 计算一个半径为特定值的圆形面积





示例L1.1



变量、常量的声明与赋值

变量

先声明，后赋值

变量数据类型　变量名;　　比如，double radius;

变量名 = 值;　　比如，radius = 20;

一步完成声明、赋值

变量数据类型　变量名 = 值;　比如，double radius = 20;

常量

final　常量数据类型　常量名 = 值; 比如，final double PI = 3.14159;

中国科学院大学 University of Chinese Academy of Sciences

从控制台读取输入

- 创建一个Scanner类型的对象

```
Scanner input = new Scanner(System.in);
```

- 选用合适的方法，将输入值转换为相应的数据类型

```
nextByte(), nextShort(), nextInt(), nextLong(), nextFloat(), nextDouble()
```

另外，还有

next()，返回String(字符串值)

nextBoolean()，返回boolean值

示例L1.2

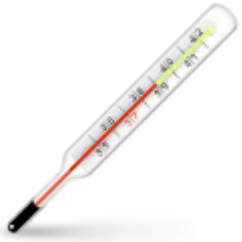


示例：温度转换

- 编写一个程序，将输入的摄氏温度转换成华氏温度

算数表达式

$$fahrenheit = \frac{9}{5} * celsius + 32$$



变量

double celsius——摄氏温度，控制台输入

double fahrenheit——记录华氏温度

示例L1.3



数值类型转换

- 考虑下面的语句：

```
int a = 9 / 5;          double a = 9 / 5;

int a = 9.0 / 5;        double a = 9.0 / 5;

int a = 9.0 / 5.0;      double a = 9.0 / 5.0;
```

中国科学院大学
University of Chinese Academy of Sciences

数值类型转换 cont.

- 类型拓宽

  将小范围类型的变量转换为大范围类型的变量

  隐式转换，程序可自动执行

  ```
  double d = 3;  (类型拓宽)
  ```

- 类型缩窄

  将大范围类型的变量转换为小范围类型的变量

  显示转换

  ```
  int i = (int)3.0;  (类型缩窄)
  int i = (int)3.9;  (截掉小数部分)
  ```

范围增加



Byte、short、int、long、float、double

流程控制——选择

- 在计算圆的面积时，如果半径的输入是一个负值，将出现不合理的计算结果。如何处理这个问题?

中国科学院大学
University of Chinese Academy of Sciences

boolean类型和运算符

- 程序中经常需要对两个值进行比较

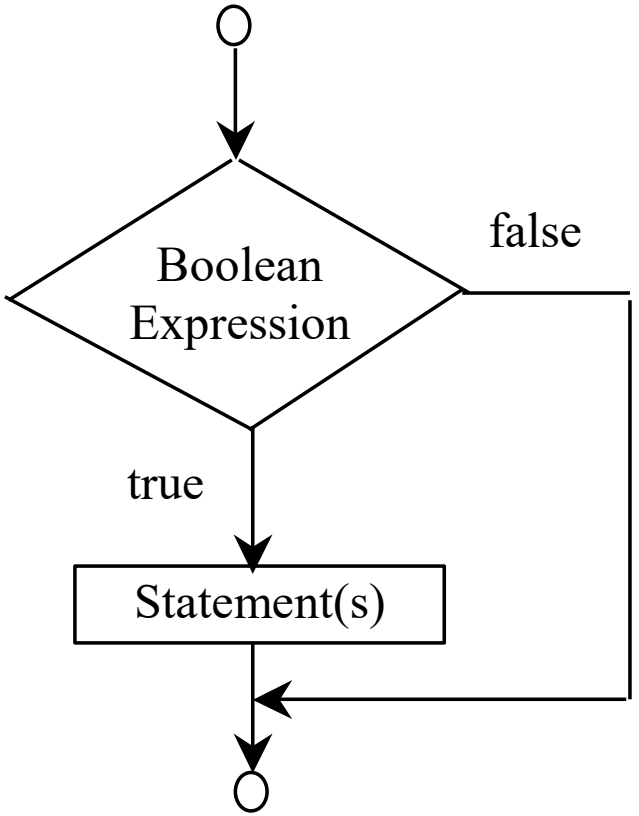
  例如： i 是否大于j

- Java提供了六种比较运算符（也被称作关系运算符），比较的结果是一个 Boolean值，true（真）或false（假）

运算符	***名称***
<	小于
<=	小于或等于
>	大于
>=	大于或等于
==	等于
!=	不等于

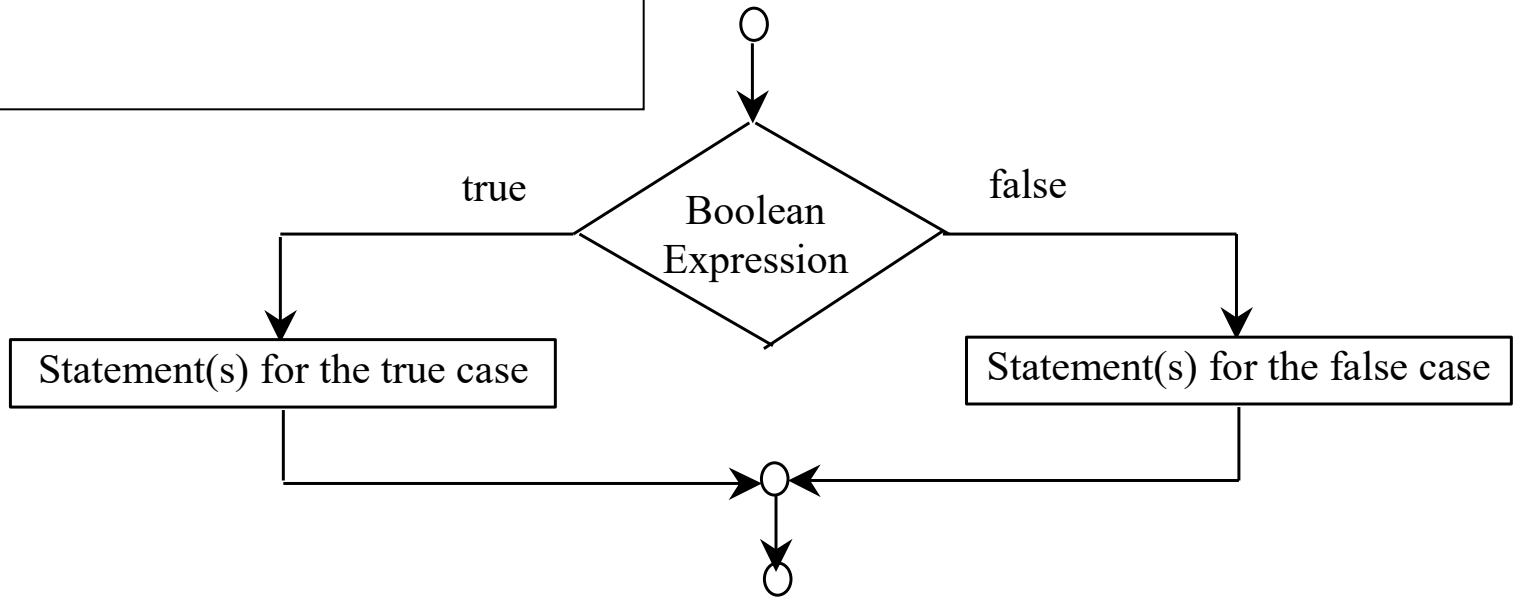
单向if语句

```
if（布尔表达式）{
   语句;
   …
}
```



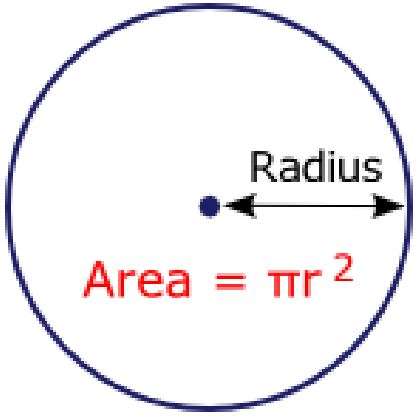
双向if语句

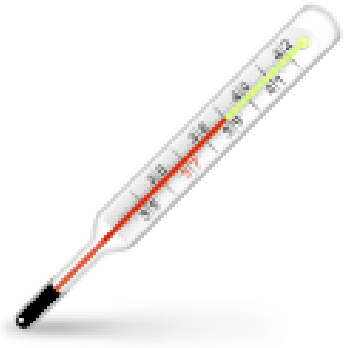
```
if（布尔表达式）{
   布尔表达式为真时执行的语句;
   …
} else {
    布尔表达式为假时执行的语句;
    …
}
```



中国科学院大学
University of Chinese Academy of Sciences

示例：输入的检查





示例L1.4



示例：对账

- 对比交易所反馈的成交额与券商自己统计的成交额

  交易所发给券商当天的成交总额

  券商自己系统里有每一笔成交的金额

  券商需要每天进行汇总、对账，确定当日记录的成交金额是否正确

示例L1.5



关于浮点运算的精度问题

- 浮点数据（double, float）的二进制表达不精确

要把小数装入计算机，总共分几步？

第一步：转换成二进制

第二步：用二进制科学计算法表示

第三步：表示成 IEEE 754 形式

在上面的第一步和第三步都有可能**丢失精度**！

比如：0.1 到 0.9 的 9 个小数中，只有 0.5 可以用二进制精确的表示

关于浮点数的精度
http://justjavac.iteye.com/blog/1725977
http://justjavac.iteye.com/blog/1724438

中国科学院大学
University of Chinese Academy of Sciences

逻辑运算符

- 如何进行多个条件的判断?

运算符	***名称***
!	逻辑非
&	逻辑与
|	逻辑或
^	逻辑异或
&&	短路与（第一个操作数为假则不判断第二个操作数）
||	短路或（第一个操作数为真则不判断第二个操作数）

中国科学院大学
University of Chinese Academy of Sciences

逻辑运算符——真值表

a	b	!a	a&b	a|b	a^b	a&&b	a||b
true	**true**	**false**	**true**	**true**	**false**	**true**	**true**
true	**false**	**false**	**false**	**true**	**true**	**false**	**true**
false	**true**	**true**	**false**	**true**	**true**	**false**	**true**
false	**false**	**true**	**false**	**false**	**false**	**false**	**false**

中国科学院大学
University of Chinese Academy of Sciences

示例：判断闰年

- 根据用户输入年份，判断是否为闰年

某年可以被4整除但不能被100整除，或者能被400整除，即为闰年

```
(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)
```

示例L1.6



Switch语句

- 用于针对某表达式的不同值，进行条件判断，从多个程序语句中，选择其中一个执行

```
switch (表达式或变量) {
   case 条件值1:
        语句块1;
        break;
   case 条件值2:
        语句块2;
        break;
   …
    default:
        语句块N;
        break;
}
```

规则

1. 表达式的值必须是下述几种类型之一：

   int，byte，char，short

2. case子句中的条件值必须是常量，且所有case子句中的值应是不同的；

3. default子句是可选项；

4. break语句用来在执行完一个case分支后使程序跳出switch语句块；否则会继续执行下去。

示例L1.7



流程控制——循环

- 如何打印一个字符串（例如：“Welcome to Java!”）100次?

100次

```
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");

...

...

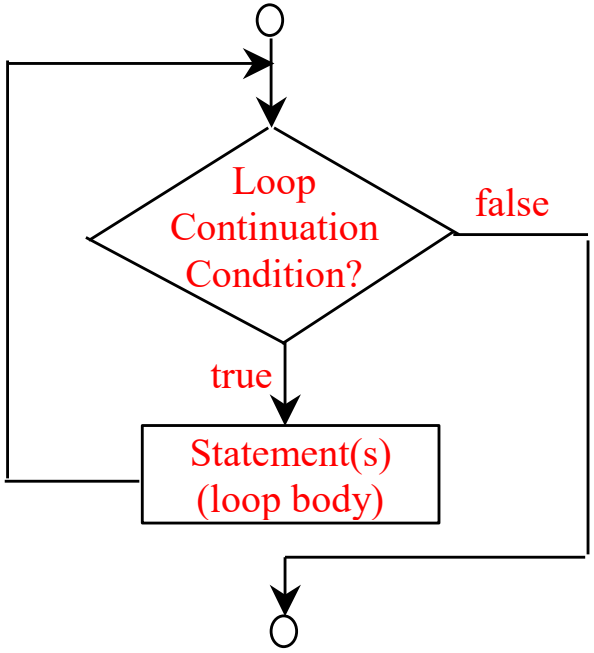
...
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
```

中国科学院大学
University of Chinese Academy of Sciences

while循环

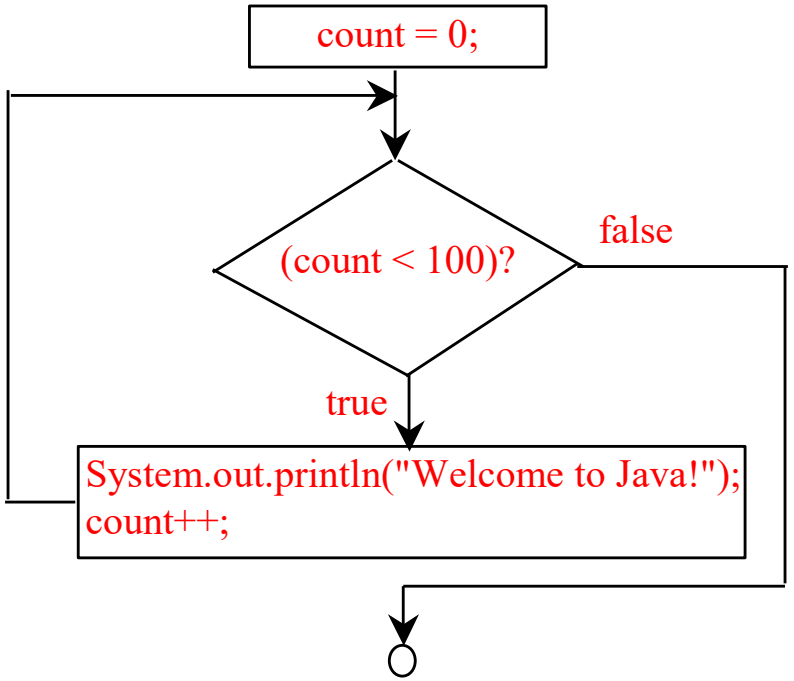
布尔表达式

```
while (循环继续条件) {
  //循环体
  循环继续条件为真时执行的语句;
  …
}
```



while循环 cont.

```
int count = 0;
while (count < 100) {
    System.out.println("Welcome to Java!");
    count++;
}
```



示例L1.8.1



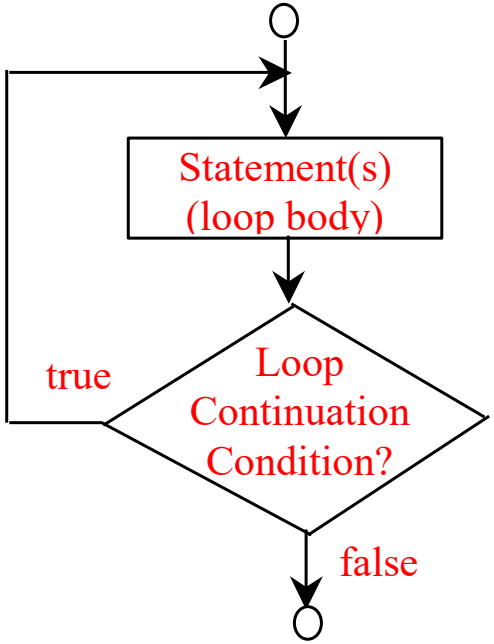
自增和自减运算符

运算符	*名称*	*说明*
++var	前置自增	变量var的值加1，使用var增加后的新值
--var	前置自减	变量var的值减1，使用var减少后的新值
var++	后置自增	变量var的值加1，使用var原来的值
var--	后置自减	变量var的值减1，使用var原来的值

- 若运算符是在变量的前面，则该变量自增1或自减1，然后返回的是变量的新值
- 若运算符是在变量的后面，则该变量自增1或自减1，然后返回的是变量的旧值

do-while循环

```
do {
  //循环体
 语句;
  …
} while (循环继续条件);
```



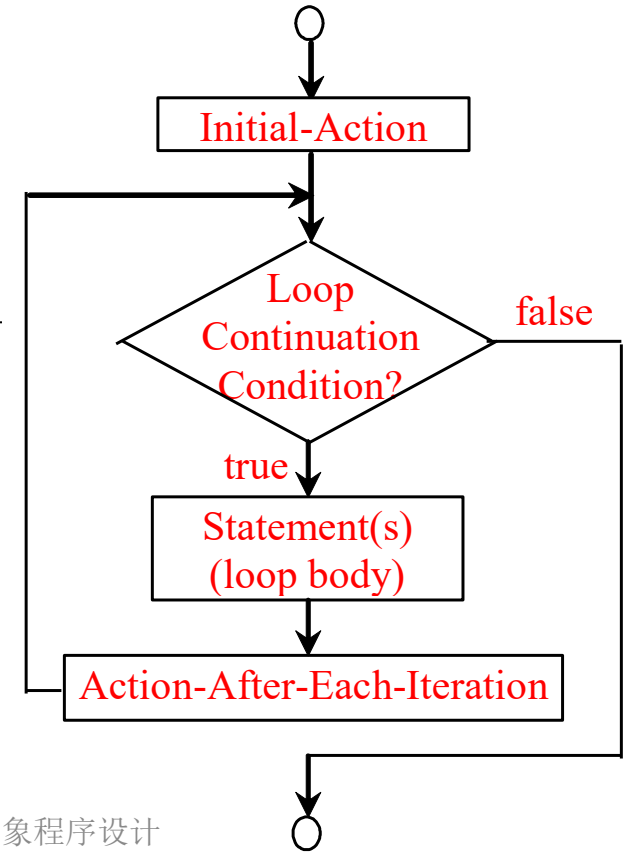

```
int count = 0;
do {
    System.out.println("Welcome to Java!");
    count++;
} while (count < 100);
```

示例L1.8.2



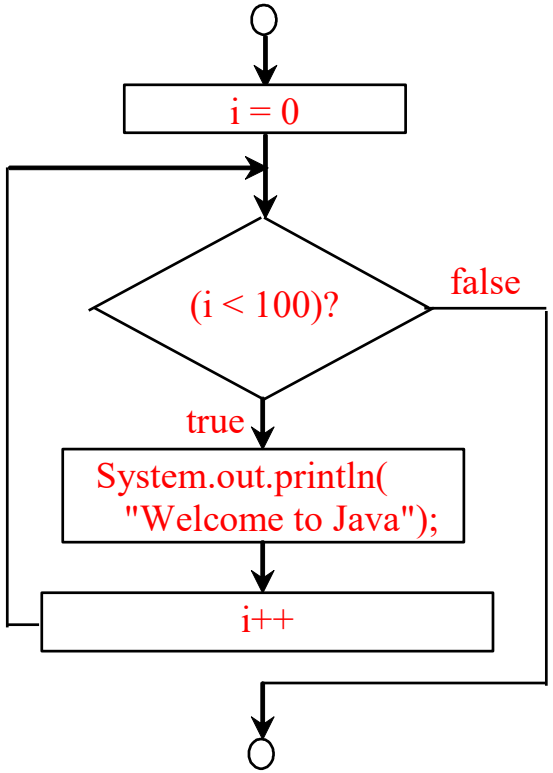
for循环

```
for（初始操作；循环继续条件；每次迭代后的操作）{
   //循环体
   语句;
   …
}
```



for循环 cont.

```
int i;
for (i = 0; i < 100; i++) {
    System.out.println("Welcome to Java!");
}
```



示例L1.8.3



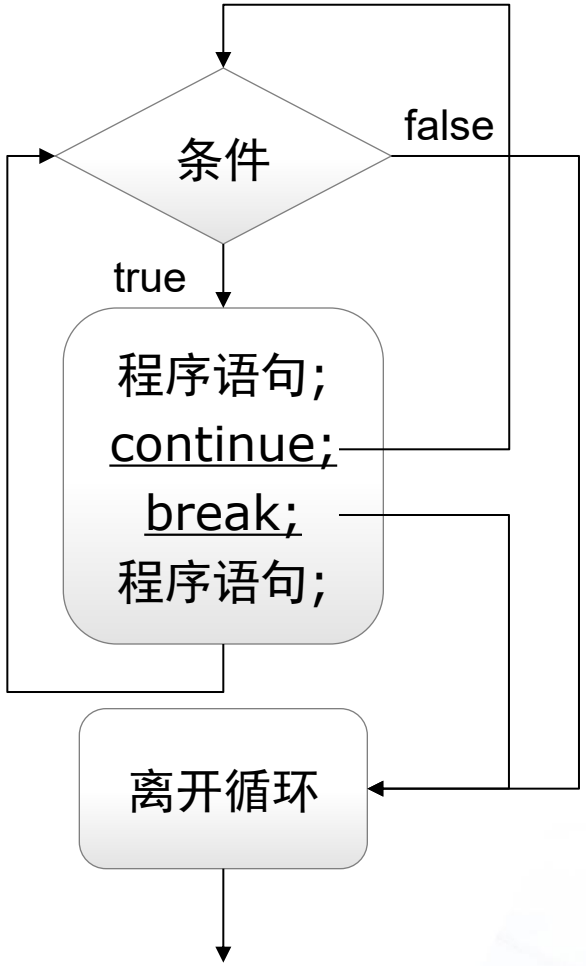
跳转语句——break和continue

- continue语句

  用于循环结构内，终止当前这一轮循环

- break语句

  用于循环结构内，跳出循环



使用哪种循环语句?

- 使用自己觉得最自然、最舒适的那种循环语句

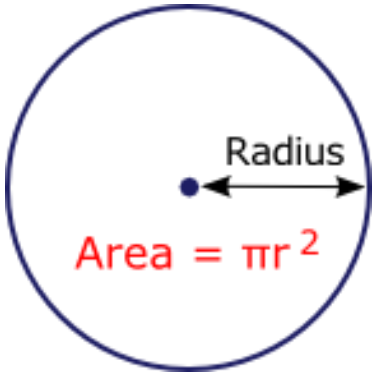
如果重复的次数是已知的，采用for循环

如果重复的次数是未知的，采用while循环

如果在检查继续条件前必须执行循环体，采用do-while循环替代while循环

中国科学院大学
University of Chinese Academy of Sciences

示例：next round? retype?





示例L1.9



示例：乘法表（嵌套循环）

```
                Multiplication Table
     1   2   3   4   5   6   7   8   9
-----------------------------------------
1  |  1   2   3   4   5   6   7   8   9
2  |  2   4   6   8  10  12  14  16  18
3  |  3   6   9  12  15  18  21  24  27
4  |  4   8  12  16  20  24  28  32  36
5  |  5  10  15  20  25  30  35  40  45
6  |  6  12  18  24  30  36  42  48  54
7  |  7  14  21  28  35  42  49  56  63
8  |  8  16  24  32  40  48  56  64  72
9  |  9  18  27  36  45  54  63  72  81
```

示例L1.10



课后练习

震爷手气壮

设计一个简单的微信群“拼手气”红包程序。要求可以根据输入的总金额、红包数量，输出每个红包的随机金额，其中，金额以“元”为单位，保留两位小数点。





中国科学院大学
University of Chinese Academy of Sciences

类和对象
一切都是对象

类（Class）

> “我们需要将自然界分解，组织成各种概念，并按其含义进行分类，否则我们根本无法交谈”
>
> ——Benjamin Lee Whorf

- 将“事物”抽象为程序“实体”，如何进行分类表达？

* 类（Class）——类是具有共同属性和行为的“实体”集合

比如：圆形

* 类 = {成员数据，成员方法}

数据，反映实体具有的状态、属性。比如，圆形的半径

方法，作用于数据上的操作。比如，计算圆形的面积

如何定义一个类

- 定义若干成员变量——数据域
- 定义若干成员方法

构造方法——用于创建类的对象（object）

一般方法——用操纵对象

中国科学院大学
University of Chinese Academy of Sciences

如何定义一个类 cont.

```
public class Circle {
  /** The radius of this circle */
  double radius = 1.0;

  /** Construct a circle object */
  Circle() {
  }

  /** Construct a circle object */
  Circle(double newRadius) {
    radius = newRadius;
  }

  /** Return the area of this circle */
  double getArea() {
    return radius * radius * 3.14159;
  }
}
```

数据域

构造方法

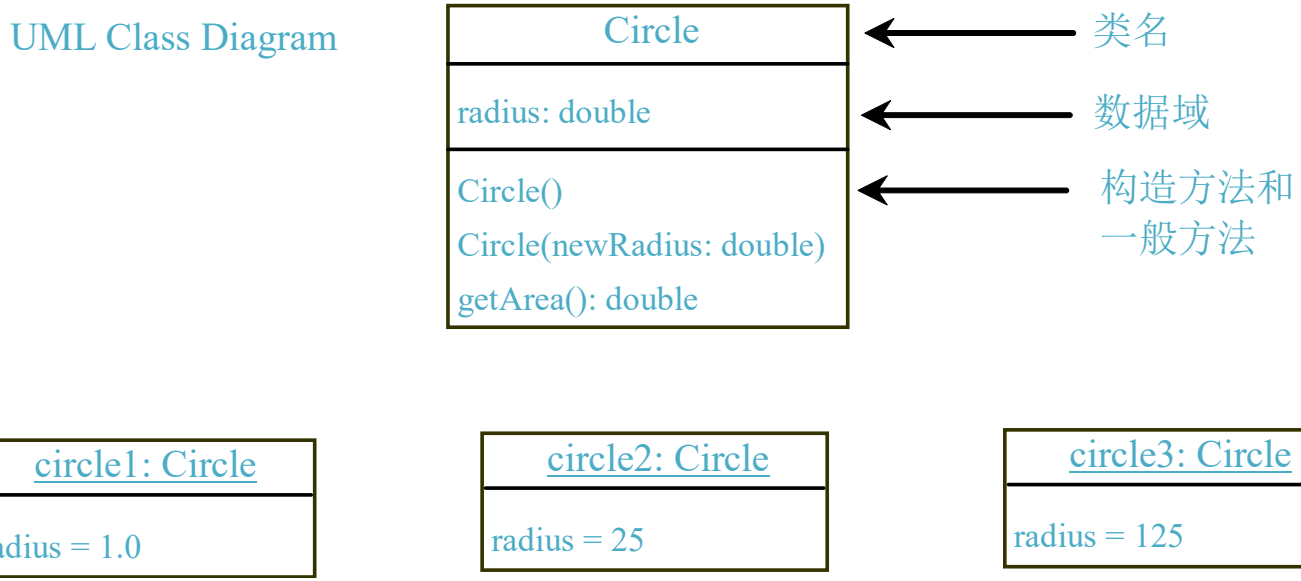
方法

中国科学院大学
University of Chinese Academy of Sciences

对象（Object）

- 对象是类的实例

  比如：半径分为10、25、125的3个圆形对象



构造方法

- 构造方法是一种特殊的方法，调用它来构造对象

  构造方法必须具备和它所属类相同的名字

  构造方法没有返回类型，甚至没有void

```
//默认构造方法：没有参数、方法体为空
Circle() {
}

//有参数的构造方法
Circle(double newRadius) {
   radius = newRadius;
}
```

创建对象

- 定义对象的“引用”（一种特殊的变量）

  例如: `Circle circle1;`

- 使用“new + 构造方法”创建对象

  例如: `Circle circle1 = new Circle( );`

  例如: `Circle circle2 = new Circle(1.0);`

“创建一个对象”、“实例化一个对象”

“构造一个对象”、“new一个对象”

...

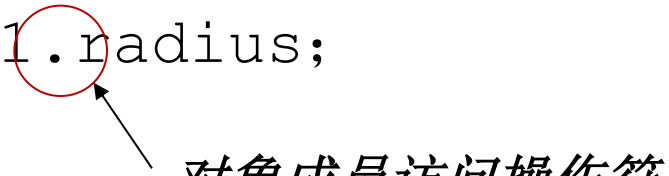
中国科学院大学
University of Chinese Academy of Sciences

对象的使用——用“引用”操纵对象

- 通过对象的“引用”访问对象

访问对象的数据

```
circle1.radius;
```



调用对象的方法

```
circle1.getArea();
```

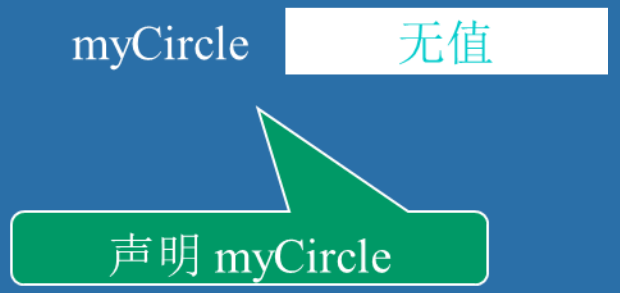
*注意：与Math，System等类方法的调用区别—静态方法

对象的使用——用“引用”操纵对象 cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



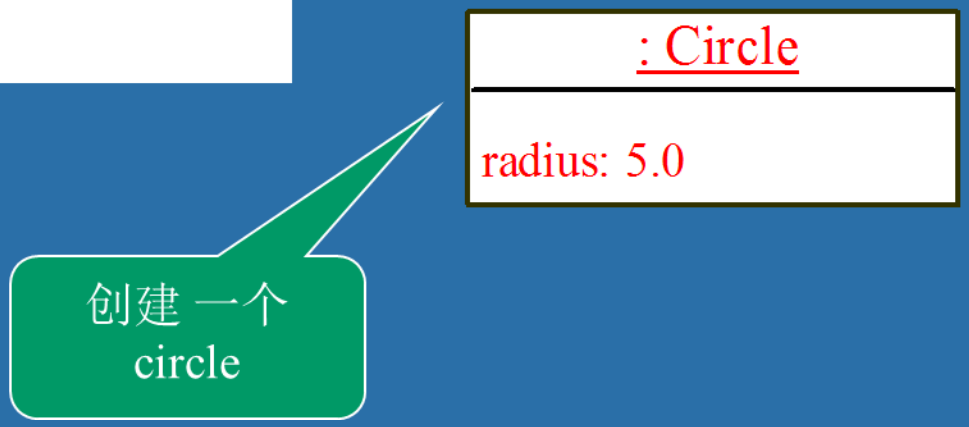
对象的使用——用“引用”操纵对象 cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

myCircle 无值



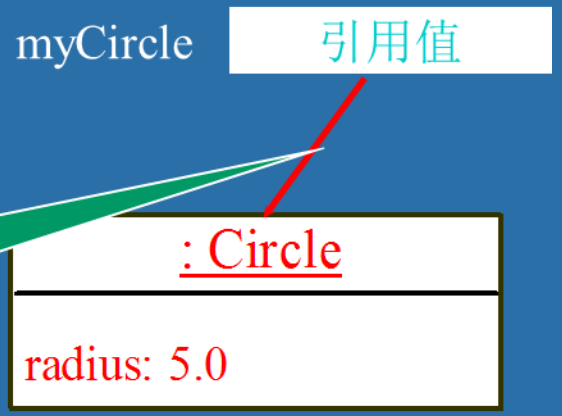
对象的使用——用“引用”操纵对象 cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

将对象引用赋值给myCircle

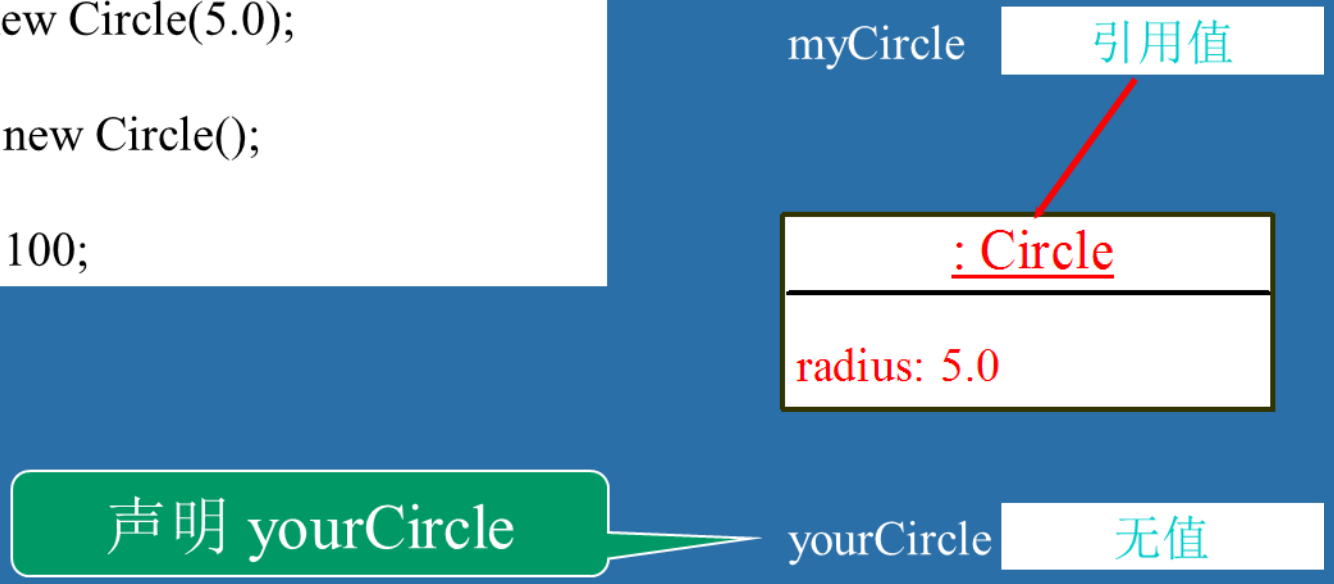


对象的使用——用“引用”操纵对象 cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

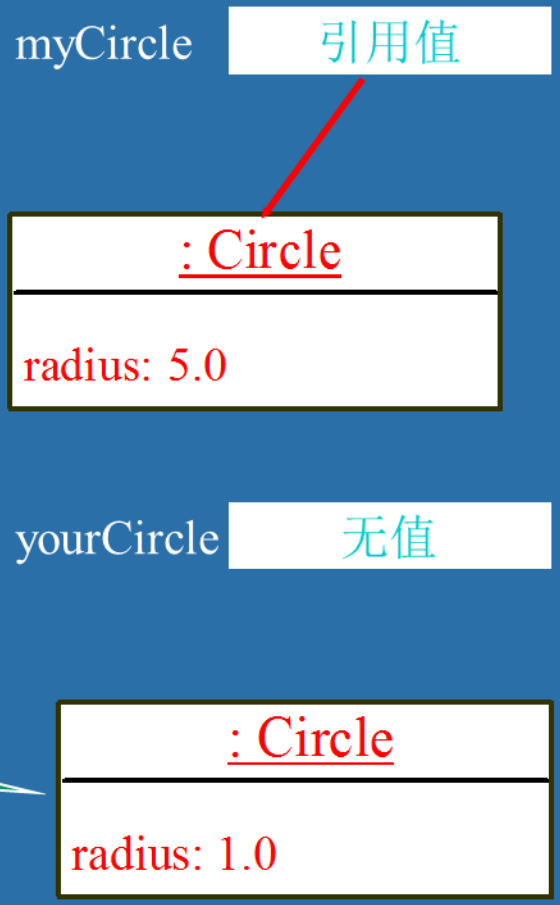


对象的使用——用“引用”操纵对象 cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

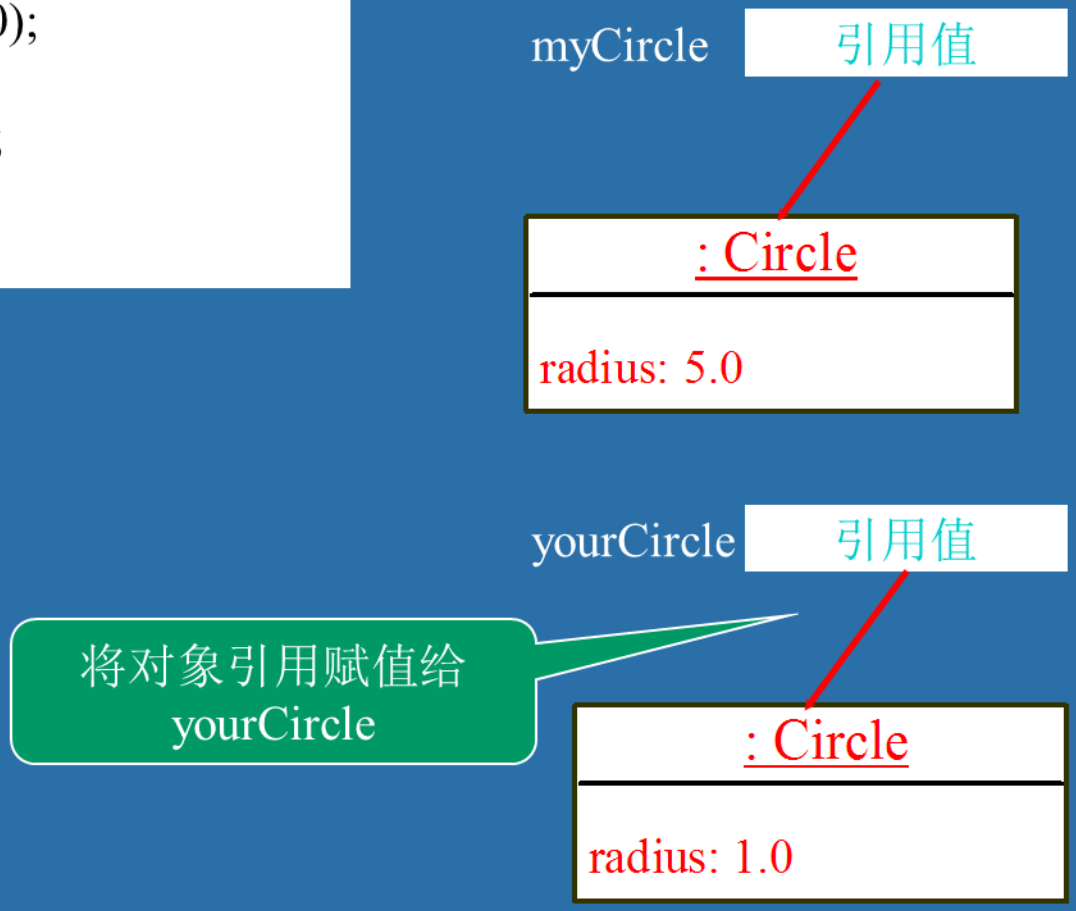


对象的使用——用“引用”操纵对象 cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

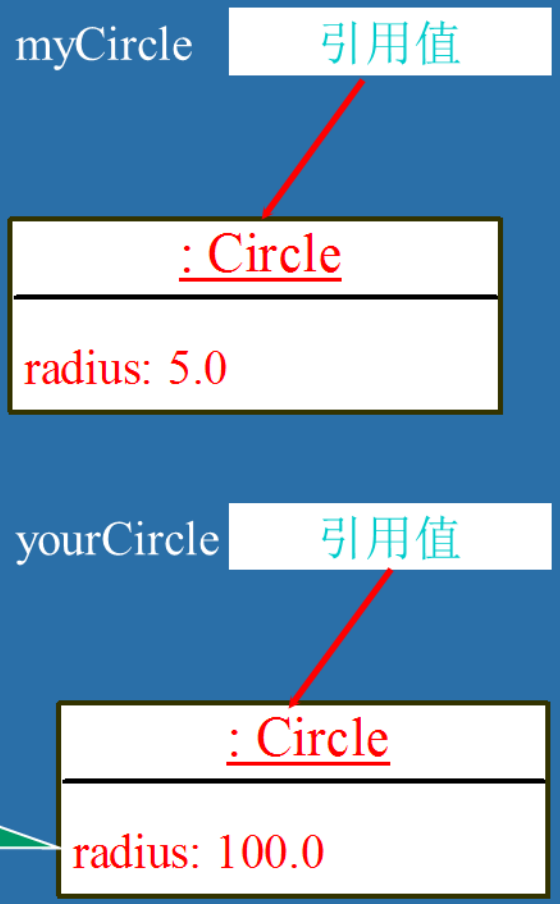


对象的使用——用“引用”操纵对象 cont.

```
Circle myCircle = new Circle(5.0);

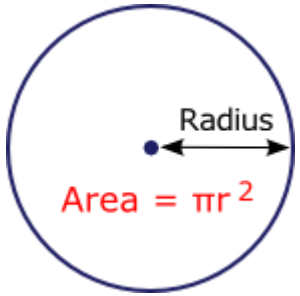
Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



示例：定义一个类，创建/使用它的对象

- Circle类



示例L2.1



- TV类



示例L2.2



Java中现成的常用类——String

- String，用于表示一串字符，不是基本数据类型，而是引用类型
- 如何声明一个String类型的变量并赋值?

  方式1: String message = new String("i am a message");

  方式2: String message = "i am a message";

  方式3: String message = "i " + "am " + "a message";

- 常用方法

  大小写转换, toUpperCase(); toLowerCase;

  判断是否相同, equals()

- 转义字符

  \n 表示换行; \" 表示引号; ...

中国科学院大学 University of Chinese Academy of Sciences

Java中现成的常用类——其它

- import 关键字
- java.util.Date
- java.util.Random
- 用户图形界面（GUI）

```
javax.swing.JFrame
javax.swing.JButton
javax.swing.JLable
javax.swing.JTextFiled
...
```

示例L2.3.1



示例L2.3.2



示例L2.3.3



面向对象程序设计方法——初探

- 从“过程”的定义，转变为“实体”的定义

  围绕更容易理解、认知的“实体”进行讨论，而不是仅仅是程序片段

如何将现实系统抽象为程序“实体”？

或者说，如何设计一个类？

中国科学院大学
University of Chinese Academy of Sciences

如何设计一个类?

- 抽象

  定义“实体”的“属性”和“行为”

  这些“属性”和“行为”是为满足某种目的而对“实体”的建模

- 是否是一个好的抽象?

  好的名字？

  属性和行为是否能够完整、准确的满足目标？

  是否有不必要的属性和行为？

中国科学院大学
University of Chinese Academy of Sciences

如何设计一个类？ Cont.

- 分别从“厂家、车主、交通管理机构”三个角度，给出“汽车”的抽象

中国科学院大学
University of Chinese Academy of Sciences

课后作业

- **安装开发环境** Eclipse/IntelliJ IDEA/Visual Studio Code/…

Eclipse， www.eclipse.org/downloads

IntelliJ IDEA， www.jetbrains.com/idea/download

Visual Studio Code， https://code.visualstudio.com/Download

…







课后作业

1. 书山有路勤为径…

以“图书馆借书、还书”为场景，为下面的每个实体设计一个类，并定义相关的属性和方法。

1）大学里的学生；2）书籍；3）大学图书馆的管理员

2. 驾照在手好多年…

分别从“厂家、车主、交通管理机构”3个角度，设计3个“汽车”的类，并定义相关的属性和方法。

作业要求：学习如何利用UML图表示“类”，提交类图文件

- *可使用UML工具绘图（如StarUML、Visio等），提交工具导出的图片*
- *也可使用其它绘图工具（如画图、Word、PowerPoint等），提交相应的文件*
- ***下周四上午9点前提交到个人repository（目录格式：repository_name/Assignment1/）***



中国科学院大学
University of Chinese Academy of Sciences

Thank You!

- 面向对象程序设计
- Object-Oriented Programming